# The sdev Book

*A Complete Guide to the sdev Programming Language*

First Edition · 2026

# Table of Contents

**Table of Contents** 3

**The sdev Book** 32

Preface . . . . . 32

What's new in this edition . . . . . 33

Conventions used in the book . . . . . 33

How to read this book . . . . . 34

**Part I — The Language** 36

**sdev Programming Language** 38

Complete Documentation, Tutorial & Reference Guide 38

Table of Contents . . . . . 38

Introduction . . . . . 39

Getting Started . . . . . 40

Running in the IDE . . . . . 40

Running in the Playground . . . . . 40

Downloading the Desktop App . . . . . 41

Your First Program . . . . . 41

Syntax Overview . . . . . 42

Comments . . . . . 42

Blocks . . . . . 42
No Semicolons Required . . . . . 43
Indentation . . . . . 43
Variables & Constants . . . . . 43
Declaring Variables . . . . . 43
Reassigning Variables . . . . . 44
Compound Assignment (shorthand patterns) . . . . 44
Built-in Constants . . . . . 44
Multiple Variables . . . . . 44
Data Types . . . . . 45
Numbers . . . . . 45
Text (Strings) . . . . . 46
Truth (Booleans) . . . . . 46
Void (Null) . . . . . 47
Lists . . . . . 47
Tomes (Dictionaries) . . . . . 47
Operators . . . . . 48
Arithmetic Operators . . . . . 48
Comparison Operators . . . . . 48
Logical Operators . . . . . 49
String & List Concatenation . . . . . 49
Pipe Operator |> . . . . . 49
Ternary Operator ~ . . . . . 50

Operator Precedence (high to low) . . . . . 50
String Operations . . . . . 51
Control Flow . . . . . 52
If / Else — ponder / otherwise . . . . . 52
While Loop — cycle . . . . . 53
For-Each Loop — iterate through . . . . . 53
For-In Loop — within . . . . . 54
Loop Control . . . . . 54
Nested Loops . . . . . 55
Functions . . . . . 55
Basic Functions — conjure . . . . . 55
Return Values — yield . . . . . 55
Default Parameters . . . . . 56
Lambda Functions (Arrow Syntax) . . . . . 56
Closures . . . . . 57
Recursive Functions . . . . . 58
Higher-Order Functions as Parameters . . . . . 58
Variadic-style Functions . . . . . 59
Object-Oriented Programming . . . . . 59
Defining a Class — essence . . . . . 59
Creating Instances — new . . . . . 60
Inheritance — extend . . . . . 60
Multi-level Inheritance . . . . . 62

Properties and Computed Properties . . . . . 63
Error Handling . . . . . 64
Try / Rescue — attempt / rescue . . . . . 64
Nested Error Handling . . . . . 65
Assertion . . . . . 65
Async & Concurrency . . . . . 66
Async Functions . . . . . 66
Await . . . . . 66
Spawn (Concurrent Execution) . . . . . 66
Delay . . . . . 67
Built-in Output Functions . . . . . 67
Built-in Math Functions . . . . . 68
Core Math . . . . . 68
Trigonometry . . . . . 68
Logarithms & Exponentials . . . . . 69
Advanced Math . . . . . 69
Random . . . . . 69
Built-in String Functions . . . . . 70
Built-in List Functions . . . . . 71
Built-in Tome (Dict) Functions . . . . . 72
Input / Output . . . . . 72
input(prompt?) — Read User Input . . . . . 72
print(...) / println(...) — Print (Aliases) . . . . . 73

Character & Code Point Functions . . . . . 73
Number Base Conversion . . . . . 74
Number Formatting & Checking . . . . . 74
String Checking Functions . . . . . 75
Regex / Pattern Matching . . . . . 75
Bitwise Operations . . . . . 76
Base64 Encoding . . . . . 77
Hash Function . . . . . 77
Time & Date . . . . . 77
Functional Programming . . . . . 78
Buffer / Memory Operations . . . . . 79
- buffer(size) — Create Byte Buffer . . . . . 79
- Buffer Methods . . . . . 80
- pointer(buffer, offset) — Memory Pointer . . . . . 80
- Pointer Methods . . . . . 80

Error Handling & Control Flow . . . . . 81
Additional Aliases . . . . . 81
Higher-Order Functions . . . . . 82
- each — Transform (Map) . . . . . 82
- sift — Filter . . . . . 82
- fold — Reduce . . . . . 82
- seek — Find First . . . . . 83
- every — All Match . . . . . 83

some — Any Match . . . . . 84
enumerate — Index + Value . . . . . 84
zip — Pair Two Lists . . . . . 84
Type System & Conversion . . . . . 84
Getting Type — essence() . . . . . 85
Converting Types — morph() . . . . . 85
Type Checking Helpers . . . . . 85
Collections . . . . . 85
Set . . . . . 85
Map . . . . . 86
Queue (FIFO) . . . . . 87
Stack (LIFO) . . . . . 87
LinkedList . . . . . 88
Matrix Operations . . . . . 89
Creating Matrices . . . . . 89
Matrix Arithmetic . . . . . 89
Vector Operations . . . . . 90
Matrix Utilities . . . . . 90
Neural Network Example . . . . . 91
Graphics & Game Development . . . . . 91
Canvas Setup . . . . . 92
Fill and Stroke . . . . . 92
Basic Shapes . . . . . 92

Text Drawing . . . . . 93
Gradients . . . . . 94
Shadows . . . . . 94
Transformations . . . . . 94
Path Drawing . . . . . 94
Turtle Graphics . . . . . 95
Sprites & Game Objects . . . . . 97
Color Functions . . . . . 97
Graphics Math . . . . . 98
Complete 2D Graphics Example — Starfield . . . . . 98
File I/O . . . . . 99
Reading Files . . . . . 99
Writing Files . . . . . 100
Append to File . . . . . 100
File Utilities . . . . . 100
Networking . . . . . 101
HTTP GET . . . . . 101
HTTP POST . . . . . 101
Examples & Recipes . . . . . 101
FizzBuzz . . . . . 101
Fibonacci (Memoized) . . . . . 102
Binary Search . . . . . 102
Quicksort . . . . . 103

Stack-Based Calculator . . . . . 103
Caesar Cipher . . . . . 104
Word Frequency Counter . . . . . 105
Prime Sieve . . . . . 105
Linked-List Graph Traversal (BFS) . . . . . 106
Turtle Mandala . . . . . 107
JavaScript Interop (JS Interpreter Only) . . . . . 108
Three Syntax Forms . . . . . 108
Basic Examples . . . . . 108
Object Literals . . . . . 109
Arrow Functions . . . . . 109
Multi-line JS Blocks . . . . . 110
Leaflet Map Integration . . . . . 110
Complete Reference Card . . . . . 111
Keywords . . . . . 111
Special Symbols . . . . . 111
Built-in Constants . . . . . 112
Quick Reference Examples . . . . . 112

**Part II — The UI Toolkit 116**

Chapter U1 — Windows in depth . . . . . 116
Sizing . . . . . 116
Multiple windows . . . . . 117
Closing a window programmatically . . . . . 117

Window-level customisation . . . . . 118
Chapter U2 — Buttons, in depth . . . . . 118
Variants . . . . . 118
Customisation tome . . . . . 119
Icon buttons . . . . . 120
Disabled state . . . . . 120
Chapter U3 — Text inputs and forms . . . . . 120
input — single line . . . . . 120
textarea — multi-line . . . . . 121
checkbox . . . . . 121
slider and select . . . . . 122
Chapter U4 — Layout primitives . . . . . 122
Rows and columns . . . . . 122
Groups . . . . . 122
Tabs . . . . . 123
Tables . . . . . 123
Chapter U5 — Reactive state with uiget / uiset . . . 123
Chapter U6 — Customisation reference . . . . . 124
Chapter U7 — Cloud workspace & auto-sync . . . . 126
How auto-sync works . . . . . 126
Manual saves . . . . . 126
Working offline . . . . . 127
Nested folders . . . . . 127

**Widget Reference Cards** **129**

audio . . . . . 129
button . . . . . 129
canvas . . . . . 130
checkbox . . . . . 130
column . . . . . 130
confirm . . . . . 131
divider . . . . . 131
endwindow . . . . . 131
group . . . . . 132
heading . . . . . 132
image . . . . . 132
input . . . . . 133
label . . . . . 133
map . . . . . 133
menu . . . . . 134
paragraph . . . . . 134
progress . . . . . 135
prompt . . . . . 135
radio . . . . . 135
row . . . . . 136
select . . . . . 136

slider . . . . . 136
spacer . . . . . 137
switch . . . . . 137
table . . . . . 137
tabs . . . . . 138
textarea . . . . . 138
toast . . . . . 138
uiget . . . . . 139
uiset . . . . . 139
video . . . . . 139
window . . . . . 140

**Part III — The Cookbook 142**

Tutorial 1 — Hello, Window . . . . . 142
Step 1 — The empty window . . . . . 142
Step 2 — Add some content . . . . . 142
Step 3 — A button that does something . . . . . 143
Step 4 — Show the count inside the window . . . . 143
What you learned . . . . . 143
Tutorial 2 — A Tip Calculator . . . . . 144
Stretch goal . . . . . 144
Tutorial 3 — A To-Do List . . . . . 144
What you learned . . . . . 146

Tutorial 4 — A Note-Taking App . . . . . 146
Tutorial 5 — A Drawing Pad . . . . . 147
Tutorial 6 — A Pomodoro Timer . . . . . 148
Tutorial 7 — A Markdown Previewer . . . . . 149
Tutorial 8 — A Weather Dashboard . . . . . 150
Tutorial 9 — A Map Explorer . . . . . 151
Tutorial 10 — A Mini Spreadsheet . . . . . 152
Tutorial 11 — A Chat Window . . . . . 153
Tutorial 12 — A File Manager . . . . . 154
Tutorial 13 — A Calculator . . . . . 155
Tutorial 14 — A Bouncing Ball Game . . . . . 156
Tutorial 15 — Your First Library . . . . . 156
Step 1 — Write the library . . . . . 157
Step 2 — Publish as a public Gist . . . . . 157
Step 3 — Use it from another program . . . . . 157
Where to go next . . . . . 157

**Appendix A — Maps & GIS (Leaflet) 159**

**sdev Leaflet Documentation 161**

Interactive Maps for sdev . . . . . 161
Table of Contents . . . . . 161
Getting Started . . . . . 162
Basic Setup . . . . . 162

HTML Setup . . . . . 162
Map Creation . . . . . 163
createMap(containerId, lat, lng, zoom) . . . . . 163
setMapView(map, lat, lng, zoom) . . . . . 163
getMapCenter(map) . . . . . 163
getMapZoom(map) . . . . . 164
getMapBounds(map) . . . . . 164
Markers & Popups . . . . . 164
addMarker(map, lat, lng, popupText?) . . . . . 164
addMarkerIcon(map, lat, lng, iconUrl, iconSize, popupText?) . . . . . 165
removeMarker(map, marker) . . . . . 165
setMarkerPosition(marker, lat, lng) . . . . . 165
bindPopup(marker, content) . . . . . 166
bindTooltip(marker, content) . . . . . 166
openPopup(marker) . . . . . 166
Shapes . . . . . 167
addCircle(map, lat, lng, radius, options?) . . . . . 167
addCircleMarker(map, lat, lng, radius, options?) . . 167
addRectangle(map, lat1, lng1, lat2, lng2, options?) 168
Polylines & Polygons . . . . . 168
addPolyline(map, points, options?) . . . . . 168
addPolygon(map, points, options?) . . . . . 169
addMultiPolygon(map, polygons, options?) . . . . . 169

Layers . . . . . 170
addTileLayer(map, urlTemplate, options?) . . . . . 170
createLayerGroup() . . . . . 170
addLayerToMap(map, layer) . . . . . 170
removeLayerFromMap(map, layer) . . . . . 171
clearLayer(layer) . . . . . 171
Events . . . . . 171
onMapClick(map, callback) . . . . . 171
onMapZoom(map, callback) . . . . . 172
onMapMove(map, callback) . . . . . 172
onMarkerClick(marker, callback) . . . . . 172
onMarkerDrag(marker, callback) . . . . . 172
Controls . . . . . 173
addZoomControl(map, position?) . . . . . 173
addScaleControl(map, options?) . . . . . 173
addAttributionControl(map, prefix?) . . . . . 173
addLayerControl(map, baseLayers, overlays) . . . . 174
GeoJSON . . . . . 174
addGeoJSON(map, geoJsonData, options?) . . . . . 174
geoJSONStyle(feature) . . . . . 175
Utilities . . . . . 176
latLng(lat, lng) . . . . . 176
distance(lat1, lng1, lat2, lng2) . . . . . 176

boundsContains(bounds, lat, lng) . . . . . 176
fitBounds(map, lat1, lng1, lat2, lng2) . . . . . 176
invalidateSize(map) . . . . . 177
Navigation & Animation . . . . . 177
panTo(map, lat, lng, options?) . . . . . 177
panBy(map, x, y) . . . . . 177
flyTo(map, lat, lng, zoom, options?) . . . . . 177
flyToBounds(map, lat1, lng1, lat2, lng2, options?) . 177
zoomIn(map, delta?) . . . . . 178
zoomOut(map, delta?) . . . . . 178
setZoom(map, zoom) . . . . . 178
setMinZoom(map, zoom) / setMaxZoom(map, zoom) 178
getMinZoom(map) / getMaxZoom(map) . . . . . 179
setMaxBounds(map, lat1, lng1, lat2, lng2) . . . . . 179
Popup & Tooltip Control . . . . . 179
closePopup(marker) . . . . . 179
openTooltip(marker) / closeTooltip(marker) . . . . 179
setPopupContent(marker, content) . . . . . 180
setTooltipContent(marker, content) . . . . . 180
Layer Styling . . . . . 180
setMarkerIcon(marker, iconUrl, iconSize) . . . . . 180
setMarkerOpacity(marker, opacity) . . . . . 180
setMarkerZIndex(marker, zIndex) . . . . . 180

setCircleRadius(circle, radius) . . . . . 181
setCircleStyle(circle, options) / setPolylineStyle / setPolygonStyle . . . . . 181
getPolylineLatLngs(polyline) . . . . . 181
setPolylineLatLngs(polyline, points) . . . . . 181
addLatLng(polyline, lat, lng) . . . . . 182
bringToFront(layer) / bringToBack(layer) . . . . . 182
Additional Events . . . . . 182
onLayerClick(layer, callback) . . . . . 182
onLayerMouseover(layer, callback) / onLayerMouseout(layer, callback) . . . . . 182
onMapDoubleClick(map, callback) . . . . . 183
onMapRightClick(map, callback) . . . . . 183
onMapMousemove(map, callback) . . . . . 183
Geolocation . . . . . 183
locate(map, options?) . . . . . 184
onLocationFound(map, callback) . . . . . 184
onLocationError(map, callback) . . . . . 184
stopLocate(map) . . . . . 184
Overlays . . . . . 184
addImageOverlay(map, imageUrl, lat1, lng1, lat2, lng2, options?) . . . . . 185
addVideoOverlay(map, videoUrl, lat1, lng1, lat2, lng2, options?) . . . . . 185
setImageOpacity(overlay, opacity) . . . . . 185

setImageUrl(overlay, url) . . . . . 185
setBounds(overlay, lat1, lng1, lat2, lng2) . . . . . 185
Feature Groups . . . . . 186
createFeatureGroup() . . . . . 186
addToFeatureGroup(featureGroup, layer) . . . . . 186
removeFromFeatureGroup(featureGroup, layer) . . 186
getFeatureGroupBounds(featureGroup) . . . . . 186
fitFeatureGroup(map, featureGroup, options?) . . . 187
eachLayer(layerGroup, callback) . . . . . 187
getLayers(layerGroup) . . . . . 187
hasLayer(layerGroup, layer) . . . . . 187
Custom Markers . . . . . 188
addDivIcon(map, lat, lng, html, className, size) . . 188
Coordinate Utilities . . . . . 188
getSize(map) . . . . . 188
latLngToContainerPoint(map, lat, lng) . . . . . 188
containerPointToLatLng(map, x, y) . . . . . 188
wrapLng(lng) . . . . . 189
wrapLat(lat) . . . . . 189
degreesToDMS(degrees) . . . . . 189
DMSToDegrees(d, m, s) . . . . . 189
metersToPixels(map, meters, lat) . . . . . 189
pixelsToMeters(map, pixels, lat) . . . . . 190

GIS Analysis Functions . . . . . 190
bearing(lat1, lng1, lat2, lng2) . . . . . 190
midpoint(lat1, lng1, lat2, lng2) . . . . . 190
destination(lat, lng, bearing, distance) . . . . . 190
area(points) . . . . . 191
length(points) . . . . . 191
centroid(points) . . . . . 191
isPointInPolygon(lat, lng, points) . . . . . 191
simplify(points, tolerance) . . . . . 192
interpolateAlong(points, fraction) . . . . . 192
Layer Visibility . . . . . 192
getLayerType(layer) . . . . . 192
isLayerVisible(layer) . . . . . 192
showLayer(map, layer) / hideLayer(map, layer) . . 193
toggleLayer(map, layer) . . . . . 193
Complete Examples . . . . . 193
Interactive City Markers . . . . . 193
Route Visualization . . . . . 194
Heatmap Zones . . . . . 195
Layer Toggle System . . . . . 196
Animated Marker . . . . . 197
Style Reference . . . . . 198
Common Style Options . . . . . 198

Icon Options . . . . . 198
Tips & Best Practices . . . . . 198
Error Handling . . . . . 199

**Appendix B — Glossary 201**

**Appendix C — Index of tutorials 204**

Colophon . . . . . 205

**Appendix F — Style guide 207**

1. Use forge for state . . . . . 207
2. Name widgets . . . . . 207
3. Group with group() . . . . . 207
4. Prefer ghost over destructive . . . . . 207
5. Cap line length . . . . . 207
6. One window per file . . . . . 207
7. Avoid eval . . . . . 208
8. Comment intent . . . . . 208
9. Use after for animation . . . . . 208
10. Cache uiget . . . . . 208
11. Keep functions small . . . . . 208
12. Lift state up . . . . . 208
13. Tabs over deep nesting . . . . . 208
14. Test in both runtimes . . . . . 209
15. Document with paragraph() . . . . . 209

**sdev Programming Language 211**

Complete Documentation, Tutorial & Reference Guide 211

Table of Contents . . . . . 211

Introduction . . . . . 212

Getting Started . . . . . 213

Running in the IDE . . . . . 213

Running in the Playground . . . . . 213

Downloading the Desktop App . . . . . 214

Your First Program . . . . . 214

Syntax Overview . . . . . 215

Comments . . . . . 215

Blocks . . . . . 215

No Semicolons Required . . . . . 216

Indentation . . . . . 216

Variables & Constants . . . . . 216

Declaring Variables . . . . . 216

Reassigning Variables . . . . . 217

Compound Assignment (shorthand patterns) . . . . 217

Built-in Constants . . . . . 217

Multiple Variables . . . . . 217

Data Types . . . . . 218

Numbers . . . . . 218

Text (Strings) . . . . . 219

Truth (Booleans) . . . . . 219
Void (Null) . . . . . 220
Lists . . . . . 220
Tomes (Dictionaries) . . . . . 220
Operators . . . . . 221
Arithmetic Operators . . . . . 221
Comparison Operators . . . . . 221
Logical Operators . . . . . 222
String & List Concatenation . . . . . 222
Pipe Operator |> . . . . . 222
Ternary Operator ~ . . . . . 223
Operator Precedence (high to low) . . . . . 223
String Operations . . . . . 224
Control Flow . . . . . 225
If / Else — ponder / otherwise . . . . . 225
While Loop — cycle . . . . . 226
For-Each Loop — iterate through . . . . . 226
For-In Loop — within . . . . . 227
Loop Control . . . . . 227
Nested Loops . . . . . 228
Functions . . . . . 228
Basic Functions — conjure . . . . . 228
Return Values — yield . . . . . 228

Default Parameters . . . . . 229
Lambda Functions (Arrow Syntax) . . . . . 229
Closures . . . . . 230
Recursive Functions . . . . . 231
Higher-Order Functions as Parameters . . . . . 231
Variadic-style Functions . . . . . 232
Object-Oriented Programming . . . . . 232
Defining a Class — essence . . . . . 232
Creating Instances — new . . . . . 233
Inheritance — extend . . . . . 233
Multi-level Inheritance . . . . . 235
Properties and Computed Properties . . . . . 236
Error Handling . . . . . 237
Try / Rescue — attempt / rescue . . . . . 237
Nested Error Handling . . . . . 238
Assertion . . . . . 238
Async & Concurrency . . . . . 239
Async Functions . . . . . 239
Await . . . . . 239
Spawn (Concurrent Execution) . . . . . 239
Delay . . . . . 240
Built-in Output Functions . . . . . 240
Built-in Math Functions . . . . . 241

Core Math . . . . . 241
Trigonometry . . . . . 241
Logarithms & Exponentials . . . . . 242
Advanced Math . . . . . 242
Random . . . . . 242
Built-in String Functions . . . . . 243
Built-in List Functions . . . . . 244
Built-in Tome (Dict) Functions . . . . . 245
Input / Output . . . . . 245
input(prompt?) — Read User Input . . . . . 245
print(...) / println(...) — Print (Aliases) . . . . . 246
Character & Code Point Functions . . . . . 246
Number Base Conversion . . . . . 247
Number Formatting & Checking . . . . . 247
String Checking Functions . . . . . 248
Regex / Pattern Matching . . . . . 248
Bitwise Operations . . . . . 249
Base64 Encoding . . . . . 250
Hash Function . . . . . 250
Time & Date . . . . . 250
Functional Programming . . . . . 251
Buffer / Memory Operations . . . . . 252
buffer(size) — Create Byte Buffer . . . . . 252

Buffer Methods . . . . . 253
pointer(buffer, offset) — Memory Pointer . . . . . 253
Pointer Methods . . . . . 253
Error Handling & Control Flow . . . . . 254
Additional Aliases . . . . . 254
Higher-Order Functions . . . . . 255
each — Transform (Map) . . . . . 255
sift — Filter . . . . . 255
fold — Reduce . . . . . 255
seek — Find First . . . . . 256
every — All Match . . . . . 256
some — Any Match . . . . . 257
enumerate — Index + Value . . . . . 257
zip — Pair Two Lists . . . . . 257
Type System & Conversion . . . . . 257
Getting Type — essence() . . . . . 258
Converting Types — morph() . . . . . 258
Type Checking Helpers . . . . . 258
Collections . . . . . 258
Set . . . . . 258
Map . . . . . 259
Queue (FIFO) . . . . . 260
Stack (LIFO) . . . . . 260

LinkedList . . . . . 261
Matrix Operations . . . . . 262
Creating Matrices . . . . . 262
Matrix Arithmetic . . . . . 262
Vector Operations . . . . . 263
Matrix Utilities . . . . . 263
Neural Network Example . . . . . 264
Graphics & Game Development . . . . . 264
Canvas Setup . . . . . 265
Fill and Stroke . . . . . 265
Basic Shapes . . . . . 265
Text Drawing . . . . . 266
Gradients . . . . . 267
Shadows . . . . . 267
Transformations . . . . . 267
Path Drawing . . . . . 267
Turtle Graphics . . . . . 268
Sprites & Game Objects . . . . . 270
Color Functions . . . . . 270
Graphics Math . . . . . 271
Complete 2D Graphics Example — Starfield . . . . 271
File I/O . . . . . 272
Reading Files . . . . . 272

Writing Files . . . . . 273
Append to File . . . . . 273
File Utilities . . . . . 273
Networking . . . . . 274
HTTP GET . . . . . 274
HTTP POST . . . . . 274
Examples & Recipes . . . . . 274
FizzBuzz . . . . . 274
Fibonacci (Memoized) . . . . . 275
Binary Search . . . . . 275
Quicksort . . . . . 276
Stack-Based Calculator . . . . . 276
Caesar Cipher . . . . . 277
Word Frequency Counter . . . . . 278
Prime Sieve . . . . . 278
Linked-List Graph Traversal (BFS) . . . . . 279
Turtle Mandala . . . . . 280
JavaScript Interop (JS Interpreter Only) . . . . . 281
Three Syntax Forms . . . . . 281
Basic Examples . . . . . 281
Object Literals . . . . . 282
Arrow Functions . . . . . 282
Multi-line JS Blocks . . . . . 283

Leaflet Map Integration . . . . . 283
Complete Reference Card . . . . . 284
Keywords . . . . . 284
Special Symbols . . . . . 284
Built-in Constants . . . . . 285
Quick Reference Examples . . . . . 285

CHAPTER 01

# The sdev Book

# The sdev Book

_The complete guide to the sdev programming language — second edition._

> *"A language is a craft. A book is a workshop." — sdev team*

## Preface

This is the **second, expanded edition** of the sdev book. It exists because the language has grown well past what the first edition could carry: a full UI toolkit, a cloud workspace, nested project folders, an autosave engine, a kernel with cooperative tasks, a bytecode VM, a fuzzy code translator, and a Leaflet-powered mapping system are now first-class citizens.

The book is organised in three movements:

1. **The Language** — the original sdev reference, polished and lightly

updated. If you're new, start here.

2. **The Toolkit** — a deep dive into every UI widget, with full

customisation tables, layout patterns, and recipes.

3. **The Cookbook** — fifteen step-by-step tutorials that build real

applications from blank canvas to working program.

A complete Leaflet/GIS reference closes the book as Appendix A. A glossary and index sit at the back so you can dip in by keyword.

## What's new in this edition

| Area | Addition | |------|----------| | UI | `window` / `endwindow`, full widget set, layout primitives, menus, tabs, tables | | UI | Per-widget customisation: variants, sizes, colors, padding, radius | | State | `uiget` / `uiset` reactive bindings | | Cloud | Auto-sync of the entire workspace including nested folders | | Cloud | Conflict resolution, offline mode, manual `Save to cloud` action | | Storage | Nested folders with full tree, drag & drop, rename, delete | | Tooling | Standalone JavaScript runtime (`sdev-interpreter.js`) | | Tooling | Standalone Python runtime (`sdev-interpreter.py`) | | Tooling | Web IDE with command palette, search, settings, terminal | | Docs | This book — both English and Bulgarian editions |

## Conventions used in the book

- `monospace` is used for code, file names, and identifiers.
- **Bold** marks the first appearance of a new term.
- A box like the one below highlights a tip:

> ***Tip.*** *Press* ***Ctrl + Enter*** *in the IDE to run the current file.*

A box like this flags a pitfall:

> ***Watch out.*** *A* `window(...)` *block must always be closed with* `endwindow`*,*
>
> *otherwise widgets that follow will silently render into the previous window.*

## How to read this book

If you have programmed before, skim Part I and jump to Part II. If sdev is your first language, work through Part I in order — every chapter builds on the previous one. The tutorials in Part III are designed to be typed by hand; resist the urge to copy-paste, the muscle memory is half the lesson.

CHAPTER 02

# Part I — The Language

# Part I — The Language

This part is the canonical sdev reference. It is comprehensive: every keyword, every built-in, every operator. If you only ever read one part of the book, read this one.

CHAPTER 03

# sdev Programming Language

# sdev Programming Language

## Complete Documentation, Tutorial & Reference Guide

## Table of Contents

1. Introduction
2. Getting Started
3. Syntax Overview
4. Variables & Constants
5. Data Types
6. Operators
7. String Operations
8. Control Flow
9. Functions
10. Object-Oriented Programming
11. Error Handling
12. Async & Concurrency
13. Built-in Output Functions
14. Built-in Math Functions

15. Built-in String Functions
16. Built-in List Functions
17. Built-in Tome (Dict) Functions
18. Higher-Order Functions
19. Type System & Conversion
20. Collections: Set, Map, Queue, Stack, LinkedList
21. Matrix Operations
22. Graphics & Game Development
23. File I/O
24. Networking
25. Examples & Recipes
26. JavaScript Interop
27. Complete Reference Card

# Introduction

**sdev** is an expressive, full-featured programming language with a deliberately unique syntax designed to feel fresh and readable. It supports:

- **Unique English-inspired keywords** — `forge`, `conjure`, `ponder`, `cycle`, `iterate`, `yield`, `yeet`, `skip`
- **Block delimiters** — `::` to open a block and `;;` to close it — no curly braces
- **First-class functions and lambdas** — `(x) -> x * 2`
- **Pipe operator** — `|>` for clean functional composition

- **Full OOP** — `essence` (classes), `extend` (inheritance), `self`, `super`, `new`
- **Error handling** — `attempt :: ... ;; rescue err :: ... ;;`
- **Built-in data structures** — Set, Map, Queue, Stack, LinkedList
- **Matrix math** — `matmul`, `transpose`, `dot`, `reshape`, etc.
- **2D Graphics & Turtle API** — canvas drawing, shapes, gradients, turtle graphics, sprites
- **Async / concurrency** — `async conjure`, `await`, `spawn`
- **JavaScript interop** — `js` keyword for seamless JS integration

sdev runs in the browser-based IDE (at `/ide`), in the Playground (`/`), and as a downloadable Electron desktop application.

# Getting Started

## Running in the IDE

Open the sdev IDE at `/ide`. Press **Ctrl+Enter** (or the **Run** button) to execute your program. Output appears in the **OUTPUT** panel at the bottom.

## Running in the Playground

Go to the main Playground at `/`. Type or paste sdev code in the editor, then click **Run**.

## Downloading the Desktop App

In the IDE, go to the **Download** button → **Electron Desktop App**. This gives you three files:

```
package.json
main.js
README.md
```

Install and run:

```
npm install
npm start
```

This opens sdev IDE as a native desktop window that wraps the full web IDE.

## Your First Program

```
// hello.sdev
speak("Hello, World!")
```

Output:

```
Hello, World!
```

A slightly more involved first program:

```
forge name be "Alice"
forge age be 30

speak("Name:", name)
```

```
speak("Age:", age)
speak("In 10 years:", age + 10)

conjure greet(person, greeting) ::
  yield greeting + ", " + person + "!"
;;

speak(greet(name, "Welcome"))
```

Output:

```
Name: Alice
Age: 30
In 10 years: 40
Welcome, Alice!
```

# Syntax Overview

## Comments

```
// This is a single-line comment
# This is also a single-line comment (Python style)
```

## Blocks

sdev uses `::` to start a block and `;;` to end it — never curly braces `{}`:

```
ponder x &gt; 5 ::
  speak("x is big")
;;

conjure add(a, b) ::
```

```
    yield a + b
  ;;
```

Blocks can also be written inline for short single-statement bodies:

```
ponder x &gt; 5 :: speak("big") ;;
```

### No Semicolons Required

Statements are separated by newlines. You do NOT need `;` at the end of lines.

```
forge a be 1
forge b be 2
forge c be a + b
speak(c)
```

### Indentation

Indentation is cosmetic but strongly recommended. The language uses `::` and `;;` for block structure, not whitespace.

## Variables & Constants

### Declaring Variables

Use `forge` to declare a new variable and `be` to assign its value:

```
forge name be "Alice"
forge age be 25
forge isActive be yep
forge score be 0.0
```

## Reassigning Variables

Just use `be` without `forge`:

```
forge count be 0
count be count + 1
count be 10
```

## Compound Assignment (shorthand patterns)

```
forge x be 10
x be x + 5    // x = 15
x be x * 2    // x = 30
x be x - 10   // x = 20
x be x / 4    // x = 5
```

## Built-in Constants

sdev provides these constants directly:

```
speak(PI)        // 3.141592653589793
speak(TAU)       // 6.283185307179586 (2 * PI)
speak(E)         // 2.718281828459045
speak(INFINITY)  // Infinity
```

## Multiple Variables

```
forge x be 10
forge y be 20
forge z be x + y

// Swap pattern
forge temp be x
x be y
y be temp
```

# Data Types

sdev has six core types:

| sdev Type | Description | Literal Example |
|----------|----------|----------|
| `number` | Integer or float | `42`, `3.14`, `-7`, `1.5e10` |
| `text` | Strings | `"hello"`, `'world'` |
| `truth` | Booleans | `yep`, `nope` |
| `void` | Null / absent | `void` |
| `list` | Arrays | `[1, 2, 3]` |
| `tome` | Dictionaries | `:: "key": "value" ;;` |

Functions and class instances are also first-class values.

## Numbers

```
forge integer be 42
forge float be 3.14159
forge negative be -100
forge scientific be 1.5e10      // 15,000,000,000
forge hex be 0xFF               // 255
```

Arithmetic is standard, with `^` for power:

```
speak(2 ^ 10)   // 1024
```

```
speak(10 % 3)   // 1
speak(7 / 2)    // 3.5
```

## Text (Strings)

```
forge single be 'Hello'
forge double be "World"

// Escape sequences
forge tab be "col1\tcol2"
forge newline be "line1\nline2"
forge quoted be "She said \"hi\""

// String concatenation
forge full be "Hello" + ", " + "World!"

// String repetition
forge stars be "*" * 10   // "**********"

// Access characters
speak("hello"[0])   // h
speak("hello"[-1])  // o (last character)
```

## Truth (Booleans)

```
forge t be yep    // true
forge f be nope   // false

// Boolean expressions
speak(5 &gt; 3)          // yep
speak(5 equals 3)     // nope
speak(yep also nope)  // nope
speak(yep either nope) // yep
speak(isnt nope)       // yep
```

## Void (Null)

```
forge nothing be void
ponder nothing equals void ::
  speak("It's void!")
;;
```

## Lists

```
forge nums be [1, 2, 3, 4, 5]
forge mixed be [1, "two", yep, void]
forge empty be []
forge nested be [[1, 2], [3, 4], [5, 6]]

// Access by index (0-based)
speak(nums[0])    // 1
speak(nums[2])    // 3
speak(nums[-1])   // 5  (last element)

// Modify element
nums[0] be 100
speak(nums)       // [100, 2, 3, 4, 5]

// List length
speak(measure(nums))   // 5

// Slicing
speak(portion(nums, 1, 3))  // [2, 3]
speak(portion(nums, 2))     // [3, 4, 5]
```

## Tomes (Dictionaries)

Tomes are key-value stores. Keys are strings.

```
forge person be ::
  "name": "Alice",
```

```
  "age": 30,
  "active": yep
;;

// Access values - both bracket and dot notation work
speak(person["name"])  // Alice
speak(person.age)      // 30

// Set new or existing keys
person["city"] be "New York"
person.age be 31

// Check if key exists
speak(contains(person, "name"))   // yep
speak(contains(person, "phone"))  // nope

// Get all keys, values, entries
speak(inscriptions(person))  // ["name", "age", "active", "city"]
speak(contents(person))      // ["Alice", 31, yep, "New York"]
speak(entries(person))       // [["name","Alice"], ...]
```

# Operators

## Arithmetic Operators

| Operator | Description | Example | Result |
|---|---|---|---|
| `+` | Addition | `5 + 3` | `8` |
| `-` | Subtraction | `10 - 4` | `6` |
| `*` | Multiplication | `4 * 3` | `12` |
| `/` | Division | `7 / 2` | `3.5` |
| `%` | Modulo | `10 % 3` | `1` |
| `^` | Power | `2 ^ 8` | `256` |
| `-x` | Negation | `-5` | `-5` |

## Comparison Operators

| Operator | Description | Example | Result |
|---------|----------|---------|---------|
| `equals` | Equal to | `5 equals 5` | `yep` |
| `differs` | Not equal | `5 differs 3` | `yep` |
| `<>` | Not equal (alt) | `5 <> 3` | `yep` |
| `<` | Less than | `3 < 5` | `yep` |
| `>` | Greater than | `5 > 3` | `yep` |
| `<=` | Less or equal | `3 <= 3` | `yep` |
| `>=` | Greater or equal | `5 >= 5` | `yep` |

## Logical Operators

| Operator | Meaning | Example |
|---------|---------|---------|
| `also` | AND | `yep also yep` → `yep` |
| `either` | OR | `nope either yep` → `yep` |
| `isnt` | NOT | `isnt nope` → `yep` |

Short-circuit evaluation applies: `also` stops at first `nope`, `either` stops at first `yep`.

## String & List Concatenation

```
"Hello" + " World"    // "Hello World"
[1, 2] + [3, 4]       // [1, 2, 3, 4]
"ha" * 3              // "hahaha"
[0] * 5               // [0, 0, 0, 0, 0]
```

## Pipe Operator `|>`

The pipe operator passes the left-hand value as the first argument to the right-hand function:

```
// Without pipe
forge result be weave(sort(sift(each([1,2,3,4,5], x -&gt; x * 2), x

// With pipe — much cleaner!
```

```
forge result be [1, 2, 3, 4, 5]
  |&gt; each(x -&gt; x * 2)
  |&gt; sift(x -&gt; x &gt; 4)
  |&gt; weave(", ")
speak(result)  // "6, 8, 10"
```

## Ternary Operator ~

Inline if/else expression:

```
forge x be 10
forge label be x &gt; 5 ~ "big" : "small"
speak(label)  // "big"

// In function calls
speak(magnitude(-5) &gt; 3 ~ "large magnitude" : "small magnitude"

// Nested ternary
forge grade be 85
forge letter be grade &gt;= 90 ~ "A" : grade &gt;= 80 ~ "B" : grad
speak(letter)  // "B"
```

## Operator Precedence (high to low)

1. `^` (power)
2. Unary `-`, `isnt`
3. `*`, `/`, `%`
4. `+`, `-`
5. `<`, `>`, `<=`, `>=`
6. `equals`, `differs`, `<>`
7. `also`
8. `either`
9. `~` (ternary)

10. |> (pipe)

# String Operations

sdev has a rich string library. All string functions return new strings — strings are immutable.

```
forge s be "  Hello, World!  "

// Case conversion
speak(upper(s))          // "  HELLO, WORLD!  "
speak(lower(s))          // "  hello, world!  "

// Trimming
speak(trim(s))           // "Hello, World!"

// Length
speak(measure(s))        // 18

// Contains / starts / ends
speak(contains(s, "World"))      // yep
speak(startswith(trim(s), "Hello"))  // yep
speak(endswith(trim(s), "!"))        // yep

// Replace (replaces ALL occurrences)
speak(replace("banana", "a", "o"))   // "bonono"

// Find index
speak(locate("hello world", "world"))  // 6
speak(locate("hello", "xyz"))          // -1

// Split and join
forge parts be shatter("a,b,c,d", ",")
speak(parts)                 // ["a", "b", "c", "d"]
speak(weave(parts, " | "))   // "a | b | c | d"
```

```
// Character list
speak(chars("abc"))   // ["a", "b", "c"]

// Reverse
speak(reverse("hello"))  // "olleh"

// Padding
speak(padLeft("5", 4, "0"))    // "0005"
speak(padRight("hi", 6, "!"))  // "hi!!!!"

// Format strings (use {} as placeholders)
speak(format("Hello, {}! You are {} years old.", "Bob", 25))
// "Hello, Bob! You are 25 years old."

// Repeat
speak(repeat("ab", 3))   // "ababab"

// Substring
speak(snatch("hello world", 6, 11))  // "world"
```

# Control Flow

## If / Else — `ponder` / `otherwise`

```
forge score be 87

ponder score &gt;= 90 ::
  speak("Grade: A")
;; otherwise ponder score &gt;= 80 ::
  speak("Grade: B")
;; otherwise ponder score &gt;= 70 ::
  speak("Grade: C")
;; otherwise ponder score &gt;= 60 ::
  speak("Grade: D")
;; otherwise ::
```

```
  speak("Grade: F")
;;
```

Inline (for short bodies):

```
ponder x &gt; 0 :: speak("positive") ;; otherwise :: speak("non-pos
```

## While Loop — `cycle`

```
forge n be 1
cycle n &lt;= 10 ::
  speak(n)
  n be n + 1
;;

// Infinite loop with break
cycle yep ::
  forge x be randint(1, 10)
  speak("Got:", x)
  ponder x equals 7 ::
    speak("Found 7! Stopping.")
    yeet    // break
  ;;
;;
```

## For-Each Loop — `iterate through`

```
forge fruits be ["apple", "banana", "cherry"]

iterate fruit through fruits ::
  speak(fruit)
;;
// apple
// banana
// cherry
```

## For-In Loop — `within`

```
// Iterate over a list
within fruit be ["apple", "banana", "cherry"] ::
  speak(fruit)
;;

// Iterate over a range
within i be sequence(5) ::
  speak(i)   // 0 1 2 3 4
;;

within i be sequence(1, 11) ::
  speak(i)   // 1 2 3 ... 10
;;

within i be sequence(0, 20, 5) ::
  speak(i)   // 0 5 10 15
;;
```

## Loop Control

### `yeet` — Break

Exit the nearest enclosing loop immediately:

```
within i be sequence(100) ::
  ponder i &gt; 5 ::
    yeet
  ;;
  speak(i)
;;
// 0 1 2 3 4 5
```

### `skip` — Continue

Skip the rest of the current iteration and go to the next:

```
within i be sequence(10) ::
  ponder i % 2 equals 0 ::
    skip   // skip even numbers
  ;;
  speak(i)   // prints only odd: 1 3 5 7 9
;;
```

### Nested Loops

```
within i be sequence(1, 4) ::
  within j be sequence(1, 4) ::
    speak(i + " x " + j + " = " + morph(i * j, "text"))
  ;;
;;
```

# Functions

### Basic Functions — `conjure`

```
conjure greet(name) ::
  speak("Hello, " + name + "!")
;;

greet("Alice")
greet("World")
```

### Return Values — `yield`

```
conjure add(a, b) ::
  yield a + b
```

```
;;

conjure max(a, b) ::
  ponder a &gt; b :: yield a ;;
  yield b
;;

forge sum be add(10, 20)
speak(sum)         // 30
speak(max(5, 9))   // 9
```

## Default Parameters

```
conjure greet(name, greeting) ::
  ponder greeting equals void :: greeting be "Hello" ;;
  yield greeting + ", " + name + "!"
;;

speak(greet("Alice"))           // Hello, Alice!
speak(greet("Bob", "Welcome"))  // Welcome, Bob!
```

## Lambda Functions (Arrow Syntax)

Single expression, one parameter:

```
forge double be x -&gt; x * 2
forge square be x -&gt; x ^ 2
forge negate be x -&gt; -x

speak(double(5))   // 10
speak(square(4))   // 16
speak(negate(7))   // -7
```

Multiple parameters (parentheses required):

```
forge add be (a, b) -&gt; a + b
```

```
forge clamp be (v, lo, hi) -&gt; v &lt; lo ~ lo : v &gt; hi ~ hi :

speak(add(3, 4))           // 7
speak(clamp(150, 0, 100))  // 100
```

Multi-statement lambda body with `::` and `;;`:

```
forge process be x -&gt; ::
  forge doubled be x * 2
  forge result be doubled + 10
  yield result
;;

speak(process(5))   // 20
```

## Closures

Functions capture variables from their enclosing scope:

```
conjure makeCounter(start) ::
  forge count be start

  conjure increment() ::
    count be count + 1
    yield count
  ;;

  conjure reset() ::
    count be start
  ;;

  yield :: "next": increment, "reset": reset ;;
;;

forge c be makeCounter(0)
speak(c.next())   // 1
speak(c.next())   // 2
```

```
speak(c.next())   // 3
c.reset()
speak(c.next())   // 1
```

## Recursive Functions

```
conjure factorial(n) ::
  ponder n &lt;= 1 :: yield 1 ;;
  yield n * factorial(n - 1)
;;

speak(factorial(10))   // 3628800

// Mutual recursion
conjure isEven(n) ::
  ponder n equals 0 :: yield yep ;;
  yield isOdd(n - 1)
;;

conjure isOdd(n) ::
  ponder n equals 0 :: yield nope ;;
  yield isEven(n - 1)
;;

speak(isEven(4))   // yep
speak(isOdd(7))    // yep
```

## Higher-Order Functions as Parameters

```
conjure applyTwice(f, x) ::
  yield f(f(x))
;;

forge double be x -&gt; x * 2
speak(applyTwice(double, 3))   // 12

conjure compose(f, g) ::
```

```
  yield x -&gt; f(g(x))
;;

forge addOne be x -&gt; x + 1
forge triple be x -&gt; x * 3
forge addOneThenTriple be compose(triple, addOne)
speak(addOneThenTriple(4))   // 15
```

### Variadic-style Functions

Use lists to simulate variadic arguments:

```
conjure sumAll(nums) ::
  yield fold(nums, 0, (acc, x) -&gt; acc + x)
;;

speak(sumAll([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))   // 55
```

## Object-Oriented Programming

sdev has full class-based OOP using `essence` for class definitions, `new` for instantiation, `self` for the instance reference, `super` for parent class access, and `extend` for inheritance.

### Defining a Class — `essence`

```
essence Person ::
  conjure init(self, name, age) ::
    self.name be name
```

```
    self.age be age
  ;;

  conjure greet(self) ::
    speak("Hi, I'm " + self.name + " and I'm " + morph(self.age, "
  ;;

  conjure birthday(self) ::
    self.age be self.age + 1
    speak(self.name + " is now " + morph(self.age, "text") + "!")
  ;;

  conjure toString(self) ::
    yield "Person(" + self.name + ", " + morph(self.age, "text") +
  ;;
;;
```

## Creating Instances — `new`

```
forge alice be new Person("Alice", 30)
forge bob be new Person("Bob", 25)

alice.greet()   // Hi, I'm Alice and I'm 30 years old.
bob.greet()     // Hi, I'm Bob and I'm 25 years old.

alice.birthday()   // Alice is now 31!
speak(alice.age)   // 31

speak(alice.toString())  // Person(Alice, 31)
```

## Inheritance — `extend`

```
essence Animal ::
  conjure init(self, name, sound) ::
    self.name be name
    self.sound be sound
    self.energy be 100
```

```
    ;;

    conjure makeSound(self) ::
      speak(self.name + " says: " + self.sound)
    ;;

    conjure eat(self, food) ::
      self.energy be self.energy + 20
      speak(self.name + " eats " + food)
    ;;

    conjure status(self) ::
      speak(self.name + " | Energy: " + morph(self.energy, "text"))
    ;;
  ;;

  essence Dog extend Animal ::
    conjure init(self, name, breed) ::
      super.init(name, "Woof!")
      self.breed be breed
      self.tricks be []
    ;;

    conjure makeSound(self) ::
      speak(self.name + " barks: WOOF WOOF!")   // Override parent
    ;;

    conjure learnTrick(self, trick) ::
      gather(self.tricks, trick)
      speak(self.name + " learned: " + trick)
    ;;

    conjure showTricks(self) ::
      ponder measure(self.tricks) equals 0 ::
        speak(self.name + " knows no tricks.")
        yield void
      ;;
      speak(self.name + " can do: " + weave(self.tricks, ", "))
    ;;
```

```
;;

forge rex be new Dog("Rex", "German Shepherd")
rex.makeSound()                // Rex barks: WOOF WOOF!
rex.eat("kibble")              // Rex eats kibble
rex.learnTrick("sit")
rex.learnTrick("shake")
rex.learnTrick("roll over")
rex.showTricks()               // Rex can do: sit, shake, roll ove
rex.status()                   // Rex | Energy: 120
```

## Multi-level Inheritance

```
essence Vehicle ::
  conjure init(self, brand, speed) ::
    self.brand be brand
    self.speed be speed
    self.fuel be 100
  ;;

  conjure describe(self) ::
    yield self.brand + " going " + morph(self.speed, "text") + " kn
  ;;
;;

essence Car extend Vehicle ::
  conjure init(self, brand, speed, doors) ::
    super.init(brand, speed)
    self.doors be doors
  ;;

  conjure honk(self) ::
    speak(self.brand + ": Beep beep!")
  ;;
;;

essence ElectricCar extend Car ::
  conjure init(self, brand, speed, doors, range) ::
```

```
    super.init(brand, speed, doors)
    self.range be range
    self.battery be 100
  ;;

  conjure charge(self) ::
    self.battery be 100
    speak(self.brand + " fully charged!")
  ;;

  conjure describe(self) ::
    yield super.describe() + " (Electric, " + morph(self.range, "to
  ;;
;;

forge tesla be new ElectricCar("Tesla Model 3", 250, 4, 500)
speak(tesla.describe())   // Tesla Model 3 going 250 km/h (Electric
tesla.honk()              // Tesla Model 3: Beep beep!
tesla.charge()            // Tesla Model 3 fully charged!
```

## Properties and Computed Properties

```
essence Circle ::
  conjure init(self, radius) ::
    self.radius be radius
  ;;

  conjure area(self) ::
    yield PI * self.radius ^ 2
  ;;

  conjure circumference(self) ::
    yield 2 * PI * self.radius
  ;;

  conjure scale(self, factor) ::
    self.radius be self.radius * factor
    yield self
```

```
  ;;

  conjure toString(self) ::
    yield "Circle(r=" + morph(nearby(self.radius * 100) / 100, "tex
  ;;
;;

forge c be new Circle(5)
speak(nearby(c.area()))           // 79
speak(nearby(c.circumference()))  // 31

c.scale(2)
speak(c.toString())   // Circle(r=10)
```

# Error Handling

## Try / Rescue — `attempt` / `rescue`

Use `attempt` to try code that might fail, and `rescue` to catch errors:

```
attempt ::
  forge result be 10 / 0
  speak(result)
;; rescue err ::
  speak("Caught an error: " + err)
;;

attempt ::
  forge nums be [1, 2, 3]
  speak(nums[10])   // index out of bounds
;; rescue err ::
  speak("Error accessing list:", err)
;;
```

## Nested Error Handling

```
conjure safeDivide(a, b) ::
  attempt ::
    ponder b equals 0 ::
      speak("Error: division by zero")
      yield void
    ;;
    yield a / b
  ;; rescue err ::
    speak("Unexpected error:", err)
    yield void
  ;;
;;

speak(safeDivide(10, 2))   // 5
speak(safeDivide(10, 0))   // Error: division by zero
```

## Assertion

```
conjure validateAge(age) ::
  attempt ::
    ponder age &lt; 0 ::
      speak("Error: Age cannot be negative")
      yield nope
    ;;
    ponder age &gt; 150 ::
      speak("Error: Age is unreasonably large")
      yield nope
    ;;
    yield yep
  ;; rescue err ::
    speak("Validation error:", err)
    yield nope
  ;;
;;
```

# Async & Concurrency

## Async Functions

Use `async conjure` to define an asynchronous function:

```
async conjure fetchData(url) ::
  forge response be await fetch(url)
  yield response
;;
```

## Await

Use `await` inside `async` functions to wait for asynchronous operations:

```
async conjure loadUserProfile(id) ::
  forge user be await fetchData("https://api.example.com/users/" +
  forge posts be await fetchData("https://api.example.com/posts?us

  yield ::
    "user": user,
    "posts": posts
  ;;
;;
```

## Spawn (Concurrent Execution)

Use `spawn` to run functions concurrently without waiting for them:

```
conjure worker(name, ms) ::
  delay(ms)
  speak(name + " finished after " + morph(ms, "text") + "ms")
;;

spawn worker("Task A", 2000)
spawn worker("Task B", 500)
spawn worker("Task C", 1000)
// Task B completes first, then C, then A
```

### Delay

Pause execution for a number of milliseconds:

```
speak("Starting...")
delay(1000)
speak("One second later...")
delay(2000)
speak("Three seconds total")
```

## Built-in Output Functions

| Function | Description | Example |
|---------|----------|---------| | `speak(...)` | Print values with spaces between them | `speak("x =", 42)` → `x = 42` | | `whisper(...)` | Print values concatenated (no spaces) | `whisper("a", "b")` → `ab` | | `shout(...)` | Print values uppercased | `shout("hello")` → `HELLO` |

```
forge x be 42
forge name be "Alice"

speak("The answer is", x)       // The answer is 42
```

```
speak(name, "scored", x, "%")   // Alice scored 42 %
whisper("[", x, "]")             // [42]
shout("warning: low battery")    // WARNING: LOW BATTERY
```

# Built-in Math Functions

## Core Math

| Function | Description | Example |
|---------|----------|---------| | `magnitude(x)` | Absolute value | `magnitude(-5)` → `5` | | `root(x)` | Square root | `root(16)` → `4` | | `ground(x)` | Floor (round down) | `ground(3.7)` → `3` | | `elevate(x)` | Ceiling (round up) | `elevate(3.2)` → `4` | | `nearby(x)` | Round to nearest int | `nearby(3.5)` → `4` | | `least(...)` | Minimum value | `least(3, 1, 4, 1)` → `1` | | `greatest(...)` | Maximum value | `greatest(3, 1, 4, 1)` → `4` |

## Trigonometry

| Function | Description | |---------|----------| | `sin(x)` | Sine (radians) | | `cos(x)` | Cosine (radians) | | `tan(x)` | Tangent (radians) | | `asin(x)` | Arcsine | | `acos(x)` | Arccosine | | `atan(x)` | Arctangent | | `atan2(y, x)` | Two-argument arctangent | | `radians(deg)` | Convert degrees to radians | | `degrees(rad)` | Convert radians to degrees |

## Logarithms & Exponentials

| Function | Description | |----------|----------| | `log(x)` | Natural logarithm | | `log10(x)` | Base-10 logarithm | | `log2(x)` | Base-2 logarithm | | `exp(x)` | e^x |

## Advanced Math

| Function | Description | Example | |----------|----------|---------| | `clamp(v, min, max)` | Constrain value to range | `clamp(150, 0, 100)` → `100` | | `lerp(a, b, t)` | Linear interpolation | `lerp(0, 100, 0.3)` → `30` | | `mapRange(v, fromLo, fromHi, toLo, toHi)` | Map value between ranges | `mapRange(50, 0, 100, 0, 1)` → `0.5` | | `sum(list)` | Sum all elements of a list | `sum([1,2,3,4])` → `10` | | `average(list)` | Mean value of a list | `average([1,2,3,4])` → `2.5` | | `sign(x)` | Sign: -1, 0, or 1 | `sign(-5)` → `-1` | | `dist(x1, y1, x2, y2)` | Distance between two points | `dist(0,0,3,4)` → `5` |

## Random

| Function | Description | Example | |----------|----------|---------| | `chaos()` | Random float 0-1 | `chaos()` → `0.7234` | | `randint(min, max)` | Random integer (inclusive) | `randint(1, 6)` → `4` | | `pick(list)` | Random element from list | `pick(["a","b","c"])` → `"b"` | | `shuffle(list)` | Shuffle a copy of list | `shuffle([1,2,3])` → `[3,1,2]` |

```
// Dice rolling
```

```
conjure rollDice(sides) ::
  yield randint(1, sides)
;;

speak("d6:", rollDice(6))
speak("d20:", rollDice(20))

// Coin flip
speak(chaos() &gt; 0.5 ~ "Heads" : "Tails")
```

# Built-in String Functions

| Function | Description | Signature |
|---------|----------|----------| | `upper(s)` | Uppercase | `upper("hi")` → `"HI"` | | `lower(s)` | Lowercase | `lower("HI")` → `"hi"` | | `trim(s)` | Remove surrounding whitespace | `trim("  hi  ")` → `"hi"` | | `reverse(s)` | Reverse string | `reverse("abc")` → `"cba"` | | `measure(s)` | String length | `measure("hello")` → `5` | | `contains(s, sub)` | Check substring | `contains("hello", "ell")` → `yep` | | `startswith(s, prefix)` | Starts with | `startswith("hello", "he")` → `yep` | | `endswith(s, suffix)` | Ends with | `endswith("hello", "lo")` → `yep` | | `replace(s, old, new)` | Replace all occurrences | `replace("aaa", "a", "b")` → `"bbb"` | | `locate(s, sub)` | Index of substring (-1 if not found) | `locate("hello", "l")` → `2` | | `shatter(s, sep)` | Split string into list | `shatter("a,b", ",")` → `["a","b"]` | | `chars(s)` | Split into list of characters | `chars("abc")` → `["a","b","c"]` | | `weave(list, sep)` | Join list with separator | `weave(["a","b"], "-")` → `"a-b"` | | `padLeft(s, w, c)` | Pad left to width | `padLeft("5", 3, "0")` → `"005"` | |

padRight(s, w, c) | Pad right to width | padRight("hi", 5, ".") → "hi..." | | format(s, ...) | Substitute {} placeholders | format("Hi {}!", "Bob") → "Hi Bob!" | | repeat(s, n) | Repeat string n times | repeat("ab", 3) → "ababab" | | snatch(s, start, end?) | Substring by index | snatch("hello", 1, 3) → "el" |

# Built-in List Functions

| Function | Description | Example | |---------|----------|---------| | measure(list) | Length of list | measure([1,2,3]) → 3 | | gather(list, item) | Append item (mutates) | gather([1,2], 3) → [1,2,3] | | pluck(list) | Remove and return last item | pluck([1,2,3]) → 3 | | snatch(list, idx) | Remove item at index | snatch([1,2,3], 1) → removes 2 | | insert(list, idx, item) | Insert at index | insert([1,3], 1, 2) → [1,2,3] | | portion(list, start, end?) | Slice | portion([1,2,3,4], 1, 3) → [2,3] | | reverse(list) | Return reversed copy | reverse([1,2,3]) → [3,2,1] | | sort(list) | Sort ascending copy | sort([3,1,2]) → [1,2,3] | | sortDesc(list) | Sort descending copy | sortDesc([1,3,2]) → [3,2,1] | | unique(list) | Remove duplicates | unique([1,1,2,2,3]) → [1,2,3] | | flatten(list) | Flatten nested lists | flatten([[1,2],[3,4]]) → [1,2,3,4] | | concat(a, b) | Concatenate two lists | concat([1,2],[3,4]) → [1,2,3,4] | | contains(list, item) | Check membership | contains([1,2,3], 2) → yep | | first(list) | First element

| `first([1,2,3])` → `1` | | `last(list)` | Last element | `last([1,2,3])` → `3` | | `clone(list)` | Deep copy | `clone([1,[2,3]])` |

# Built-in Tome (Dict) Functions

| Function | Description | |----------|----------| | `inscriptions(tome)` | Get all keys as a list | | `contents(tome)` | Get all values as a list | | `entries(tome)` | Get `[[key, value], ...]` pairs | | `contains(tome, key)` | Check if key exists | | `merge(t1, t2, ...)` | Merge dicts (later overrides earlier) | | `erase(tome, key)` | Remove key |

```
forge a be :: "x": 1, "y": 2 ;;
forge b be :: "y": 99, "z": 3 ;;
forge c be merge(a, b)
speak(c)  // {"x": 1, "y": 99, "z": 3}
```

# Input / Output

## `input(prompt?)` — Read User Input

Prompts the user for input and returns the entered text:

```
forge name be input("What is your name? ")
speak("Hello, " + name + "!")
```

```
forge age be morph(input("Enter your age: "), "number")
speak("You are " + age + " years old")
```

### `print(...)` / `println(...)` — Print (Aliases)

Standard aliases for `speak()`:

```
print("Hello")
println("World")
```

## Character & Code Point Functions

| Function | Description | Example |
|---------|----------|---------|
| `chr(n)` | Number to character | `chr(65)` → `"A"` |
| `ord(c)` | Character to number | `ord("A")` → `65` |

```
speak(chr(72) + chr(105))  // "Hi"
speak(ord("A"))             // 65

// Build a Caesar cipher
conjure encrypt(text, shift) ::
  forge result be ""
  iterate through text ::
    forge code be ord(item) + shift
    result be result + chr(code)
  ;;
  yield result
;;
speak(encrypt("ABC", 3))  // "DEF"
```

# Number Base Conversion

| Function | Description | Example | |---------|----------|---------| | `hex(n)` | Number to hex string | `hex(255)` → `"0xFF"` | | `oct(n)` | Number to octal | `oct(8)` → `"0o10"` | | `bin(n)` | Number to binary | `bin(10)` → `"0b1010"` | | `parseNum(str, base?)` | Parse string with base | `parseNum("FF", 16)` → `255` |

```
speak(hex(255))        // "0xFF"
speak(bin(42))         // "0b101010"
speak(oct(64))         // "0o100"
speak(parseNum("1010", 2))  // 10
speak(parseNum("FF", 16))   // 255
```

# Number Formatting & Checking

| Function | Description | Example | |---------|----------|---------| | `toFixed(n, digits)` | Format decimal places | `toFixed(3.14159, 2)` → `"3.14"` | | `toPrecision(n, p)` | Format to precision | `toPrecision(123.456, 4)` → `"123.5"` | | `isNaN(v)` | Check if NaN | `isNaN(0/0)` → `yep` | | `isFinite(v)` | Check if finite | `isFinite(INFINITY)` → `nope` | | `isInteger(v)` | Check if integer | `isInteger(3.0)` → `yep` |

```
forge pi be 3.14159265
speak(toFixed(pi, 2))      // "3.14"
speak(toPrecision(pi, 4))  // "3.142"
```

```
speak(isInteger(42))       // yep
speak(isFinite(1/0))       // nope
```

# String Checking Functions

| Function | Description | Example |
|---------|----------|---------| | `capitalize(s)` | First char uppercase | `capitalize("hello")` → `"Hello"` | | `title(s)` | Title Case | `title("hello world")` → `"Hello World"` | | `center(s, width, char?)` | Center-pad | `center("hi", 10, "-")` → `"----hi----"` | | `trimLeft(s)` | Trim left whitespace | `trimLeft("  hi")` → `"hi"` | | `trimRight(s)` | Trim right whitespace | `trimRight("hi  ")` → `"hi"` | | `isUpper(s)` | All uppercase? | `isUpper("ABC")` → `yep` | | `isLower(s)` | All lowercase? | `isLower("abc")` → `yep` | | `isDigit(s)` | All digits? | `isDigit("123")` → `yep` | | `isAlpha(s)` | All alphabetic? | `isAlpha("abc")` → `yep` | | `isAlphaNum(s)` | All alphanumeric? | `isAlphaNum("abc123")` → `yep` | | `isSpace(s)` | All whitespace? | `isSpace(" ")` → `yep` |

```
speak(capitalize("hello world"))  // "Hello world"
speak(title("the quick brown fox"))  // "The Quick Brown Fox"
speak(center("TITLE", 20, "="))  // "=======TITLE========"
speak(isDigit("42"))   // yep
speak(isAlpha("hello"))  // yep
```

# Regex / Pattern Matching

| Function | Description | |----------|----------| | `match(text, pattern)` | First regex match (returns list or void) | | `matchAll(text, pattern)` | All regex matches | | `replaceRegex(text, pattern, replacement)` | Regex replace (global) | | `test(text, pattern)` | Test if pattern matches |

```
forge text be "Hello 123 World 456"
speak(match(text, "\\d+"))           // ["123"]
speak(matchAll(text, "\\d+"))        // [["123"], ["456"]]
speak(replaceRegex(text, "\\d+", "#"))  // "Hello # World #"
speak(test(text, "\\d+"))           // yep

// Validate email
conjure isEmail(s) ::
  yield test(s, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$
;;
speak(isEmail("user@example.com"))  // yep
```

# Bitwise Operations

| Function | Description | Example | |----------|----------|---------| | `bitAnd(a, b)` | Bitwise AND | `bitAnd(5, 3)` → `1` | | `bitOr(a, b)` | Bitwise OR | `bitOr(5, 3)` → `7` | | `bitXor(a, b)` | Bitwise XOR | `bitXor(5, 3)` → `6` | | `bitNot(a)` | Bitwise NOT | `bitNot(0)` → `-1` | | `bitShiftLeft(a, n)` | Left shift | `bitShiftLeft(1, 3)` → `8` | | `bitShiftRight(a, n)` | Right shift | `bitShiftRight(8, 2)` → `2` |

```
// Flags / bitmask example
forge READ be 1
```

```
forge WRITE be 2
forge EXEC be 4

forge perms be bitOr(READ, WRITE)  // 3
speak(bitAnd(perms, READ) &gt; 0)     // yep (has read)
speak(bitAnd(perms, EXEC) &gt; 0)     // nope (no exec)
```

# Base64 Encoding

| Function | Description | |----------|----------| | `base64encode(text)` | Encode text to base64 | | `base64decode(text)` | Decode base64 to text |

```
forge encoded be base64encode("Hello, World!")
speak(encoded)  // "SGVsbG8sIFdvcmxkIQ=="
speak(base64decode(encoded))  // "Hello, World!"
```

# Hash Function

```
speak(hash("hello"))     // deterministic 32-bit integer hash
speak(hash([1, 2, 3]))   // works on any value
```

# Time & Date

| Function | Description | |----------|----------| | `now()` | Current timestamp in milliseconds | | `timestamp()` | ISO 8601 string | | `time()` | Detailed time tome with year, month, day, etc. | | `formatTime(ms)` | Format milliseconds

to ISO string |

```
forge t be time()
speak("Year: " + t.year)
speak("Month: " + t.month)
speak("Day: " + t.day)
speak("Hour: " + t.hour)
speak("ISO: " + t.iso)

forge start be now()
// ... do work ...
forge elapsed be now() - start
speak("Took " + elapsed + "ms")
```

# Functional Programming

| Function | Description | |----------|----------| | `compose(f, g, ...)` | Right-to-left function composition | | `pipe(value, f, g, ...)` | Left-to-right value piping | | `curry(fn, arity)` | Currying | | `memoize(fn)` | Cache function results | | `tap(value, fn)` | Execute fn with value, return value | | `times(n, fn)` | Call fn n times with index | | `groupBy(list, fn)` | Group elements by key function | | `chunk(list, size)` | Split list into chunks |

```
// Compose
conjure double(x) :: yield x * 2 ;;
conjure addOne(x) :: yield x + 1 ;;
forge doubleAndAdd be compose(addOne, double)
speak(doubleAndAdd(5))  // 11

// Pipe
forge result be pipe(5, double, addOne, double)
```

```
speak(result)  // 22

// Memoize (cache expensive computations)
conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  yield fib(n - 1) + fib(n - 2)
;;
forge fastFib be memoize(fib)
speak(fastFib(10))  // 55

// Group and chunk
forge words be ["apple", "avocado", "banana", "blueberry", "cherry
speak(groupBy(words, w -&gt; charAt(w, 0)))
// :: a: ["apple", "avocado"], b: ["banana", "blueberry"], c: ["che

speak(chunk([1,2,3,4,5,6,7], 3))  // [[1,2,3], [4,5,6], [7]]

// Times
speak(times(5, i -&gt; i * i))  // [0, 1, 4, 9, 16]
```

# Buffer / Memory Operations

sdev provides low-level byte buffer operations for systems programming:

## `buffer(size)` — Create Byte Buffer

```
forge mem be buffer(1024)
mem.set(0, 255)
mem.set(1, 128)
speak(mem.get(0))    // 255
speak(mem.get(1))    // 128
speak(mem.size())    // 1024
```

## Buffer Methods

| Method | Description | |--------|----------| | `buf.get(index)` | Read byte at index | | `buf.set(index, value)` | Write byte at index (0-255) | | `buf.fill(value)` | Fill entire buffer | | `buf.slice(start, end)` | Get portion as list | | `buf.toList()` | Convert to list of numbers | | `buf.toText()` | Decode as UTF-8 text | | `buf.fromString(text)` | Write UTF-8 text into buffer | | `buf.copyTo(target)` | Copy data to another buffer | | `buf.size()` | Get buffer size |

## `pointer(buffer, offset)` — Memory Pointer

```
forge mem be buffer(256)
forge ptr be pointer(mem, 0)
ptr.write(42)
speak(ptr.read())        // 42

forge ptr2 be ptr.advance(4)
ptr2.writeU16(1024)
speak(ptr2.readU16())    // 1024

ptr2.writeU32(0xDEADBEEF)
speak(hex(ptr2.readU32()))  // "0xDEADBEEF"
```

## Pointer Methods

| Method | Description | |--------|----------| | `ptr.read()` | Read byte | | `ptr.write(v)` | Write byte | | `ptr.advance(n?)` | Return new pointer at offset+n | | `ptr.readU16()` | Read 16-bit unsigned (little-endian) | | `ptr.readU32()` | Read 32-bit unsigned (little-endian) | | `ptr.writeU16(v)` | Write

16-bit unsigned | | `ptr.writeU32(v)` | Write 32-bit unsigned |

# Error Handling & Control Flow

| Function | Description | |----------|----------| | `exit(code?)` | Terminate program with exit code | | `panic(message)` | Fatal error (kernel panic) | | `throw(message)` | Throw custom error |

```
// Throw and catch
attempt ::
  forge x be input("Enter a number: ")
  forge n be morph(x, "number")
  ponder n &lt; 0 ::
    throw("Negative numbers not allowed!")
  ;;
  speak("Square root: " + root(n))
rescue err ::
  speak("Error: " + err)
;;

// Exit
ponder badCondition ::
  exit(1)
;;
```

# Additional Aliases

| Alias | Original | Description | |-------|----------|-------------| | `print()` | `speak()` | Standard print | | `range()` | `sequence()` | Standard range | | `typeof()` | `gettype()` | Type checking | | `sleep()` | `delay()` | Pause (no-op in browser) | | `keys()` | `inscriptions()` | Dict keys | | `values()` | `contents()` | Dict values | | `freeze(obj)` | — | Make object immutable | | `isFrozen(obj)` | — | Check if frozen | | `hash(value)` | — | 32-bit hash code |

# Higher-Order Functions

## `each` — Transform (Map)

Apply a function to every element and return a new list:

```
forge nums be [1, 2, 3, 4, 5]
speak(each(nums, x -&gt; x ^ 2))          // [1, 4, 9, 16, 25]
speak(each(nums, (x, i) -&gt; x + i))     // [1, 3, 5, 7, 9]  (valu
speak(each(["a","b","c"], upper))       // ["A", "B", "C"]
```

## `sift` — Filter

Keep only elements where the predicate returns `yep`:

```
forge nums be [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
speak(sift(nums, x -&gt; x % 2 equals 0))       // [2, 4, 6, 8, 10
speak(sift(nums, x -&gt; x &gt; 5))                // [6, 7, 8, 9,
speak(sift(["", "hi", "", "world"], x -&gt; measure(x) &gt; 0))  //
```

## `fold` — Reduce

Reduce a list to a single value using an accumulator:

```
forge nums be [1, 2, 3, 4, 5]
speak(fold(nums, 0, (acc, x) -&gt; acc + x))   // 15 (sum)
speak(fold(nums, 1, (acc, x) -&gt; acc * x))   // 120 (product)

// Build a string
speak(fold(["a","b","c"], "", (acc, x) -&gt; acc + x + "-"))  // "a

// Find max
speak(fold(nums, nums[0], (a, b) -&gt; a &gt; b ~ a : b))   // 5
```

## `seek` — Find First

Return the first element matching a predicate, or `void`:

```
forge people be [
  :: "name": "Alice", "age": 22 ;;,
  :: "name": "Bob", "age": 35 ;;,
  :: "name": "Carol", "age": 28 ;;
]

forge adult be seek(people, p -&gt; p.age &gt;= 30)
speak(adult.name)   // Bob

forge nobody be seek(people, p -&gt; p.age &gt; 100)
speak(nobody)   // void
```

## `every` — All Match

Returns `yep` if ALL elements satisfy the predicate:

```
speak(every([2,4,6,8], x -&gt; x % 2 equals 0))  // yep
speak(every([1,2,3], x -&gt; x &gt; 0))             // yep
speak(every([1,-2,3], x -&gt; x &gt; 0))            // nope
```

### `some` — Any Match

Returns `yep` if ANY element satisfies the predicate:

```
speak(some([1,3,5,6], x -&gt; x % 2 equals 0))  // yep (6 is even)
speak(some([1,3,5,7], x -&gt; x % 2 equals 0))  // nope
```

### `enumerate` — Index + Value

Pairs each element with its index:

```
speak(enumerate(["a","b","c"]))
// [[0,"a"], [1,"b"], [2,"c"]]

within pair be enumerate(["red","green","blue"]) ::
  speak(pair[0] + ": " + pair[1])
;;
// 0: red
// 1: green
// 2: blue
```

### `zip` — Pair Two Lists

Combine two lists element-by-element:

```
forge keys be ["name", "age", "city"]
forge vals be ["Alice", 30, "NYC"]
speak(zip(keys, vals))
// [["name","Alice"], ["age",30], ["city","NYC"]]
```

# Type System & Conversion

### Getting Type — `essence()`

Returns the type name as a string:

```
speak(essence(42))          // "number"
speak(essence("hi"))        // "text"
speak(essence(yep))         // "truth"
speak(essence(void))        // "void"
speak(essence([1,2,3]))     // "list"
speak(essence(::;;))        // "tome"
```

### Converting Types — `morph()`

```
speak(morph("42", "number"))     // 42
speak(morph(42, "text"))         // "42"
speak(morph(1, "truth"))         // yep
speak(morph(0, "truth"))         // nope
speak(morph("3.14", "number"))   // 3.14
```

### Type Checking Helpers

```
speak(essence(x) equals "number")      // Check if number
speak(essence(x) equals "text")        // Check if string
speak(essence(x) equals "list")        // Check if list
speak(essence(x) equals "tome")        // Check if dict
speak(x equals void)                   // Check if null
```

# Collections

## Set

A collection of unique elements. Duplicate additions are silently ignored.

```
forge s be Set()
s.add(1)
s.add(2)
s.add(2)    // Duplicate ignored
s.add(3)

speak(s.size())     // 3
speak(s.has(2))     // yep
speak(s.has(99))    // nope
speak(s.values())   // [1, 2, 3]

s.remove(2)
speak(s.values())   // [1, 3]

s.clear()
speak(s.size())     // 0
speak(s.isEmpty())  // yep
```

**Set methods:** `add(v)`, `remove(v)`, `has(v)`, `values()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## Map

A key-value store where keys can be any type (including objects):

```
forge m be Map()
m.set("name", "Alice")
m.set("age", 30)
m.set("active", yep)

speak(m.get("name"))     // Alice
speak(m.has("age"))      // yep
```

```
speak(m.keys())          // ["name", "age", "active"]
speak(m.values())        // ["Alice", 30, yep]
speak(m.entries())       // [["name","Alice"], ["age",30], ["activ
speak(m.size())          // 3

m.delete("active")
speak(m.size())          // 2
m.clear()
```

**Map methods:** `set(k, v)`, `get(k)`, `has(k)`, `delete(k)`, `keys()`, `values()`, `entries()`, `size()`, `isEmpty()`, `clear()`

## Queue (FIFO)

First-in, first-out data structure:

```
forge q be Queue()
q.enqueue("first")
q.enqueue("second")
q.enqueue("third")

speak(q.peek())      // first  (look without removing)
speak(q.dequeue())   // first  (remove and return)
speak(q.peek())      // second
speak(q.size())      // 2
speak(q.isEmpty())   // nope
```

**Queue methods:** `enqueue(v)`, `dequeue()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## Stack (LIFO)

Last-in, first-out data structure:

```
forge s be Stack()
s.push(10)
```

```
s.push(20)
s.push(30)

speak(s.peek())   // 30  (look without removing)
speak(s.pop())    // 30  (remove and return)
speak(s.pop())    // 20
speak(s.size())   // 1

// Stack-based undo system
forge history be Stack()
history.push("action1")
history.push("action2")
history.push("action3")

speak("Undoing:", history.pop())  // action3
speak("Undoing:", history.pop())  // action2
```

**Stack methods:** `push(v)`, `pop()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## LinkedList

A doubly-linked list with O(1) front/back operations:

```
forge list be LinkedList()
list.append(1)
list.append(2)
list.append(3)
list.prepend(0)

speak(list.toList())   // [0, 1, 2, 3]
speak(list.size())     // 4
speak(list.get(2))     // 2

list.remove(1)         // Remove element with value 1
speak(list.toList())   // [0, 2, 3]

speak(list.head())     // 0
```

```
speak(list.tail())     // 3
```

**LinkedList methods:** `append(v)`, `prepend(v)`, `remove(v)`, `get(idx)`, `size()`, `head()`, `tail()`, `toList()`, `clear()`

# Matrix Operations

sdev has a comprehensive built-in matrix math library useful for data science and machine learning.

## Creating Matrices

```
// Matrix filled with a value
forge zeros be matrix(3, 3, 0)
// [[0,0,0], [0,0,0], [0,0,0]]

forge ones be matrix(2, 4, 1)
// [[1,1,1,1], [1,1,1,1]]

// Identity matrix
forge eye be identity(4)
// [[1,0,0,0], [0,1,0,0], [0,0,1,0], [0,0,0,1]]

// Reshape a flat list into a matrix
forge m be reshape([1,2,3,4,5,6], 2, 3)
// [[1,2,3], [4,5,6]]
```

## Matrix Arithmetic

```
forge a be [[1, 2], [3, 4]]
forge b be [[5, 6], [7, 8]]

// Element-wise operations
```

```
speak(matadd(a, b))     // [[6,8], [10,12]]
speak(matsub(a, b))     // [[-4,-4], [-4,-4]]
speak(matscale(a, 2))   // [[2,4], [6,8]]

// Matrix multiplication
speak(matmul(a, b))
// [[1*5+2*7, 1*6+2*8], [3*5+4*7, 3*6+4*8]]
// = [[19,22], [43,50]]

// Transpose
speak(transpose(a))
// [[1,3], [2,4]]
```

## Vector Operations

```
forge v1 be [1, 2, 3]
forge v2 be [4, 5, 6]

// Dot product
speak(dot(v1, v2))   // 1*4 + 2*5 + 3*6 = 32

// Element-wise multiply via each
forge elem be each(v1, (x, i) -&gt; x * v2[i])
speak(elem)   // [4, 10, 18]
```

## Matrix Utilities

| Function | Description | |----------|----------| | `matrix(rows, cols, fill)` | Create filled matrix | | `identity(n)` | Create n×n identity matrix | | `transpose(m)` | Transpose rows and columns | | `matmul(a, b)` | Matrix multiplication | | `matadd(a, b)` | Element-wise addition | | `matsub(a, b)` | Element-wise subtraction | | `matscale(m, s)` | Scalar multiplication | | `dot(a, b)` | Dot product of

two vectors | | `reshape(list, rows, cols)` | Reshape list to 2D matrix | | `flatten(m)` | Flatten 2D matrix to 1D list | | `shape(m)` | Get `[rows, cols]` of matrix | | `sum(m)` | Sum all elements | | `mean(list)` | Mean of a 1D list |

## Neural Network Example

```
// Sigmoid activation function
conjure sigmoid(x) ::
  yield 1 / (1 + exp(-x))
;;

// Feedforward neural network layer
conjure layer(inputs, weights, biases) ::
  // inputs: [1 x n], weights: [n x m], biases: [1 x m]
  forge z be matmul(inputs, weights)
  forge zb be matadd(z, biases)
  // Apply sigmoid element-wise
  yield each(zb, row -&gt; each(row, x -&gt; sigmoid(x)))
;;

forge input be [[0.5, 0.3, 0.2]]
forge w1 be [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]]
forge b1 be [[0.1, 0.1]]

forge output be layer(input, w1, b1)
speak("Layer output:", output)
```

# Graphics & Game Development

sdev has a built-in 2D graphics and turtle graphics API. Graphics commands render to the Canvas panel in the IDE.

## Canvas Setup

```
canvas(800, 600)    // Set canvas dimensions (width, height)
clear("#1a1a2e")    // Clear with a background color
```

## Fill and Stroke

```
fill("blue")          // Set fill color
noFill()              // Disable fill
stroke("red", 2)      // Set stroke: color, width
noStroke()            // Disable stroke
lineWidth(3)          // Set line width separately
alpha(0.5)            // Set global transparency (0–1)
lineCap("round")      // Line cap: "round", "square", "butt"
lineJoin("round")     // Line join: "round", "bevel", "miter"
```

## Basic Shapes

```
// Rectangle: rect(x, y, width, height, cornerRadius?)
fill("steelblue")
rect(50, 50, 200, 100)
rect(300, 50, 200, 100, 20)    // Rounded corners

// Circle: circle(x, y, radius)
fill("tomato")
circle(150, 300, 60)

// Ellipse: ellipse(x, y, radiusX, radiusY, rotation?)
fill("gold")
ellipse(350, 300, 100, 50)
```

```
ellipse(550, 300, 100, 50, 45)   // Rotated 45°

// Line: line(x1, y1, x2, y2)
stroke("white", 2)
line(50, 400, 750, 400)

// Arc: arc(x, y, radius, startAngle, endAngle, counterclockwise?)
stroke("lime", 3)
arc(200, 500, 50, 0, 270)

// Triangle: triangle(x1, y1, x2, y2, x3, y3)
fill("orchid")
triangle(400, 450, 450, 550, 350, 550)

// Polygon: polygon([[x,y], ...])
fill("coral")
polygon([[600, 400], [650, 430], [640, 490], [560, 490], [550, 430

// Star: star(x, y, outerR, innerR, points?)
fill("gold")
star(150, 150, 50, 25)          // 5-pointed star
star(300, 150, 60, 25, 8)       // 8-pointed star

// Heart: heart(x, y, size)
fill("red")
heart(500, 150, 50)

// Point: point(x, y, size?)
fill("white")
point(100, 100)
point(150, 100, 8)
```

## Text Drawing

```
fill("white")
text("Hello, World!", 100, 100)           // Basic text
text("Large Text", 100, 150, 36)          // With size
text("Bold Text", 100, 200, 24, "bold")   // With style
```

```
font("Arial", "bold")                    // Set font family and s
textAlign("center", "middle")            // Horizontal: left/cente
```

## Gradients

```
// Linear gradient: fill with gradient from (x1,y1) to (x2,y2)
// Color stops: [position, color] where position is 0.0 to 1.0
linearGradient(0, 0, 400, 0, [0, "red"], [0.5, "yellow"], [1, "gree
rect(0, 0, 400, 100)

// Radial gradient: from inner circle to outer circle
radialGradient(200, 200, 0, 200, 200, 150, [0, "white"], [0.5, "cya
circle(200, 200, 150)
```

## Shadows

```
shadow("rgba(0,0,0,0.5)", 15, 5, 5)   // color, blur, offsetX, offs
fill("gold")
circle(200, 200, 80)
noShadow()
```

## Transformations

```
save()                // Push current state
translate(200, 200)   // Move origin to (200, 200)
rotate(0.785)         // Rotate 45° (in radians)
scale(1.5)            // Scale uniformly
rect(-50, -50, 100, 100)   // Draw centered rectangle
restore()             // Pop saved state

resetTransform()      // Reset all transforms to identity
```

## Path Drawing

```
// Custom polygon path
fill("purple")
beginPath()
moveTo(100, 100)
lineTo(200, 80)
lineTo(250, 150)
lineTo(200, 220)
lineTo(100, 200)
closePath()
fillPath()
strokePath()

// Bezier curve
stroke("cyan", 2)
beginPath()
moveTo(100, 300)
bezierTo(150, 200, 250, 400, 300, 300)   // cp1x, cp1y, cp2x, cp2y
strokePath()

// Quadratic curve
beginPath()
moveTo(100, 400)
quadraticTo(200, 350, 300, 400)           // cpx, cpy, x, y
strokePath()
```

## Turtle Graphics

The turtle graphics API lets you draw by commanding a turtle that moves and draws lines as it goes.

```
canvas(500, 500)
clear("#0d0d20")
turtle()           // Create turtle at center, facing up

// Basic movement
forward(100)       // Move forward 100 units (drawing as it goes)
right(90)          // Turn right 90 degrees
forward(100)
```

```
right(90)
forward(100)
right(90)
forward(100)       // This draws a square!

// Pen control
penup()            // Lift pen — move without drawing
goto(250, 250)     // Jump to coordinates
pendown()          // Lower pen — start drawing again

// Pen style
pencolor("lime")   // Set pen color (any CSS color)
penwidth(3)        // Set pen width

// Advanced
setheading(45)     // Set absolute direction (degrees)
speak(heading())   // Get current heading
speak(pos())       // Get current [x, y] position
home()             // Return to start position

// Turtle circle
turtleCircle(50)        // Draw full circle
turtleCircle(50, 72)    // Polygon approximation (72 steps)

// Dot and stamp
dot(10, "red")     // Draw a filled dot
stamp()            // Leave a mark of the turtle shape

// EXAMPLE: Psychedelic spiral
canvas(600, 600)
clear("#000000")
turtle()
within i be sequence(0, 300) ::
  pencolor(hue(i * 1.2))
  penwidth(1 + i * 0.02)
  forward(i)
  right(91)
;;
```

## Sprites & Game Objects

```
// Create sprite: createSprite(x, y, width, height, color?)
forge player be createSprite(100, 100, 40, 40, "blue")
forge enemy be createSprite(400, 300, 40, 40, "red")

// Access/set properties
player.x be 150
player.velocityX be 5
player.velocityY be -2

// Update position (applies velocity)
updateSprite(player)

// Draw sprite
drawSprite(player)
drawSprite(enemy)

// Collision detection
ponder spriteCollides(player, enemy) ::
  speak("HIT!")
;;

// Move by delta
moveSprite(player, 10, 5)
```

## Color Functions

```
// CSS color names work directly
fill("red")
fill("steelblue")
fill("rgba(255, 0, 0, 0.5)")

// Helper functions
forge red be rgb(255, 0, 0)
forge transparent be rgba(0, 255, 0, 0.3)

// HSL color — hue(hue, saturation?, lightness?)
```

```
forge cyan be hue(180)              // Cyan (s=100, l=50 defaults)
forge purple be hue(270, 80, 60)    // Custom HSL
forge faded be hsla(240, 100, 50, 0.4)

// Rainbow cycle (great for spirals!)
within i be sequence(360) ::
  fill(hue(i))
  rect(i, 0, 1, 50)
;;

// Random color
fill(randomColor())
```

## Graphics Math

```
// Convert degrees/radians
speak(radians(180))   // 3.14159...
speak(degrees(PI))    // 180

// Lerp (smooth transitions)
forge pos be lerp(0, 100, 0.25)   // 25

// Map value between ranges
forge brightness be mapRange(50, 0, 100, 0, 255)   // 127.5

// Clamp
forge safe be clamp(150, 0, 100)   // 100

// Distance between two points
speak(dist(0, 0, 3, 4))   // 5
```

## Complete 2D Graphics Example — Starfield

```
canvas(800, 600)
```

```
// Create 200 stars
forge stars be each(sequence(200), i -&gt; ::
  "x": randint(0, 800),
  "y": randint(0, 600),
  "r": chaos() * 2 + 0.5,
  "speed": chaos() * 2 + 0.5,
  "bright": chaos() * 0.7 + 0.3
;;)

clear("#000011")

within star be stars ::
  fill(rgba(255, 255, 255, star.bright))
  circle(star.x, star.y, star.r)
;;

// Draw nebula in center
radialGradient(400, 300, 0, 400, 300, 200,
  [0, "rgba(100,50,200,0.4)"],
  [0.5, "rgba(50,100,255,0.2)"],
  [1, "rgba(0,0,0,0)"]
)
circle(400, 300, 200)
```

# File I/O

> ***Note:*** *File I/O is available in the Python standalone interpreter and native desktop builds. In the browser-based IDE, these functions are simulated.*

## Reading Files

```
// Read entire file as text
```

```
forge content be decipher("data.txt")
speak(content)

// Read and parse JSON
forge config be unetch(decipher("config.json"))
speak(config.version)
speak(config.name)
```

## Writing Files

```
// Write text file
inscribe("output.txt", "Hello, World!\n")

// Write JSON with indentation
forge data be :: "score": 100, "name": "Alice", "level": 5 ;;
inscribe("save.json", etch(data))
```

## Append to File

```
appendFile("log.txt", "User logged in\n")
appendFile("log.txt", "User viewed dashboard\n")
```

## File Utilities

| Function | Description |
|---|---|
| `decipher(path)` | Read file as text |
| `inscribe(path, content)` | Write text to file |
| `appendFile(path, content)` | Append to file |
| `fileExists(path)` | Returns `yep`/`nope` |
| `deleteFile(path)` | Delete a file |
| `listDir(path)` | Get list of files in directory |

# Networking

> ***Note:*** *Network functions are available in async contexts. In the browser they use* `fetch`*. In native builds they use Node.js http.*

## HTTP GET

```
async conjure getUsers() ::
  forge data be await fetch("https://jsonplaceholder.typicode.com/u
  yield data
;;
```

## HTTP POST

```
async conjure createUser(name, email) ::
  forge response be await fetch("https://api.example.com/users", :
    "method": "POST",
    "headers": :: "Content-Type": "application/json" ;;,
    "body": :: "name": name, "email": email ;;
  ;;)
  yield response
;;
```

# Examples & Recipes

## FizzBuzz

```
within i be sequence(1, 101) ::
  ponder i % 15 equals 0 ::
```

```
    speak("FizzBuzz")
  ;; otherwise ponder i % 3 equals 0 ::
    speak("Fizz")
  ;; otherwise ponder i % 5 equals 0 ::
    speak("Buzz")
  ;; otherwise ::
    speak(i)
  ;;
;;
```

## Fibonacci (Memoized)

```
forge memo be :: ;;

conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  ponder contains(memo, morph(n, "text")) ::
    yield memo[morph(n, "text")]
  ;;
  forge result be fib(n - 1) + fib(n - 2)
  memo[morph(n, "text")] be result
  yield result
;;

within i be sequence(0, 30) ::
  speak("fib(" + morph(i, "text") + ") =", fib(i))
;;
```

## Binary Search

```
conjure binarySearch(arr, target) ::
  forge lo be 0
  forge hi be measure(arr) - 1

  cycle lo &lt;= hi ::
    forge mid be ground((lo + hi) / 2)
    ponder arr[mid] equals target ::
```

```
      yield mid
    ;; otherwise ponder arr[mid] &lt; target ::
      lo be mid + 1
    ;; otherwise ::
      hi be mid - 1
    ;;
  ;;

  yield -1
;;

forge sorted be [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
speak(binarySearch(sorted, 23))   // 5
speak(binarySearch(sorted, 50))   // -1
```

## Quicksort

```
conjure quicksort(arr) ::
  ponder measure(arr) &lt;= 1 :: yield arr ;;

  forge pivot be arr[ground(measure(arr) / 2)]
  forge less be sift(arr, x -&gt; x &lt; pivot)
  forge equal be sift(arr, x -&gt; x equals pivot)
  forge greater be sift(arr, x -&gt; x &gt; pivot)

  yield quicksort(less) + equal + quicksort(greater)
;;

speak(quicksort([3, 6, 8, 10, 1, 2, 1]))
// [1, 1, 2, 3, 6, 8, 10]
```

## Stack-Based Calculator

```
conjure calculate(expression) ::
  forge tokens be shatter(expression, " ")
  forge stack be Stack()
```

```
  within token be tokens ::
    ponder token equals "+" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a + b)
    ;; otherwise ponder token equals "-" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a - b)
    ;; otherwise ponder token equals "*" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a * b)
    ;; otherwise ponder token equals "/" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a / b)
    ;; otherwise ::
      stack.push(morph(token, "number"))
    ;;
  ;;

  yield stack.pop()
;;

speak(calculate("3 4 + 2 *"))    // (3+4)*2 = 14
speak(calculate("5 1 2 + 4 * + 3 -"))  // 5+(1+2)*4-3 = 14
```

## Caesar Cipher

```
conjure caesarEncode(text, shift) ::
  forge result be ""
  within ch be chars(text) ::
    forge code be locate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", upper(ch))
    ponder code &gt;= 0 ::
      forge shifted be (code + shift) % 26
      result be result + "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[shifted]
    ;; otherwise ::
```

```
      result be result + ch
    ;;
  ;;
  yield result
;;

speak(caesarEncode("HELLO WORLD", 3))   // KHOOR ZRUOG
speak(caesarEncode("KHOOR ZRUOG", 23))  // HELLO WORLD (decode with
```

## Word Frequency Counter

```
conjure wordFrequency(text) ::
  forge words be shatter(lower(trim(text)), " ")
  forge freq be ::;;

  within word be words ::
    forge w be replace(replace(word, ".", ""), ",", "")
    ponder measure(w) &gt; 0 ::
      ponder contains(freq, w) ::
        freq[w] be freq[w] + 1
      ;; otherwise ::
        freq[w] be 1
      ;;
    ;;
  ;;

  yield freq
;;

forge text be "the quick brown fox jumps over the lazy dog the fox
forge freq be wordFrequency(text)
speak(freq)
// Sort by frequency
forge sorted be sort(inscriptions(freq), k -&gt; -freq[k])
```

## Prime Sieve

```
conjure sieve(limit) ::
  forge primes be each(sequence(2, limit + 1), x -&gt; x)
  forge result be []

  cycle measure(primes) &gt; 0 ::
    forge p be primes[0]
    gather(result, p)
    primes be sift(primes, x -&gt; x % p differs 0)
  ;;

  yield result
;;

speak(sieve(50))
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

## Linked-List Graph Traversal (BFS)

```
// BFS using a Queue
conjure bfs(graph, start) ::
  forge visited be Set()
  forge queue be Queue()
  forge order be []

  queue.enqueue(start)
  visited.add(start)

  cycle isnt queue.isEmpty() ::
    forge node be queue.dequeue()
    gather(order, node)

    within neighbor be graph[node] ::
      ponder isnt visited.has(neighbor) ::
        visited.add(neighbor)
        queue.enqueue(neighbor)
      ;;
    ;;
  ;;
```

```
  yield order
;;

forge graph be ::
  "A": ["B", "C"],
  "B": ["A", "D", "E"],
  "C": ["A", "F"],
  "D": ["B"],
  "E": ["B", "F"],
  "F": ["C", "E"]
;;

speak(bfs(graph, "A"))  // ["A", "B", "C", "D", "E", "F"]
```

## Turtle Mandala

```
canvas(600, 600)
clear("#000000")
turtle()
penwidth(1.5)

forge petals be 12
forge petalAngle be 360 / petals

within i be sequence(petals) ::
  pencolor(hue(i * petalAngle))
  within j be sequence(60) ::
    forward(j * 0.5)
    right(6)
  ;;
  penup()
  home()
  pendown()
  setheading(i * petalAngle)
;;
```

# JavaScript Interop (JS Interpreter Only)

The browser-based interpreter includes a special `js` keyword for calling JavaScript functions directly from sdev code. This allows integration with browser APIs, DOM manipulation, and third-party JS libraries.

## Three Syntax Forms

```
// 1. Single-line expression
js &lt;expression&gt;;

// 2. Parenthesized form (for objects, multi-line expressions)
js (
  &lt;expression&gt;;
)

// 3. Statement block
js {
  &lt;statements&gt;;
}
```

## Basic Examples

```
// Browser dialog
js alert("Hello from sdev!")

// Read browser properties
forge width be js window.innerWidth
speak("Width:", width)

// DOM manipulation
js document.body.style.backgroundColor = "#1a1a2e"
```

```
js document.title = "My sdev App"

// Math
forge angle be js Math.PI / 6
forge result be js Math.sin(0.5)
```

## Object Literals

```
// Use parenthesized form for object literals
forge opts be js ({
  dragging: true,
  zoom: 12,
  center: [51.505, -0.09]
})

// Nested objects
forge config be js ({
  api: {
    url: "https://api.example.com",
    timeout: 5000
  },
  debug: false
})
```

## Arrow Functions

```
// Map/filter in JS
forge doubled be js [1, 2, 3, 4].map(x =&gt; x * 2)
forge filtered be js [1, 2, 3, 4].filter(x =&gt; x &gt; 2)

// Event handlers
js document.getElementById("btn").addEventListener("click", (e) =&g
  console.log("Clicked!", e.target)
})

// Use sdev variables in JS arrows
forge multiplier be 5
```

```
forge result be js [1, 2, 3].map(x =&gt; x * multiplier)
```

## Multi-line JS Blocks

```
js {
  const canvas = document.createElement("canvas")
  canvas.width = 400
  canvas.height = 300
  document.body.appendChild(canvas)

  const ctx = canvas.getContext("2d")
  ctx.fillStyle = "navy"
  ctx.fillRect(0, 0, 400, 300)
  ctx.fillStyle = "white"
  ctx.font = "24px monospace"
  ctx.fillText("Made in sdev!", 100, 150)
}
```

## Leaflet Map Integration

```
// Initialize a Leaflet map
forge map be js L.map("map-container").setView([51.505, -0.09], 13

js L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
  attribution: "© OpenStreetMap contributors"
})).addTo(map)

// Add markers
conjure addMarker(lat, lng, label) ::
  js L.marker([lat, lng]).addTo(map).bindPopup(label)
;;

addMarker(51.505, -0.09, "London, UK")
addMarker(48.8566, 2.3522, "Paris, France")
addMarker(52.52, 13.405, "Berlin, Germany")
```

# Complete Reference Card

## Keywords

| Keyword | Purpose | |---------|---------| | `forge` | Declare a variable | | `be` | Assign a value | | `conjure` | Declare a function | | `yield` | Return a value from a function | | `ponder` | `if` statement | | `otherwise` | `else` clause | | `cycle` | `while` loop | | `iterate` | `for-each` loop (iterate x through list) | | `through` | Used with iterate | | `within` | `for-in` loop (within x be list) | | `yeet` | `break` — exit loop | | `skip` | `continue` — skip iteration | | `attempt` | `try` block | | `rescue` | `catch` block | | `yep` | Boolean `true` | | `nope` | Boolean `false` | | `void` | Null / absent value | | `also` | Logical AND | | `either` | Logical OR | | `isnt` | Logical NOT | | `equals` | Equality comparison | | `differs` | Inequality comparison | | `essence` | Define a class | | `extend` | Class inheritance | | `self` | Instance reference inside class methods | | `super` | Parent class reference | | `new` | Create class instance | | `summon` | Import from Gist | | `async` | Mark function as async | | `await` | Wait for async operation | | `spawn` | Run function concurrently |

## Special Symbols

| Symbol | Purpose | |--------|---------| | `::` | Start a block | | `;;` | End a block | | `->` | Lambda arrow | | `\|>` | Pipe operator | | `~` | Ternary condition operator | | `:` | Ternary else / dict key-value separator | | `//` | Line comment | | `#`

| Line comment (Python style) | | ^ | Power operator | | <> | Inequality (alternative to differs) |

## Built-in Constants

| Constant | Value |
|---|---|
| PI | 3.141592653589793 |
| TAU | 6.283185307179586 |
| E | 2.718281828459045 |
| INFINITY | Infinity |

## Quick Reference Examples

```
// ── Variables ──
forge x be 42
forge name be "Alice"
forge active be yep

// ── Arithmetic ──
speak(2 ^ 10)     // 1024
speak(10 % 3)     // 1
speak(-5 + 8)     // 3

// ── String ──
speak("Hello" + " " + "World")
speak(upper("hello"))        // HELLO
speak(measure("hello"))      // 5

// ── List ──
forge nums be [1, 2, 3, 4, 5]
gather(nums, 6)              // [1,2,3,4,5,6]
speak(nums[-1])              // 6
speak(sort(nums))            // [1,2,3,4,5,6]

// ── Conditionals ──
ponder x &gt; 0 ::
  speak("positive")
;; otherwise ::
```

```
  speak("non-positive")
;;

// ── Ternary ──
forge label be x &gt; 0 ~ "pos" : "neg"

// ── Loop ──
within i be sequence(5) :: speak(i) ;;  // 0 1 2 3 4
cycle x &gt; 0 :: x be x - 1 ;;

// ── Function ──
conjure add(a, b) :: yield a + b ;;
forge double be x -&gt; x * 2

// ── Pipe ──
[1,2,3,4,5] |&gt; each(x -&gt; x * 2) |&gt; sift(x -&gt; x &gt; 4)
// [6, 8, 10]

// ── OOP ──
essence Dog ::
  conjure init(self, name) :: self.name be name ;;
  conjure bark(self) :: speak(self.name + ": WOOF!") ;;
;;
forge rex be new Dog("Rex")
rex.bark()

// ── Error Handling ──
attempt ::
  forge n be morph("not a number", "number")
;; rescue err ::
  speak("Error:", err)
;;

// ── Higher-order ──
forge evens be sift(sequence(10), x -&gt; x % 2 equals 0)
forge sum be fold(evens, 0, (a, b) -&gt; a + b)
speak(sum)   // 20

// ── Collections ──
```

```
forge s be Set()
s.add(1) s.add(2) s.add(2)
speak(s.values())   // [1, 2]

forge q be Queue()
q.enqueue("a") q.enqueue("b")
speak(q.dequeue())   // a

// ── Matrix ──
forge a be [[1,2],[3,4]]
forge b be [[5,6],[7,8]]
speak(matmul(a, b))   // [[19,22],[43,50]]

// ── Graphics ──
canvas(400, 400)
clear("#111")
fill(hue(120))
circle(200, 200, 100)

// ── Turtle ──
turtle()
within i be sequence(4) ::
  forward(100)
  right(90)
;;
```

*This documentation covers sdev version 1.x. For the latest updates and additional examples, visit the sdev IDE.*

CHAPTER 04

# Part II — The UI Toolkit

# Part II — The UI Toolkit

This part of the book documents every UI primitive added since the first edition. The toolkit is shared between the JavaScript runtime and the Python runtime; programs using only the documented API will run unchanged on either.

## Chapter U1 — Windows in depth

A **window** is the top-level container for any visual program. Every widget lives inside one. You open a window with `window(title, width, height)` and close it with `endwindow`.

```
window("Hello", 480, 320)
  heading("Hello, world!", 1)
  paragraph("This is my first window.")
endwindow
```

### Sizing

The two numeric arguments are the **initial** width and height in CSS pixels. Both runtimes treat them as a *suggestion*: the browser will respect them but allow the

user to resize the window with the mouse, and the Python runtime will apply them as the initial inner dimensions of the Tk window.

Common sizes you might use:

| Use case | Suggested size | |----------|----------------| | Pop-up dialog | `320 × 200` | | Form / settings | `480 × 600` | | Editor panel | `720 × 540` | | Dashboard | `960 × 640` | | Full-screen app | `1280 × 800` | | Tiny utility | `280 × 160` | | Mobile-style | `360 × 640` |

## Multiple windows

You can open more than one window in the same program. Each `window(...)` block is independent; widgets in window B cannot see widgets in window A unless they share an `uiget`/`uiset` name.

```
window("Editor", 720, 540)
  textarea("buffer", 24)
endwindow

window("Preview", 480, 540)
  paragraph(uiget("buffer"))
endwindow
```

## Closing a window programmatically

Call `closewindow(title)` to dismiss a window from code. To re-open the same window, simply re-execute the `window(...)/endwindow` block — it is a no-op if a window with the same title is already open.

### Window-level customisation

After the size argument you may pass an **options tome**:

```
window("My App", 480, 600, {
  resizable: true,
  alwaysOnTop: false,
  background: "#0F172A",
  foreground: "#F8FAFC",
  padding: 16,
  radius: 12,
  centered: true
})
```

All keys are optional. The runtime quietly ignores keys it does not recognise so your program stays portable.

## Chapter U2 — Buttons, in depth

`button(label, onclick)` is the most-used widget in any application. The default form gives you a primary-coloured button that calls a function when clicked.

```
button("Click me", () -&gt; speak("clicked!"))
```

### Variants

The optional third argument selects a visual variant:

| Variant | Looks like | Use for | |---------|----------|---------| | `primary` | Solid accent colour, white text | The main action of a screen. | | `secondary` | Muted background, dark text | Supporting actions next to a primary. | | `ghost` | Transparent until hover | Tertiary actions, toolbars. | | `outline` | Border only | Filter toggles, segmented controls. | | `destructive` | Red background | Delete, leave, or any irreversible action. | | `link` | Looks like a hyperlink | Navigation inside flowing text. |

## Customisation tome

A fourth argument lets you override almost anything:

```
button("Save", saveFn, "primary", {
  width: 160,
  height: 44,
  radius: 10,
  background: "#0891B2",
  foreground: "#FFFFFF",
  hoverBackground: "#0E7490",
  shadow: "0 4px 12px rgba(0,0,0,.25)",
  icon: "save",
  url: "https://example.com",
  newTab: true,
  disabled: false,
  tooltip: "Saves the current document"
})
```

If `url` is supplied the button behaves like a link — clicking it opens the URL (in a new tab when `newTab` is true) **after** the `onclick` function has run. This makes

buttons useful for both side-effects and navigation.

### Icon buttons

Pass `icon: "<name>"` to put a Lucide-style icon to the left of the label. A common pattern:

```
row()
  button("", deleteRow, "ghost", {icon: "trash", tooltip: "Delete"
  button("", editRow,   "ghost", {icon: "edit",  tooltip: "Edit"})
endrow
```

### Disabled state

Set `disabled: true` to grey-out the button. Combine with a state variable to lock the UI during long operations:

```
button("Submit", onSubmit, "primary", {disabled: uiget("loading")}
```

## Chapter U3 — Text inputs and forms

Every input widget takes a `name` as its first argument. The current value of the input is available anywhere via `uiget(name)`, and writing `uiset(name, value)` updates the widget.

### `input` — single line

```
input("email", "you@example.com")
button("Greet", () -&gt; speak("Hi " + uiget("email")))
```

Customisation:

```
input("email", "you@example.com", "", {
  width: 320,
  type: "email",     // text | email | password | number | tel | u
  maxLength: 80,
  required: true,
  autocomplete: "email",
  background: "#0F172A",
  foreground: "#E2E8F0",
  border: "#334155",
  radius: 8,
  padding: 10
})
```

## `textarea` — multi-line

```
textarea("note", 6, "Write something...")
```

A fourth options argument supports `width`, `height`, `resizable`, `monospace`, `wrap` (`hard` or `soft`).

### `checkbox`

```
checkbox("agree", "I agree to the terms")
if uiget("agree")
  speak("Thanks for agreeing")
end
```

Both runtimes treat the second argument as the visible label. If you want a checkbox without a label (e.g. inside a table cell) pass an empty string.

### `slider` and `select`

```
slider("volume", 0, 100, 5)
select("theme", ["light", "dark", "auto"])
```

Both widgets accept the same customisation tome as `input` (width, padding, colours).

# Chapter U4 — Layout primitives

By default widgets stack vertically, top to bottom. To change this you wrap children in a layout block.

## Rows and columns

```
row()
  button("Cancel", cancelFn, "ghost")
  button("Save",   saveFn,   "primary")
endrow
```

`row()` accepts an options tome: `gap`, `align` (`start`, `center`, `end`, `stretch`), `justify` (`start`, `center`, `end`, `between`, `around`), `wrap`.

## Groups

A `group(title)` block draws a labelled border around its children — perfect for grouping related form fields.

```
group("Account")
  input("name", "Your name")
  input("email", "you@example.com")
endgroup
```

## Tabs

```
tabs()
  tab("General")
    input("name", "Project name")
  endtab
  tab("Advanced")
    checkbox("experimental", "Enable experimental features")
  endtab
endtabs
```

## Tables

```
table(
  ["Name", "Age", "Country"],
  [
    ["Ada",   36, "UK"],
    ["Linus", 54, "FI"]
  ]
)
```

Options include `striped`, `bordered`, `compact`, `onRowClick(idx)`.

# Chapter U5 — Reactive state with `uiget` / `uiset`

Every widget that has a `name` exposes its current value through `uiget(name)`. The value is reactive: any expression in the same window that reads it will re-render when the value changes.

```
input("first", "Ada")
input("last",  "Lovelace")
paragraph("Hello, " + uiget("first") + " " + uiget("last") + "!")
```

To programmatically change a value, call `uiset(name, value)`. This is how you implement reset buttons, computed fields, or remote data loaders:

```
button("Reset", () -&gt; uiset("first", "") and uiset("last", ""),
```

Bound names are scoped to the **runtime session**, not the window. Two windows that use the same name share state — useful for editor + preview pairs.

# Chapter U6 — Customisation reference

This chapter is a single big table you can refer back to. Each row is a key you may pass in the options tome of any widget.

| Key | Type | Applies to | Notes |
|-----|------|------------|-------|
| `width` | number (px) | all | Fixed width. |
| `height` | number (px) | all | Fixed height. |
| `minWidth/minHeight` | number | all | Lower bound. |

`maxWidth/maxHeight` | number | all | Upper bound. | | `padding` | number or [t,r,b,l] | containers, buttons, inputs | Inner spacing. | | `margin` | number or [t,r,b,l] | all | Outer spacing. | | `radius` | number (px) | most | Corner radius. | | `background` | colour string | most | Solid background colour. | | `foreground` | colour string | text widgets | Text colour. | | `border` | colour string | most | Border colour. | | `borderWidth` | number (px) | most | Border thickness. | | `shadow` | CSS shadow string | window, button, group | Drop shadow. | | `font` | string | text widgets | CSS font-family. | | `fontSize` | number (px) | text widgets | Font size. | | `fontWeight` | number | text widgets | Boldness. | | `letterSpacing` | number (em) | text widgets | | | `align` | start|center|end|stretch | row, column, group | Cross-axis. | | `justify` | start|center|end|between|around | row, column | Main-axis. | | `gap` | number (px) | row, column, group | Space between children. | | `wrap` | true|false | row | Allow wrap to new line. | | `icon` | icon name | button, menuitem | Lucide icon name. | | `tooltip` | string | most | Hover help. | | `disabled` | boolean | inputs, buttons | Greys out and ignores events. | | `hidden` | boolean | all | Removes from layout when true. | | `url` | string | button, label | Click navigates to URL. | | `newTab` | boolean | button, label | Open url in a new tab. | | `onChange` | function | inputs | Fired on every value change. | | `onFocus/onBlur` | function | inputs | | | `onMount/onUnmount` | function | all | Lifecycle hooks. | | `hoverBackground` | colour string | button | | | `activeBackground` | colour string | button | | | `focusRing` | colour string | inputs,

buttons | | | `placeholderColor` | colour string | inputs, textarea | | | `rows/cols` | number | textarea | Initial size. | | `resizable` | true|false | window, textarea | | | `monospace` | boolean | textarea, label | Use a monospace font. | | `wrap` | "hard"|"soft" | textarea | Line-wrapping policy. |

# Chapter U7 — Cloud workspace & auto-sync

From the second edition onward, the IDE keeps your entire workspace — files **and** the nested folder tree — in the cloud. You don't have to think about it.

## How auto-sync works

Every change you make in the editor is debounced (≈ 800 ms) and written to the cloud as a small patch. When you reload the page, the IDE pulls the latest snapshot before painting any UI, so what you see is always what was last saved.

If two browser tabs edit the same file at once, the **last write wins** at the file level, but the loser's changes are kept in a sibling file named `<original>.conflict-<timestamp>`. Open it, copy what you need, and delete it.

## Manual saves

Open the **File** menu and choose **Save to cloud** (Ctrl + S) to flush pending writes immediately. This is also the action to use right before publishing or sharing a project.

## Working offline

If the network goes down the IDE keeps working against an in-memory copy of the workspace. A small banner in the status bar warns you. As soon as the connection comes back, every queued change is pushed in order.

## Nested folders

The file tree is a real tree. Right-click any folder for **New file**, **New folder**, **Rename**, **Delete**, **Duplicate**, **Move…**. Drag-and-drop works between any two folders.

Programs can also touch the workspace at runtime through `readFile(path)` and `writeFile(path, content)`. Paths are POSIX-style and rooted at the workspace root.

CHAPTER 05

# Widget Reference Cards

# Widget Reference Cards

One card per widget, in alphabetical order. Each card has the signature, a one-line description, and a short example.

### `audio`

**Signature.** `audio(src)`

**Description.** Audio player widget.

**Example.**

```
audio(src)
```

### `button`

**Signature.** `button(label, onclick, variant?)`

**Description.** Clickable button. Variants: `primary`, `secondary`, `ghost`, `destructive`, `link`.

**Example.**

```
button(label, onclick, variant?)
```

### `canvas`

**Signature.** `canvas(name, w, h)`

**Description.** Drawable canvas surface inside a window.

**Example.**

```
canvas(name, w, h)
```

### `checkbox`

**Signature.** `checkbox(name, label)`

**Description.** Boolean toggle. State at `uiget(name)`.

**Example.**

```
checkbox(name, label)
```

### `column`

**Signature.** `column() ... endcolumn`

**Description.** Lay children vertically.

**Example.**

```
column() ... endcolumn
```

## confirm

**Signature.** `confirm(msg, fn)`

**Description.** Show a confirm dialog and run `fn` if accepted.

**Example.**

```
confirm(msg, fn)
```

## divider

**Signature.** `divider()`

**Description.** Horizontal rule between widgets.

**Example.**

```
divider()
```

## endwindow

**Signature.** `endwindow`

**Description.** Close the current window block.

**Example.**

```
endwindow
```

### `group`

**Signature.** `group(title) ... endgroup`

**Description.** Bordered group with optional title.

**Example.**

```
group(title) ... endgroup
```

### `heading`

**Signature.** `heading(text, level=1..6)`

**Description.** Headline text inside a window.

**Example.**

```
heading(text, level=1..6)
```

### `image`

**Signature.** `image(src, w?, h?, alt?)`

**Description.** Render an image, local or remote URL.

**Example.**

```
image(src, w?, h?, alt?)
```

## `input`

**Signature.** `input(name, placeholder?, value?)`

**Description.** Single-line text input. Value is bound to `uiget(name)`.

**Example.**

```
input(name, placeholder?, value?)
```

## `label`

**Signature.** `label(text)`

**Description.** Inline label, no margin.

**Example.**

```
label(text)
```

## `map`

**Signature.** `map(lat, lng, zoom)`

**Description.** Embedded Leaflet map.

**Example.**

```
map(lat, lng, zoom)
```

## `menu`

**Signature.** `menu(title) ... endmenu / menuitem(t,fn)`

**Description.** Top-of-window menu bar.

**Example.**

```
menu(title) ... endmenu / menuitem(t,fn)
```

## `paragraph`

**Signature.** `paragraph(text)`

**Description.** Block of body text.

**Example.**

```
paragraph(text)
```

## `progress`

**Signature.** `progress(value, max?)`

**Description.** Progress bar.

**Example.**

```
progress(value, max?)
```

## `prompt`

**Signature.** `prompt(msg, fn)`

**Description.** Show a prompt dialog and pass the typed string to `fn`.

**Example.**

```
prompt(msg, fn)
```

## `radio`

**Signature.** `radio(name, options)`

**Description.** Mutually exclusive choice list.

**Example.**

```
radio(name, options)
```

### `row`

**Signature.** `row() ... endrow`

**Description.** Lay children horizontally.

**Example.**

```
row() ... endrow
```

### `select`

**Signature.** `select(name, options)`

**Description.** Drop-down selector.

**Example.**

```
select(name, options)
```

### `slider`

**Signature.** `slider(name, min, max, step?)`

**Description.** Numeric range slider.

**Example.**

```
slider(name, min, max, step?)
```

### `spacer`

**Signature.** `spacer(px)`

**Description.** Empty vertical gap.

**Example.**

```
spacer(px)
```

### `switch`

**Signature.** `switch(name, label)`

**Description.** On/off switch.

**Example.**

```
switch(name, label)
```

### `table`

**Signature.** `table(headers, rows)`

**Description.** Data table.

**Example.**

```
table(headers, rows)
```

### `tabs`

**Signature.** `tabs() ... endtabs / tab(t)..endtab`

**Description.** Tabbed container with multiple panels.

**Example.**

```
tabs() ... endtabs / tab(t)..endtab
```

### `textarea`

**Signature.** `textarea(name, rows?, placeholder?)`

**Description.** Multi-line text input.

**Example.**

```
textarea(name, rows?, placeholder?)
```

### `toast`

**Signature.** `toast(msg, kind?)`

**Description.** Show a non-blocking notification.

**Example.**

```
toast(msg, kind?)
```

### `uiget`

**Signature.** `uiget(name)`

**Description.** Read the bound value of a widget.

**Example.**

```
uiget(name)
```

### `uiset`

**Signature.** `uiset(name, value)`

**Description.** Programmatically write a widget value.

**Example.**

```
uiset(name, value)
```

### `video`

**Signature.** `video(src, w?, h?)`

**Description.** Embedded video player.

**Example.**

```
video(src, w?, h?)
```

## `window`

**Signature.** `window(title, width, height)`

**Description.** Open a top-level application window. Closed with `endwindow`.

**Example.**

```
window(title, width, height)
```

CHAPTER 06

# Part III — The Cookbook

# Part III — The Cookbook

Fifteen step-by-step tutorials. Each one is short enough to type by hand in 15–30 minutes and large enough to teach a real lesson.

## Tutorial 1 — Hello, Window

**Goal.** Open a window, show a heading and a paragraph, react to a button.

### Step 1 — The empty window

```
window("Hello", 360, 240)
endwindow
```

Run with **Ctrl + Enter**. A 360×240 window appears. It is empty because we haven't put anything inside it.

### Step 2 — Add some content

```
window("Hello", 360, 240)
  heading("Welcome to sdev", 1)
  paragraph("This is the smallest possible windowed program.")
endwindow
```

## Step 3 — A button that does something

```
forge clicks be 0

window("Hello", 360, 280)
  heading("Welcome to sdev", 1)
  paragraph("Click the button below.")
  button("Click me", () -&gt; {
    clicks = clicks + 1
    speak("clicked " + clicks + " times")
  }, "primary")
endwindow
```

`speak(...)` writes to the OUTPUT panel. Try clicking the button several times and watch the counter climb.

## Step 4 — Show the count inside the window

Reactive bindings make this easy:

```
window("Hello", 360, 280)
  heading("Welcome to sdev", 1)
  paragraph("You have clicked " + uiget("count") + " times.")
  button("Click me", () -&gt; uiset("count", (uiget("count") or 0)
endwindow
```

## What you learned

- `window(...) ... endwindow` opens a window.
- `heading`, `paragraph`, `button` are widgets.
- `uiget` and `uiset` give every widget reactive state.

# Tutorial 2 — A Tip Calculator

**Goal.** Use a slider, a number input, and a computed paragraph.

```
window("Tip Calculator", 380, 320)
  heading("Tip calculator", 2)

  group("Bill")
    input("amount", "0.00", "20.00", {type: "number", width: 160})
  endgroup

  group("Tip")
    slider("tip", 0, 30, 1)
    paragraph("Tip: " + uiget("tip") + "%")
  endgroup

  group("Result")
    forge total be elevate(uiget("amount") * (1 + uiget("tip")/100
    heading("Total: $" + total, 3)
  endgroup
endwindow
```

`elevate(x, n)` rounds to $n$ decimal places. The result is fully reactive: move the slider, change the bill — everything updates immediately.

## Stretch goal

Add a `select("people", ["1","2","3","4","5"])` and divide the total by the selected number of people.

# Tutorial 3 — A To-Do List

**Goal.** Render a list, add items, remove items, persist to local storage.

```
forge todos be []

conjure addTodo():
  forge text be uiget("new")
  if text != ""
    todos = todos + [text]
    uiset("new", "")
    writeFile("todos.json", json.stringify(todos))
  end
end

conjure removeTodo(idx):
  todos = todos.removeAt(idx)
  writeFile("todos.json", json.stringify(todos))
end

# load on startup
attempt
  todos = json.parse(readFile("todos.json"))
rescue _
  todos = []
end

window("Todo", 420, 540)
  heading("My todos", 2)

  row({gap: 8})
    input("new", "What needs doing?", "", {width: 280})
    button("Add", addTodo, "primary")
  endrow

  divider()

  for i in range(todos.length)
    row({gap: 8, align: "center"})
```

```
      paragraph((i+1) + ". " + todos[i])
      button("", () -&gt; removeTodo(i), "ghost", {icon: "trash"})
    endrow
  end
endwindow
```

## What you learned

- Lists are first-class; `+ [text]` appends.
- `writeFile` and `readFile` persist to the workspace (and the cloud).
- A `for` loop is a perfectly good way to render a dynamic list.

# Tutorial 4 — A Note-Taking App

**Goal.** Multiple notes, switch between them, save automatically.

```
forge notes be tome()
forge current be "Untitled"

attempt
  notes = json.parse(readFile("notes.json"))
rescue _
  notes["Untitled"] = ""
end

conjure save():
  notes[current] = uiget("body")
  writeFile("notes.json", json.stringify(notes))
```

```
end

conjure switchTo(name):
  current = name
  uiset("body", notes[name])
end

window("Notes", 720, 540)
  row({gap: 0})
    column({width: 200, background: "#0F172A", padding: 12})
      heading("Notes", 4)
      for name in keys(notes)
        button(name, () -&gt; switchTo(name), "ghost", {width: 176
      end
      button("+ New", () -&gt; {
        notes["Note " + (keys(notes).length + 1)] = ""
        save()
      }, "primary", {width: 176})
    endcolumn

    column({padding: 12})
      heading(current, 3)
      textarea("body", 18, "Write here...", {onChange: save})
    endcolumn
  endrow
endwindow
```

`onChange: save` runs the save function on every keystroke. Combined with the cloud auto-sync this means nothing is ever lost.

# Tutorial 5 — A Drawing Pad

**Goal.** Use the `canvas` widget, react to mouse events, save as PNG.

```
window("Draw", 600, 480)
  heading("Drawing pad", 3)
  row({gap: 8})
    select("color", ["black","red","green","blue","orange","purple
    slider("size", 1, 20, 1)
    button("Clear", () -&gt; canvasClear("pad"), "ghost")
    button("Save PNG", () -&gt; canvasExport("pad", "drawing.png")
  endrow
  canvas("pad", 580, 380, {
    background: "#FFFFFF",
    border: "#CBD5E1",
    onMouseDrag: (x,y) -&gt; canvasDot("pad", x, y, uiget("size"),
  })
endwindow
```

`canvasDot`, `canvasClear`, `canvasExport` are all part of the standard graphics module. The full reference is in Chapter 22 of Part I.

# Tutorial 6 — A Pomodoro Timer

**Goal.** Tabs, progress bars, audio cues, settings panel.

```
forge running be false
forge remaining be 25 * 60
forge totalLen be 25 * 60

conjure tick():
  if running and remaining &gt; 0
    remaining = remaining - 1
    uiset("progress", (totalLen - remaining) / totalLen * 100)
    after(1000, tick)
  elif remaining &lt;= 0
```

```
      audio("done.mp3")
      toast("Time's up!", "success")
      running = false
    end
  end

  window("Pomodoro", 360, 360)
    tabs()
      tab("Timer")
        heading(formatTime(remaining), 1)
        progress(uiget("progress") or 0, 100)
        row({gap: 8})
          button("Start", () -&gt; { running = true ; tick() }, "pri
          button("Pause", () -&gt; { running = false }, "secondary")
          button("Reset", () -&gt; { running = false ; remaining = t
        endrow
      endtab
      tab("Settings")
        slider("len", 5, 60, 5)
        button("Apply", () -&gt; {
          totalLen = uiget("len") * 60
          remaining = totalLen
        }, "primary")
      endtab
    endtabs
  endwindow
```

`after(ms, fn)` is a built-in scheduler that runs `fn` after `ms` milliseconds without blocking the UI.

# Tutorial 7 — A Markdown Previewer

**Goal.** Side-by-side textarea + rendered preview, in 25 lines of code.

```
window("Markdown Previewer", 900, 600)
  row({gap: 0})
    column({width: 440, padding: 8})
      heading("Source", 4)
      textarea("md", 28, "# Hello\n\nType **markdown** here.", {mon
    endcolumn
    column({padding: 16, background: "#FFFFFF"})
      heading("Preview", 4)
      html(renderMarkdown(uiget("md") or ""))
    endcolumn
  endrow
endwindow
```

`html(...)` injects rendered HTML into the window. `renderMarkdown(s)` is in the standard library and produces safe, sanitised HTML.

# Tutorial 8 — A Weather Dashboard

**Goal.** Make a real HTTP request, parse JSON, display the result.

```
conjure load():
  forge city be uiget("city")
  forge resp be await http.get("https://api.example.com/weather?q=
  forge data be json.parse(resp.body)
  uiset("temp",     data.temp)
  uiset("humidity", data.humidity)
  uiset("desc",     data.description)
```

```
end

window("Weather", 380, 360)
  row({gap: 8})
    input("city", "City", "Sofia")
    button("Load", load, "primary")
  endrow
  group("Now")
    heading(uiget("temp") + "°C", 1)
    paragraph(uiget("desc") or "—")
    paragraph("Humidity: " + (uiget("humidity") or "—") + "%")
  endgroup
endwindow
```

`http.get` returns a future; `await` blocks the **fiber**, not the UI, so the window stays responsive.

# Tutorial 9 — A Map Explorer

**Goal.** Show a Leaflet map, add markers, search.

```
forge places be [
  ["Sofia", 42.6977, 23.3219],
  ["Plovdiv", 42.1354, 24.7453],
  ["Varna", 43.2141, 27.9147]
]

window("Map", 720, 520)
  input("q", "Search a place")
  map("m", 42.7, 25.0, 7, {width: 700, height: 420})
  for p in places
    marker("m", p[1], p[2], p[0])
  end
  button("Find", () -&gt; {
    forge q be lower(uiget("q"))
    for p in places
```

```
      if contains(lower(p[0]), q)
        flyTo("m", p[1], p[2], 11)
        escape
      end
    end
  }, "primary")
endwindow
```

The map widget is documented in full in Appendix A.

# Tutorial 10 — A Mini Spreadsheet

**Goal.** Editable table with a tiny formula evaluator.

```
forge cells be matrix(8, 5, () -&gt; "")

conjure setCell(r, c, v):
  cells[r][c] = v
end

conjure render(v):
  if startsWith(v, "=")
    return eval(substring(v, 1))
  end
  return v
end

window("Sheet", 640, 420)
  heading("Mini sheet", 3)
  for r in range(8)
    row({gap: 4})
      for c in range(5)
        input("c" + r + "_" + c, "", cells[r][c], {
          width: 110,
```

```
          onChange: () -&gt; setCell(r, c, uiget("c" + r + "_" + c
        })
      end
    endrow
  end
  divider()
  paragraph("Sum of A1: " + render(cells[0][0]))
endwindow
```

# Tutorial 11 — A Chat Window

**Goal.** Sticky scroll, async streaming, message bubbles.

```
forge messages be []

conjure send():
  forge msg be uiget("input")
  if msg == "" then return end
  messages = messages + [{from: "me", text: msg}]
  uiset("input", "")
  await stream("https://api.example.com/chat", msg, (chunk) -&gt; 
    if last(messages).from != "bot"
      messages = messages + [{from: "bot", text: ""}]
    end
    last(messages).text = last(messages).text + chunk
  })
end

window("Chat", 480, 600)
  scrollarea({stickToBottom: true, height: 480})
    for m in messages
      paragraph(m.text, {
        background: m.from == "me" ? "#0891B2" : "#1E293B",
        foreground: "#FFFFFF",
        padding: 10,
        radius: 12,
```

```
        align: m.from == "me" ? "end" : "start"
      })
    end
  endscrollarea
  row({gap: 8})
    input("input", "Say something…")
    button("Send", send, "primary")
  endrow
endwindow
```

# Tutorial 12 — A File Manager

**Goal.** Render the workspace folder tree, expand on click, drag & drop.

```
conjure renderNode(node, depth):
  for child in node.children
    row({gap: 4, padding: depth * 12})
      if child.type == "folder"
        button("▯ " + child.name, () -&gt; child.expanded = not chi
      else
        button("▯ " + child.name, () -&gt; openFile(child.path), "
      end
    endrow
    if child.type == "folder" and child.expanded
      renderNode(child, depth + 1)
    end
  end
end

window("Files", 360, 600)
  heading("Workspace", 3)
  forge tree be readWorkspaceTree()
  renderNode(tree, 0)
endwindow
```

`readWorkspaceTree()` returns the live, nested folder tree synchronised with the cloud.

# Tutorial 13 — A Calculator

**Goal.** Grid layout, button variants, keyboard shortcuts.

```
forge expr be ""

conjure press(key):
  if key == "C" then expr = ""
  elif key == "=" then expr = "" + eval(expr)
  else expr = expr + key end
  uiset("display", expr)
end

window("Calc", 280, 360)
  paragraph(uiget("display") or "0", {
    background: "#0F172A", foreground: "#F8FAFC",
    padding: 14, radius: 8, align: "end", monospace: true
  })
  for row in [["7","8","9","/"],["4","5","6","*"],["1","2","3","-"
    row({gap: 6})
      for k in row
        button(k, () -&gt; press(k), k == "=" ? "primary" : "secon
               {width: 56, height: 48})
      end
    endrow
  end
  button("Clear", () -&gt; press("C"), "destructive", {width: 240}
endwindow
```

# Tutorial 14 — A Bouncing Ball Game

**Goal.** Animation loop on a canvas with collision detection.

```
forge x be 100; forge y be 100
forge vx be 3;  forge vy be 2

conjure frame():
  canvasClear("g", "#0F172A")
  canvasCircle("g", x, y, 16, "#0891B2")
  x = x + vx ; y = y + vy
  if x &lt; 16 or x &gt; 484 then vx = -vx end
  if y &lt; 16 or y &gt; 284 then vy = -vy end
  after(16, frame)
end

window("Bounce", 520, 340)
  canvas("g", 500, 300)
  frame()
endwindow
```

60 frames per second with `after(16, frame)`. Replace the body of `frame` with your own physics for a real game.

# Tutorial 15 — Your First Library

**Goal.** Package useful code as a Gist and `summon` it from any program.

## Step 1 — Write the library

Create a file `mylib.sdev` with one helper:

```
conjure greet(who):
  return "Hello, " + who + "!"
end
```

## Step 2 — Publish as a public Gist

In the IDE, **File → Publish as Gist**. Copy the URL — it looks like `https://gist.github.com/<user>/<hash>`.

## Step 3 — Use it from another program

```
summon "https://gist.github.com/&lt;user&gt;/&lt;hash&gt;" as mylib

speak(mylib.greet("world"))
```

`summon` downloads the Gist on first use, caches it locally, and re-uses the cache on subsequent runs. Append `?v=2` to the URL to bust the cache when you publish a new version.

## Where to go next

You have now seen every part of the language and the toolkit. Pick a project that excites you, open the IDE, and start typing. The full reference for every function used in these tutorials is one click away in the Web IDE under **Help → Documentation** or in Part I of this book.

CHAPTER 07

# Appendix A — Maps & GIS (Leaflet)

# Appendix A — Maps & GIS (Leaflet)

Full reference for the mapping subsystem.

CHAPTER 08

# sdev Leaflet Documentation

# sdev Leaflet Documentation

## Interactive Maps for sdev

The sdev Leaflet module provides powerful geographic mapping capabilities, allowing you to create interactive maps, markers, shapes, and more using the familiar sdev syntax.

## Table of Contents

1. Getting Started
2. Map Creation
3. Markers & Popups
4. Shapes
5. Polylines & Polygons
6. Layers
7. Events
8. Controls
9. GeoJSON
10. Utilities
11. Complete Examples

# Getting Started

## Basic Setup

To use Leaflet features in sdev, first create a map container:

```
// Create a map centered on coordinates with zoom level
forge myMap be createMap("map-container", 51.505, -0.09, 13)
```

## HTML Setup

Your HTML page needs a container div and Leaflet CSS/JS:

```
&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
    &lt;link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4
    &lt;script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js
    &lt;script src="sdev-interpreter.js"&gt;&lt;/script&gt;
    &lt;style&gt;
        #map-container { height: 500px; width: 100%; }
    &lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
    &lt;div id="map-container"&gt;&lt;/div&gt;
    &lt;script&gt;
        const interpreter = new SdevInterpreter();
        interpreter.run(`
            forge map be createMap("map-container", 51.505, -0.09,
            addMarker(map, 51.505, -0.09, "Hello from sdev!")
        `);
    &lt;/script&gt;
&lt;/body&gt;
```

```
&lt;/html&gt;
```

# Map Creation

## createMap(containerId, lat, lng, zoom)

Creates a new Leaflet map instance.

| Parameter | Type | Description | |---------|---|---|---| | containerId | text | ID of the HTML container element | | lat | number | Initial latitude center | | lng | number | Initial longitude center | | zoom | number | Initial zoom level (1-18) |

```
// Create a map of London
forge londonMap be createMap("map", 51.505, -0.09, 13)

// Create a map of New York
forge nyMap be createMap("nyc-map", 40.7128, -74.0060, 12)
```

## setMapView(map, lat, lng, zoom)

Changes the map's center and zoom level.

```
forge map be createMap("map", 0, 0, 2)

// Pan to Paris
setMapView(map, 48.8566, 2.3522, 14)
```

## getMapCenter(map)

Returns the current center coordinates as a tome (dictionary).

```
forge center be getMapCenter(map)
speak("Lat: " + morph(center["lat"], "text"))
speak("Lng: " + morph(center["lng"], "text"))
```

### getMapZoom(map)

Returns the current zoom level.

```
forge zoom be getMapZoom(map)
speak("Current zoom: " + morph(zoom, "text"))
```

### getMapBounds(map)

Returns the visible map bounds.

```
forge bounds be getMapBounds(map)
// bounds contains: north, south, east, west
```

## Markers & Popups

### addMarker(map, lat, lng, popupText?)

Adds a marker to the map with an optional popup.

```
forge map be createMap("map", 51.505, -0.09, 13)

// Simple marker
forge marker1 be addMarker(map, 51.505, -0.09)
```

```
// Marker with popup
forge marker2 be addMarker(map, 51.51, -0.08, "Click me!")
```

## addMarkerIcon(map, lat, lng, iconUrl, iconSize, popupText?)

Adds a marker with a custom icon.

| Parameter | Type | Description |
|----------|------|----------| | iconUrl | text | URL to the icon image | | iconSize | list | [width, height] in pixels |

```
forge map be createMap("map", 51.505, -0.09, 13)

forge customMarker be addMarkerIcon(
    map,
    51.505,
    -0.09,
    "https://example.com/pin.png",
    [32, 32],
    "Custom icon marker!"
)
```

## removeMarker(map, marker)

Removes a marker from the map.

```
forge marker be addMarker(map, 51.505, -0.09, "Temporary")
// Later...
removeMarker(map, marker)
```

## setMarkerPosition(marker, lat, lng)

Moves an existing marker to new coordinates.

```
forge marker be addMarker(map, 51.505, -0.09)

// Animate marker movement
forge i be 0
cycle i &lt; 100 ::
    setMarkerPosition(marker, 51.505 + i * 0.001, -0.09 + i * 0.00
    delay(50)
    i be i + 1
;;
```

## bindPopup(marker, content)

Attaches a popup to an existing marker.

```
forge marker be addMarker(map, 51.505, -0.09)
bindPopup(marker, "&lt;b&gt;Bold text!&lt;/b&gt;&lt;br&gt;HTML wor
```

## bindTooltip(marker, content)

Attaches a tooltip (shows on hover) to a marker.

```
forge marker be addMarker(map, 51.505, -0.09)
bindTooltip(marker, "Hover tooltip")
```

## openPopup(marker)

Programmatically opens the marker's popup.

```
forge marker be addMarker(map, 51.505, -0.09, "Hello!")
openPopup(marker)
```

# Shapes

## addCircle(map, lat, lng, radius, options?)

Adds a circle to the map.

| Parameter | Type | Description |
|----------|------|----------|
| radius | number | Radius in meters |
| options | tome | Style options (optional) |

```
forge map be createMap("map", 51.505, -0.09, 13)

// Simple circle
forge circle1 be addCircle(map, 51.505, -0.09, 500)

// Styled circle
forge options be ::
    "color": "#ff0000",
    "fillColor": "#ff6666",
    "fillOpacity": 0.5,
    "weight": 2
;;
forge circle2 be addCircle(map, 51.51, -0.08, 300, options)
```

## addCircleMarker(map, lat, lng, radius, options?)

Adds a circle marker (radius in pixels, not meters).

```
forge dot be addCircleMarker(map, 51.505, -0.09, 10, ::
    "color": "#3388ff",
    "fillColor": "#3388ff",
    "fillOpacity": 0.8
```

```
;;)
```

### addRectangle(map, lat1, lng1, lat2, lng2, options?)

Adds a rectangle defined by opposite corners.

```
forge rect be addRectangle(
    map,
    51.49, -0.10,  // Southwest corner
    51.52, -0.06,  // Northeast corner
    :: "color": "#ff7800", "weight": 1 ;;
)
```

## Polylines & Polygons

### addPolyline(map, points, options?)

Draws a line through multiple points.

| Parameter | Type | Description | |----------|------|----------| | points | list | List of [lat, lng] coordinate pairs | | options | tome | Style options |

```
forge map be createMap("map", 51.505, -0.09, 13)

forge route be [
    [51.505, -0.09],
    [51.51, -0.08],
    [51.52, -0.06],
    [51.515, -0.05]
]
```

```
forge line be addPolyline(map, route, ::
    "color": "#ff0000",
    "weight": 4,
    "opacity": 0.8,
    "dashArray": "10, 10"
;;)
```

## addPolygon(map, points, options?)

Creates a closed polygon shape.

```
forge triangle be [
    [51.509, -0.08],
    [51.503, -0.06],
    [51.51, -0.047]
]

forge poly be addPolygon(map, triangle, ::
    "color": "#00ff00",
    "fillColor": "#00ff88",
    "fillOpacity": 0.4
;;)
```

## addMultiPolygon(map, polygons, options?)

Creates multiple polygons as a single layer.

```
forge shapes be [
    [[51.51, -0.12], [51.51, -0.10], [51.52, -0.10], [51.52, -0.12
    [[51.51, -0.08], [51.51, -0.06], [51.52, -0.06], [51.52, -0.08
]

forge multiPoly be addMultiPolygon(map, shapes, ::
    "color": "#9900ff"
;;)
```

# Layers

## addTileLayer(map, urlTemplate, options?)

Adds a custom tile layer (base map).

```
forge map be createMap("map", 51.505, -0.09, 13)

// OpenStreetMap (default)
addTileLayer(map, "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.
    "attribution": "© OpenStreetMap contributors"
;;)

// Satellite imagery
addTileLayer(map, "https://server.arcgisonline.com/ArcGIS/rest/serv
    "attribution": "© Esri"
;;)
```

## createLayerGroup()

Creates an empty layer group for organizing layers.

```
forge markers be createLayerGroup()
addMarker(markers, 51.505, -0.09, "Marker 1")
addMarker(markers, 51.51, -0.08, "Marker 2")
addLayerToMap(map, markers)
```

## addLayerToMap(map, layer)

Adds a layer or layer group to the map.

```
forge group be createLayerGroup()
// Add items to group...
addLayerToMap(map, group)
```

### removeLayerFromMap(map, layer)

Removes a layer from the map.

```
removeLayerFromMap(map, markers)
```

### clearLayer(layer)

Removes all items from a layer group.

```
clearLayer(markers)
```

## Events

### onMapClick(map, callback)

Handles map click events.

```
forge map be createMap("map", 51.505, -0.09, 13)

onMapClick(map, (event) -&gt; ::
    forge lat be event["lat"]
    forge lng be event["lng"]
    addMarker(map, lat, lng, "Clicked at " + morph(lat, "text") +
;;)
```

## onMapZoom(map, callback)

Handles zoom changes.

```
onMapZoom(map, (event) -&gt; ::
    forge zoom be getMapZoom(map)
    speak("Zoom changed to: " + morph(zoom, "text"))
;;)
```

## onMapMove(map, callback)

Handles map movement (pan).

```
onMapMove(map, (event) -&gt; ::
    forge center be getMapCenter(map)
    speak("Map moved to: " + morph(center["lat"], "text") + ", " +
;;)
```

## onMarkerClick(marker, callback)

Handles marker click events.

```
forge marker be addMarker(map, 51.505, -0.09)

onMarkerClick(marker, (event) -&gt; ::
    speak("Marker was clicked!")
;;)
```

## onMarkerDrag(marker, callback)

Handles marker drag events (marker must be draggable).

```
forge marker be addMarker(map, 51.505, -0.09)
```

```
setMarkerDraggable(marker, yep)

onMarkerDrag(marker, (event) -&gt; ::
    forge pos be getMarkerPosition(marker)
    speak("Dragged to: " + morph(pos["lat"], "text"))
;;)
```

# Controls

## addZoomControl(map, position?)

Adds zoom controls to the map.

| Position | Description | |---------|----------| | "topleft" | Top left corner | | "topright" | Top right corner | | "bottomleft" | Bottom left corner | | "bottomright" | Bottom right corner |

```
addZoomControl(map, "bottomright")
```

## addScaleControl(map, options?)

Adds a scale indicator.

```
addScaleControl(map, ::
    "position": "bottomleft",
    "metric": yep,
    "imperial": nope
;;)
```

## addAttributionControl(map, prefix?)

Adds attribution text.

```
addAttributionControl(map, "Powered by sdev")
```

### addLayerControl(map, baseLayers, overlays)

Adds a layer switcher control.

```
forge osm be addTileLayer(map, "https://{s}.tile.openstreetmap.org/
forge satellite be addTileLayer(map, "https://server.arcgisonline.c

forge bases be ::
    "Streets": osm,
    "Satellite": satellite
;;

forge markers be createLayerGroup()
forge overlays be ::
    "Markers": markers
;;

addLayerControl(map, bases, overlays)
```

## GeoJSON

### addGeoJSON(map, geoJsonData, options?)

Adds GeoJSON data to the map.

```
forge geojson be ::
```

```
    "type": "FeatureCollection",
    "features": [
        ::
            "type": "Feature",
            "geometry": ::
                "type": "Point",
                "coordinates": [-0.09, 51.505]
            ;;,
            "properties": ::
                "name": "London"
            ;;
        ;;
    ]
;;

forge layer be addGeoJSON(map, geojson, ::
    "style": ::
        "color": "#ff0000",
        "weight": 2
    ;;
;;)
```

## geoJSONStyle(feature)

Custom style function for GeoJSON features.

```
forge layer be addGeoJSON(map, geojson, ::
    "style": (feature) -&gt; ::
        ponder feature["properties"]["type"] equals "road" ::
            yield :: "color": "#888888", "weight": 3 ;;
        ;;
        otherwise ::
            yield :: "color": "#00ff00", "weight": 1 ;;
        ;;
    ;;
;;)
```

# Utilities

## latLng(lat, lng)

Creates a coordinate object.

```
forge coord be latLng(51.505, -0.09)
speak("Latitude: " + morph(coord["lat"], "text"))
```

## distance(lat1, lng1, lat2, lng2)

Calculates distance between two points in meters.

```
forge dist be distance(51.505, -0.09, 51.51, -0.08)
speak("Distance: " + morph(dist, "text") + " meters")
```

## boundsContains(bounds, lat, lng)

Checks if a point is within bounds.

```
forge bounds be getMapBounds(map)
forge inside be boundsContains(bounds, 51.505, -0.09)
ponder inside ::
    speak("Point is visible on map")
;;
```

## fitBounds(map, lat1, lng1, lat2, lng2)

Adjusts the map view to fit the given bounds.

```
fitBounds(map, 51.49, -0.12, 51.52, -0.05)
```

### invalidateSize(map)

Recalculates map size (use after container resize).

```
invalidateSize(map)
```

## Navigation & Animation

### panTo(map, lat, lng, options?)

Smoothly pans the map to a new center.

```
panTo(map, 48.8566, 2.3522)
```

### panBy(map, x, y)

Pans the map by a given number of pixels.

```
panBy(map, 100, 50)  // Pan right 100px, down 50px
```

### flyTo(map, lat, lng, zoom, options?)

Animates the map to a new position with a smooth flying effect.

```
flyTo(map, 40.7128, -74.0060, 14)  // Fly to New York
```

### flyToBounds(map, lat1, lng1, lat2, lng2, options?)

Animates to fit the given bounds.

```
flyToBounds(map, 51.49, -0.12, 51.52, -0.05)
```

## zoomIn(map, delta?)

Increases the zoom level by delta (default 1).

```
zoomIn(map)        // Zoom in by 1
zoomIn(map, 2)     // Zoom in by 2
```

## zoomOut(map, delta?)

Decreases the zoom level by delta (default 1).

```
zoomOut(map)
```

## setZoom(map, zoom)

Sets the zoom level directly.

```
setZoom(map, 15)
```

## setMinZoom(map, zoom) / setMaxZoom(map, zoom)

Sets zoom constraints.

```
setMinZoom(map, 5)
setMaxZoom(map, 18)
```

### getMinZoom(map) / getMaxZoom(map)

Gets the current zoom constraints.

```
forge minZ be getMinZoom(map)
forge maxZ be getMaxZoom(map)
```

### setMaxBounds(map, lat1, lng1, lat2, lng2)

Restricts the map to a given geographical area.

```
setMaxBounds(map, 51.0, -0.5, 52.0, 0.5)  // Lock to London area
```

## Popup & Tooltip Control

### closePopup(marker)

Closes an open popup.

```
closePopup(marker)
```

### openTooltip(marker) / closeTooltip(marker)

Opens or closes a marker's tooltip.

```
openTooltip(marker)
closeTooltip(marker)
```

### setPopupContent(marker, content)

Updates popup content dynamically.

```
setPopupContent(marker, "Updated content!")
```

### setTooltipContent(marker, content)

Updates tooltip content dynamically.

```
setTooltipContent(marker, "New tooltip text")
```

## Layer Styling

### setMarkerIcon(marker, iconUrl, iconSize)

Changes a marker's icon.

```
setMarkerIcon(marker, "https://example.com/new-icon.png", [32, 32]
```

### setMarkerOpacity(marker, opacity)

Sets marker transparency (0-1).

```
setMarkerOpacity(marker, 0.5)
```

### setMarkerZIndex(marker, zIndex)

Controls marker stacking order.

```
setMarkerZIndex(marker, 1000)
```

## setCircleRadius(circle, radius)

Updates a circle's radius in meters.

```
setCircleRadius(myCircle, 750)
```

## setCircleStyle(circle, options) / setPolylineStyle / setPolygonStyle

Updates styling of shapes.

```
setCircleStyle(myCircle, :: "color": "#ff0000", "fillOpacity": 0.8
setPolylineStyle(myLine, :: "color": "#00ff00", "weight": 5 ;;)
setPolygonStyle(myPoly, :: "fillColor": "#0000ff" ;;)
```

## getPolylineLatLngs(polyline)

Gets all points of a polyline.

```
forge points be getPolylineLatLngs(myRoute)
```

## setPolylineLatLngs(polyline, points)

Replaces all points of a polyline.

```
setPolylineLatLngs(myRoute, [[51.5, -0.1], [51.6, -0.05]])
```

### addLatLng(polyline, lat, lng)

Adds a point to a polyline.

```
addLatLng(myRoute, 51.52, -0.08)
```

### bringToFront(layer) / bringToBack(layer)

Controls layer z-ordering.

```
bringToFront(importantLayer)
bringToBack(backgroundLayer)
```

## Additional Events

### onLayerClick(layer, callback)

Handles click events on any layer.

```
onLayerClick(myCircle, (e) -&gt; ::
    speak("Circle clicked at " + morph(e["lat"], "text"))
;;)
```

### onLayerMouseover(layer, callback) / onLayerMouseout(layer, callback)

Handles hover events.

```
onLayerMouseover(myPolygon, (e) -&gt; ::
```

```
    setPolygonStyle(myPolygon, :: "fillOpacity": 0.8 ;;)
;;)

onLayerMouseout(myPolygon, (e) -&gt; ::
    setPolygonStyle(myPolygon, :: "fillOpacity": 0.4 ;;)
;;)
```

### onMapDoubleClick(map, callback)

Handles double-click events.

```
onMapDoubleClick(map, (e) -&gt; ::
    setMapView(map, e["lat"], e["lng"], getMapZoom(map) + 1)
;;)
```

### onMapRightClick(map, callback)

Handles right-click (context menu) events.

```
onMapRightClick(map, (e) -&gt; ::
    speak("Right click at " + morph(e["lat"], "text"))
;;)
```

### onMapMousemove(map, callback)

Tracks mouse movement over the map.

```
onMapMousemove(map, (e) -&gt; ::
    updateCoordinateDisplay(e["lat"], e["lng"])
;;)
```

## Geolocation

### locate(map, options?)

Starts browser geolocation.

```
locate(map, :: "setView": yep, "maxZoom": 16 ;;)
```

### onLocationFound(map, callback)

Handles successful location.

```
onLocationFound(map, (e) -&gt; ::
    speak("Found you at " + morph(e["lat"], "text") + ", " + morph
    addMarker(map, e["lat"], e["lng"], "You are here!")
;;)
```

### onLocationError(map, callback)

Handles geolocation errors.

```
onLocationError(map, (e) -&gt; ::
    speak("Location error: " + e["message"])
;;)
```

### stopLocate(map)

Stops continuous location updates.

```
stopLocate(map)
```

## Overlays

## addImageOverlay(map, imageUrl, lat1, lng1, lat2, lng2, options?)

Adds an image overlay to the map.

```
forge overlay be addImageOverlay(map, "historical-map.jpg", 51.4,
```

## addVideoOverlay(map, videoUrl, lat1, lng1, lat2, lng2, options?)

Adds a video overlay.

```
forge video be addVideoOverlay(map, "timelapse.mp4", 51.4, -0.2, 5
```

## setImageOpacity(overlay, opacity)

Sets overlay transparency.

```
setImageOpacity(overlay, 0.7)
```

## setImageUrl(overlay, url)

Changes the overlay image.

```
setImageUrl(overlay, "new-image.jpg")
```

## setBounds(overlay, lat1, lng1, lat2, lng2)

Repositions an overlay.

```
setBounds(overlay, 51.3, -0.3, 51.7, 0.2)
```

# Feature Groups

## createFeatureGroup()

Creates a feature group (like layer group but with bounds).

```
forge group be createFeatureGroup()
```

## addToFeatureGroup(featureGroup, layer)

Adds a layer to the feature group.

```
addToFeatureGroup(group, myMarker)
addToFeatureGroup(group, myCircle)
```

## removeFromFeatureGroup(featureGroup, layer)

Removes a layer from the feature group.

```
removeFromFeatureGroup(group, myMarker)
```

## getFeatureGroupBounds(featureGroup)

Gets the combined bounds of all layers.

```
forge bounds be getFeatureGroupBounds(group)
```

## fitFeatureGroup(map, featureGroup, options?)

Zooms the map to fit all layers in the group.

```
fitFeatureGroup(map, group)
```

## eachLayer(layerGroup, callback)

Iterates over all layers in a group.

```
eachLayer(group, (layer) -&gt; ::
    speak("Processing layer")
;;)
```

## getLayers(layerGroup)

Returns all layers as a list.

```
forge layers be getLayers(group)
```

## hasLayer(layerGroup, layer)

Checks if a layer exists in the group.

```
ponder hasLayer(group, myMarker) ::
    speak("Marker is in group")
;;
```

# Custom Markers

## addDivIcon(map, lat, lng, html, className, size)

Creates a marker with custom HTML content.

```
forge customMarker be addDivIcon(map, 51.505, -0.09, "&lt;div class
```

# Coordinate Utilities

## getSize(map)

Gets map container size in pixels.

```
forge size be getSize(map)
speak("Width: " + morph(size["width"], "text") + ", Height: " + mo
```

## latLngToContainerPoint(map, lat, lng)

Converts coordinates to pixel position.

```
forge pixel be latLngToContainerPoint(map, 51.505, -0.09)
speak("Pixel X: " + morph(pixel["x"], "text"))
```

## containerPointToLatLng(map, x, y)

Converts pixel position to coordinates.

```
forge coord be containerPointToLatLng(map, 200, 150)
```

## wrapLng(lng)

Wraps longitude to -180 to 180.

```
forge wrapped be wrapLng(370)  // Returns 10
```

## wrapLat(lat)

Clamps latitude to -90 to 90.

```
forge clamped be wrapLat(95)  // Returns 90
```

## degreesToDMS(degrees)

Converts decimal degrees to DMS string.

```
forge dms be degreesToDMS(51.5074)  // "51° 30' 26.64""
```

## DMSToDegrees(d, m, s)

Converts DMS to decimal degrees.

```
forge deg be DMSToDegrees(51, 30, 26.64)  // 51.5074
```

## metersToPixels(map, meters, lat)

Converts meters to pixels at the current zoom.

```
forge px be metersToPixels(map, 1000, 51.505)
```

### pixelsToMeters(map, pixels, lat)

Converts pixels to meters at the current zoom.

```
forge m be pixelsToMeters(map, 100, 51.505)
```

## GIS Analysis Functions

### bearing(lat1, lng1, lat2, lng2)

Calculates bearing between two points (0-360 degrees).

```
forge b be bearing(51.5, -0.1, 48.8, 2.3)  // London to Paris
speak("Bearing: " + morph(b, "text") + "°")
```

### midpoint(lat1, lng1, lat2, lng2)

Calculates the midpoint between two coordinates.

```
forge mid be midpoint(51.5, -0.1, 48.8, 2.3)
addMarker(map, mid["lat"], mid["lng"], "Midpoint")
```

### destination(lat, lng, bearing, distance)

Calculates destination point given bearing and distance.

```
forge dest be destination(51.5, -0.1, 90, 50000)  // 50km east of l
addMarker(map, dest["lat"], dest["lng"], "Destination")
```

## area(points)

Calculates polygon area in square meters.

```
forge polygon be [[51.5, -0.1], [51.5, 0.0], [51.4, 0.0], [51.4, -0
forge sqMeters be area(polygon)
speak("Area: " + morph(ground(sqMeters / 1000000), "text") + " km²
```

## length(points)

Calculates polyline length in meters.

```
forge route be [[51.5, -0.1], [51.52, -0.08], [51.55, -0.05]]
forge len be length(route)
speak("Route length: " + morph(ground(len), "text") + " meters")
```

## centroid(points)

Calculates the center point of a polygon.

```
forge center be centroid(polygon)
addMarker(map, center["lat"], center["lng"], "Center")
```

## isPointInPolygon(lat, lng, points)

Checks if a point is inside a polygon.

```
ponder isPointInPolygon(51.45, -0.05, polygon) ::
    speak("Point is inside the polygon")
;; otherwise ::
    speak("Point is outside")
;;
```

### simplify(points, tolerance)

Simplifies a polyline using Douglas-Peucker algorithm.

```
forge simplified be simplify(complexRoute, 0.0001)
```

### interpolateAlong(points, fraction)

Gets a point at a fraction (0-1) along a polyline.

```
forge halfway be interpolateAlong(route, 0.5)
addMarker(map, halfway["lat"], halfway["lng"], "Halfway point")
```

## Layer Visibility

### getLayerType(layer)

Gets the type of a layer.

```
forge type be getLayerType(myLayer)  // "marker", "circle", etc.
```

### isLayerVisible(layer)

Checks if a layer is currently on the map.

```
ponder isLayerVisible(myMarker) ::
    speak("Marker is visible")
;;
```

### showLayer(map, layer) / hideLayer(map, layer)

Shows or hides a layer.

```
hideLayer(map, secretMarker)
// Later...
showLayer(map, secretMarker)
```

### toggleLayer(map, layer)

Toggles layer visibility.

```
toggleLayer(map, myLayer)
```

## Complete Examples

### Interactive City Markers

```
// Create a world map with major cities
forge map be createMap("map", 20, 0, 2)

// City data
forge cities be [
    :: "name": "London", "lat": 51.505, "lng": -0.09 ;;,
    :: "name": "Paris", "lat": 48.8566, "lng": 2.3522 ;;,
    :: "name": "New York", "lat": 40.7128, "lng": -74.0060 ;;,
    :: "name": "Tokyo", "lat": 35.6762, "lng": 139.6503 ;;,
    :: "name": "Sydney", "lat": -33.8688, "lng": 151.2093 ;;
]

// Add markers for each city
```

```
each(cities, (city) -&gt; ::
    forge marker be addMarker(map, city["lat"], city["lng"], city[
    bindTooltip(marker, city["name"])
;;)

// Click to add new markers
onMapClick(map, (e) -&gt; ::
    addMarker(map, e["lat"], e["lng"], "New Location")
;;)
```

## Route Visualization

```
// Visualize a hiking route
forge map be createMap("map", 51.505, -0.09, 14)

forge trailPoints be [
    [51.500, -0.10],
    [51.502, -0.095],
    [51.505, -0.09],
    [51.508, -0.085],
    [51.510, -0.08],
    [51.512, -0.075]
]

// Draw the route
forge trail be addPolyline(map, trailPoints, ::
    "color": "#e74c3c",
    "weight": 5,
    "opacity": 0.8
;;)

// Add start and end markers
addMarker(map, 51.500, -0.10, "□ Start")
addMarker(map, 51.512, -0.075, "□ Finish")

// Add distance markers along the way
forge totalDist be 0
forge i be 1
```

```
cycle i &lt; measure(trailPoints) ::
    forge prev be pluck(trailPoints, i - 1)
    forge curr be pluck(trailPoints, i)
    forge dist be distance(prev[0], prev[1], curr[0], curr[1])
    totalDist be totalDist + dist

    addCircleMarker(map, curr[0], curr[1], 5, ::
        "color": "#3498db",
        "fillColor": "#3498db",
        "fillOpacity": 1
    ;;)
    i be i + 1
;;

speak("Total distance: " + morph(ground(totalDist), "text") + " met
```

## Heatmap Zones

```
// Create density visualization with circles
forge map be createMap("map", 51.505, -0.09, 13)

// Data points with intensity
forge hotspots be [
    :: "lat": 51.505, "lng": -0.09, "intensity": 100 ;;,
    :: "lat": 51.510, "lng": -0.08, "intensity": 75 ;;,
    :: "lat": 51.500, "lng": -0.10, "intensity": 50 ;;,
    :: "lat": 51.508, "lng": -0.095, "intensity": 90 ;;
]

// Create gradient circles for each hotspot
each(hotspots, (spot) -&gt; ::
    // Outer glow
    addCircle(map, spot["lat"], spot["lng"], spot["intensity"] * 5
        "color": "transparent",
        "fillColor": "#ff6600",
        "fillOpacity": 0.2
    ;;)
```

```
    // Inner core
    addCircle(map, spot["lat"], spot["lng"], spot["intensity"] * 2
        "color": "transparent",
        "fillColor": "#ff0000",
        "fillOpacity": 0.5
    ;;)
;;)
```

## Layer Toggle System

```
// Multi-layer map with toggle controls
forge map be createMap("map", 51.505, -0.09, 13)

// Create layer groups
forge restaurants be createLayerGroup()
forge hotels be createLayerGroup()
forge attractions be createLayerGroup()

// Add restaurant markers
addMarker(restaurants, 51.505, -0.09, "Pizza Place")
addMarker(restaurants, 51.508, -0.085, "Sushi Bar")
addMarker(restaurants, 51.502, -0.095, "Burger Joint")

// Add hotel markers
addMarker(hotels, 51.510, -0.08, "Grand Hotel")
addMarker(hotels, 51.500, -0.10, "Budget Inn")

// Add attraction markers
addMarker(attractions, 51.507, -0.09, "Museum")
addMarker(attractions, 51.503, -0.07, "Park")

// Add all layers to map
addLayerToMap(map, restaurants)
addLayerToMap(map, hotels)
addLayerToMap(map, attractions)

// Create layer control
forge overlays be ::
```

```
    "□ Restaurants": restaurants,
    "□ Hotels": hotels,
    "□ Attractions": attractions
;;

addLayerControl(map, ::;;, overlays)
```

## Animated Marker

```
// Animate a marker along a path
forge map be createMap("map", 51.505, -0.09, 14)

forge path be [
    [51.500, -0.10],
    [51.502, -0.095],
    [51.505, -0.09],
    [51.508, -0.085],
    [51.510, -0.08]
]

// Draw the path
addPolyline(map, path, :: "color": "#3498db", "weight": 3, "dashAr

// Create moving marker
forge mover be addMarker(map, path[0][0], path[0][1], "□")

// Animate along path
conjure animateMarker(marker, points, index) ::
    ponder index &gt;= measure(points) ::
        yield void
    ;;

    forge target be pluck(points, index)
    setMarkerPosition(marker, target[0], target[1])

    delay(500)
    animateMarker(marker, points, index + 1)
;;
```

```
animateMarker(mover, path, 0)
```

# Style Reference

## Common Style Options

| Property | Type | Description | |----------|------|----------| | color | text | Stroke color (hex or name) | | weight | number | Stroke width in pixels | | opacity | number | Stroke opacity (0-1) | | fillColor | text | Fill color | | fillOpacity | number | Fill opacity (0-1) | | dashArray | text | Stroke dash pattern | | lineCap | text | Line cap style | | lineJoin | text | Line join style |

## Icon Options

| Property | Type | Description | |----------|------|----------| | iconUrl | text | URL to icon image | | iconSize | list | [width, height] | | iconAnchor | list | [x, y] anchor point | | popupAnchor | list | [x, y] popup offset | | shadowUrl | text | URL to shadow image |

# Tips & Best Practices

1. **Performance**: Use layer groups for many markers
2. **Memory**: Remove unused layers with `removeLayerFromMap`

3. **Mobile**: Use `invalidateSize` after orientation changes
4. **Clustering**: For 100+ markers, consider marker clustering
5. **Tile Caching**: Custom tile layers can be cached for offline use

# Error Handling

```
attempt ::
    forge map be createMap("nonexistent-id", 0, 0, 10)
;; rescue error ::
    speak("Failed to create map: " + error)
;;
```

*sdev Leaflet Module — Mapping made magical* □□

CHAPTER 09

# Appendix B — Glossary

# Appendix B — Glossary

A quick lookup of every keyword and built-in name introduced in this book.

| Symbol | Meaning | |--------|---------| | `after(ms, fn)` | Schedule `fn` to run after `ms` milliseconds. | | `attempt … rescue` | sdev's exception handling block. | | `audio(src)` | Plays an audio file or URL. | | `button(label, onclick, variant?, options?)` | A clickable widget. | | `canvas(name, w, h, options?)` | A 2D drawing surface. | | `checkbox(name, label, options?)` | A boolean toggle. | | `conjure` | Defines a function. | | `elevate(n, k)` | Rounds `n` to `k` decimal places. | | `endwindow` | Closes the current window block. | | `escape` | Breaks out of the innermost loop. | | `essence` | Defines a class. | | `eternal` | Marks an immutable binding. | | `forge` | Declares a variable. | | `group(title) … endgroup` | A bordered group of widgets. | | `heading(text, level)` | A headline widget. | | `html(s)` | Injects raw, sanitised HTML into a window. | | `input(name, placeholder, value, options?)` | A single-line text input. | | `json.parse(s) / json.stringify(v)` | JSON conversion. | | `map(name, lat, lng, zoom, options?)` | A Leaflet map widget. | | `marker(map, lat, lng, label?)` | Adds a marker to a map. | | `matrix(rows, cols, fillFn)` | Creates a matrix. | | `menu(title) … endmenu / menuitem` | A menu bar. | | `paragraph(text, options?)` | A block of body

text. | | `progress(value, max?)` | A progress bar. | | `range(n)` | A list 0..n-1. | | `readFile(path) / writeFile(path, content)` | Workspace file I/O. | | `row() … endrow / column() … endcolumn` | Layout primitives. | | `select(name, options, options?)` | A drop-down selector. | | `slider(name, min, max, step?)` | A numeric range widget. | | `speak(x)` | Writes `x` to the OUTPUT panel. | | `summon URL as name` | Imports a Gist module. | | `table(headers, rows, options?)` | A data table. | | `tabs() … endtabs / tab(t) … endtab` | A tabbed container. | | `textarea(name, rows, placeholder, options?)` | A multi-line input. | | `toast(msg, kind?)` | A non-blocking notification. | | `tome()` | Creates an empty dictionary. | | `uiget(name) / uiset(name, value)` | Reactive widget state. | | `window(title, w, h, options?)` | Opens a top-level window. |

CHAPTER 10

# Appendix C — Index of tutorials

# Appendix C — Index of tutorials

| # | Title | Teaches | |---|-------|---------| | 1 | Hello, Window | Build your very first windowed application. | | 2 | A Tip Calculator | Inputs, sliders, live computation, and a result panel. | | 3 | A To-Do List | State, lists, removing items, persisting to local storage. | | 4 | A Note-Taking App | Textareas, multiple notes, save & load through the cloud. | | 5 | A Drawing Pad | Canvas widget, mouse events, color picker, save as PNG. | | 6 | A Pomodoro Timer | Tabs, progress bars, audio cues, settings panel. | | 7 | A Markdown Previewer | Side-by-side textarea + rendered preview. | | 8 | A Weather Dashboard | HTTP requests, JSON parsing, displaying remote data. | | 9 | A Map Explorer | Leaflet map with markers, tooltips, and a search box. | | 10 | A Mini Spreadsheet | Tables, in-place editing, formula evaluation. | | 11 | A Chat Window | Sticky scroll, async streaming, message bubbles. | | 12 | A File Manager | Nested folder tree, drag & drop, cloud sync. | | 13 | A Calculator | Grid layout, button variants, keyboard shortcuts. | | 14 | A Bouncing Ball Game | Canvas + animation loop with collisions. | | 15 | Your First Library | Package code as a Gist and `summon` it from anywhere. |

# Colophon

This book was generated from the sdev source repository on 2026-05-03. The body is set in DejaVu Serif, headings in DejaVu Sans Bold, code in DejaVu Sans Mono. The PDF is rendered with ReportLab. Source markdown lives in the project's `public/` directory.

CHAPTER 11

# Appendix F — Style guide

# Appendix F — Style guide

Notes on writing readable sdev:

## 1. Use forge for state

Avoid hidden globals; declare with forge near use.

## 2. Name widgets

Always pass a stable name to inputs so uiget works after refresh.

## 3. Group with group()

Borders aid scanning of long forms.

## 4. Prefer ghost over destructive

Reserve red for irreversible actions.

## 5. Cap line length

Wrap at 100 columns to keep diffs readable.

## 6. One window per file

Easier to review, easier to ship as a Gist.

## 7. Avoid eval

Use a tome lookup instead of eval whenever possible.

## 8. Comment intent

// what + why beats // how.

## 9. Use after for animation

setInterval-style loops can starve the UI.

## 10. Cache uiget

Read once into a local forge if you use it 3+ times in a render.

## 11. Keep functions small

If a conjure is over 40 lines, split it.

## 12. Lift state up

Two windows that mirror data should share a uiget name.

## 13. Tabs over deep nesting

Reach for tabs() before piling more groups.

## 14. Test in both runtimes

JS and Python should behave identically — if not, file a bug.

## 15. Document with paragraph()

A short paragraph at the top of every window beats a README.

CHAPTER 12

# sdev Programming Language

# sdev Programming Language

## Complete Documentation, Tutorial & Reference Guide

## Table of Contents

1. Introduction
2. Getting Started
3. Syntax Overview
4. Variables & Constants
5. Data Types
6. Operators
7. String Operations
8. Control Flow
9. Functions
10. Object-Oriented Programming
11. Error Handling
12. Async & Concurrency
13. Built-in Output Functions
14. Built-in Math Functions

15. Built-in String Functions
16. Built-in List Functions
17. Built-in Tome (Dict) Functions
18. Higher-Order Functions
19. Type System & Conversion
20. Collections: Set, Map, Queue, Stack, LinkedList
21. Matrix Operations
22. Graphics & Game Development
23. File I/O
24. Networking
25. Examples & Recipes
26. JavaScript Interop
27. Complete Reference Card

# Introduction

**sdev** is an expressive, full-featured programming language with a deliberately unique syntax designed to feel fresh and readable. It supports:

- **Unique English-inspired keywords** — `forge`, `conjure`, `ponder`, `cycle`, `iterate`, `yield`, `yeet`, `skip`
- **Block delimiters** — `::` to open a block and `;;` to close it — no curly braces
- **First-class functions and lambdas** — `(x) -> x * 2`
- **Pipe operator** — `|>` for clean functional composition

- **Full OOP** — `essence` (classes), `extend` (inheritance), `self`, `super`, `new`
- **Error handling** — `attempt :: ... ;; rescue err :: ... ;;`
- **Built-in data structures** — Set, Map, Queue, Stack, LinkedList
- **Matrix math** — `matmul`, `transpose`, `dot`, `reshape`, etc.
- **2D Graphics & Turtle API** — canvas drawing, shapes, gradients, turtle graphics, sprites
- **Async / concurrency** — `async conjure`, `await`, `spawn`
- **JavaScript interop** — `js` keyword for seamless JS integration

sdev runs in the browser-based IDE (at `/ide`), in the Playground (`/`), and as a downloadable Electron desktop application.

# Getting Started

## Running in the IDE

Open the sdev IDE at `/ide`. Press **Ctrl+Enter** (or the **Run** button) to execute your program. Output appears in the **OUTPUT** panel at the bottom.

## Running in the Playground

Go to the main Playground at `/`. Type or paste sdev code in the editor, then click **Run**.

## Downloading the Desktop App

In the IDE, go to the **Download** button → **Electron Desktop App**. This gives you three files:

```
package.json
main.js
README.md
```

Install and run:

```
npm install
npm start
```

This opens sdev IDE as a native desktop window that wraps the full web IDE.

## Your First Program

```
// hello.sdev
speak("Hello, World!")
```

Output:

```
Hello, World!
```

A slightly more involved first program:

```
forge name be "Alice"
forge age be 30

speak("Name:", name)
```

```
speak("Age:", age)
speak("In 10 years:", age + 10)

conjure greet(person, greeting) ::
  yield greeting + ", " + person + "!"
;;

speak(greet(name, "Welcome"))
```

Output:

```
Name: Alice
Age: 30
In 10 years: 40
Welcome, Alice!
```

# Syntax Overview

## Comments

```
// This is a single-line comment
# This is also a single-line comment (Python style)
```

## Blocks

sdev uses `::` to start a block and `;;` to end it — never curly braces `{}`:

```
ponder x &gt; 5 ::
  speak("x is big")
;;

conjure add(a, b) ::
```

```
    yield a + b
  ;;
```

Blocks can also be written inline for short single-statement bodies:

```
ponder x &gt; 5 :: speak("big") ;;
```

### No Semicolons Required

Statements are separated by newlines. You do NOT need `;` at the end of lines.

```
forge a be 1
forge b be 2
forge c be a + b
speak(c)
```

### Indentation

Indentation is cosmetic but strongly recommended. The language uses `::` and `;;` for block structure, not whitespace.

## Variables & Constants

### Declaring Variables

Use `forge` to declare a new variable and `be` to assign its value:

```
forge name be "Alice"
forge age be 25
forge isActive be yep
forge score be 0.0
```

## Reassigning Variables

Just use `be` without `forge`:

```
forge count be 0
count be count + 1
count be 10
```

## Compound Assignment (shorthand patterns)

```
forge x be 10
x be x + 5    // x = 15
x be x * 2    // x = 30
x be x - 10   // x = 20
x be x / 4    // x = 5
```

## Built-in Constants

sdev provides these constants directly:

```
speak(PI)        // 3.141592653589793
speak(TAU)       // 6.283185307179586 (2 * PI)
speak(E)         // 2.718281828459045
speak(INFINITY)  // Infinity
```

## Multiple Variables

```
forge x be 10
forge y be 20
forge z be x + y

// Swap pattern
forge temp be x
x be y
y be temp
```

# Data Types

sdev has six core types:

| sdev Type | Description | Literal Example |
|----------|----------|----------|
| `number` | Integer or float | `42`, `3.14`, `-7`, `1.5e10` |
| `text` | Strings | `"hello"`, `'world'` |
| `truth` | Booleans | `yep`, `nope` |
| `void` | Null / absent | `void` |
| `list` | Arrays | `[1, 2, 3]` |
| `tome` | Dictionaries | `:: "key": "value" ;;` |

Functions and class instances are also first-class values.

## Numbers

```
forge integer be 42
forge float be 3.14159
forge negative be -100
forge scientific be 1.5e10      // 15,000,000,000
forge hex be 0xFF               // 255
```

Arithmetic is standard, with `^` for power:

```
speak(2 ^ 10)   // 1024
```

```
speak(10 % 3)   // 1
speak(7 / 2)    // 3.5
```

## Text (Strings)

```
forge single be 'Hello'
forge double be "World"

// Escape sequences
forge tab be "col1\tcol2"
forge newline be "line1\nline2"
forge quoted be "She said \"hi\""

// String concatenation
forge full be "Hello" + ", " + "World!"

// String repetition
forge stars be "*" * 10   // "**********"

// Access characters
speak("hello"[0])   // h
speak("hello"[-1])  // o (last character)
```

## Truth (Booleans)

```
forge t be yep    // true
forge f be nope   // false

// Boolean expressions
speak(5 &gt; 3)          // yep
speak(5 equals 3)     // nope
speak(yep also nope)  // nope
speak(yep either nope) // yep
speak(isnt nope)       // yep
```

## Void (Null)

```
forge nothing be void
ponder nothing equals void ::
  speak("It's void!")
;;
```

## Lists

```
forge nums be [1, 2, 3, 4, 5]
forge mixed be [1, "two", yep, void]
forge empty be []
forge nested be [[1, 2], [3, 4], [5, 6]]

// Access by index (0-based)
speak(nums[0])    // 1
speak(nums[2])    // 3
speak(nums[-1])   // 5  (last element)

// Modify element
nums[0] be 100
speak(nums)       // [100, 2, 3, 4, 5]

// List length
speak(measure(nums))   // 5

// Slicing
speak(portion(nums, 1, 3))  // [2, 3]
speak(portion(nums, 2))     // [3, 4, 5]
```

## Tomes (Dictionaries)

Tomes are key-value stores. Keys are strings.

```
forge person be ::
  "name": "Alice",
```

```
  "age": 30,
  "active": yep
;;

// Access values - both bracket and dot notation work
speak(person["name"])  // Alice
speak(person.age)      // 30

// Set new or existing keys
person["city"] be "New York"
person.age be 31

// Check if key exists
speak(contains(person, "name"))   // yep
speak(contains(person, "phone"))  // nope

// Get all keys, values, entries
speak(inscriptions(person))  // ["name", "age", "active", "city"]
speak(contents(person))      // ["Alice", 31, yep, "New York"]
speak(entries(person))       // [["name","Alice"], ...]
```

# Operators

## Arithmetic Operators

| Operator | Description | Example | Result |
|---|---|---|---|
| `+` | Addition | `5 + 3` | `8` |
| `-` | Subtraction | `10 - 4` | `6` |
| `*` | Multiplication | `4 * 3` | `12` |
| `/` | Division | `7 / 2` | `3.5` |
| `%` | Modulo | `10 % 3` | `1` |
| `^` | Power | `2 ^ 8` | `256` |
| `-x` | Negation | `-5` | `-5` |

## Comparison Operators

| Operator | Description | Example | Result |
|---------|-----------|---------|-------|
| `equals` | Equal to | `5 equals 5` | `yep` |
| `differs` | Not equal | `5 differs 3` | `yep` |
| `<>` | Not equal (alt) | `5 <> 3` | `yep` |
| `<` | Less than | `3 < 5` | `yep` |
| `>` | Greater than | `5 > 3` | `yep` |
| `<=` | Less or equal | `3 <= 3` | `yep` |
| `>=` | Greater or equal | `5 >= 5` | `yep` |

## Logical Operators

| Operator | Meaning | Example |
|---------|---------|---------|
| `also` | AND | `yep also yep` → `yep` |
| `either` | OR | `nope either yep` → `yep` |
| `isnt` | NOT | `isnt nope` → `yep` |

Short-circuit evaluation applies: `also` stops at first `nope`, `either` stops at first `yep`.

## String & List Concatenation

```
"Hello" + " World"    // "Hello World"
[1, 2] + [3, 4]       // [1, 2, 3, 4]
"ha" * 3              // "hahaha"
[0] * 5               // [0, 0, 0, 0, 0]
```

## Pipe Operator `|>`

The pipe operator passes the left-hand value as the first argument to the right-hand function:

```
// Without pipe
forge result be weave(sort(sift(each([1,2,3,4,5], x -&gt; x * 2), x

// With pipe — much cleaner!
```

```
forge result be [1, 2, 3, 4, 5]
  |&gt; each(x -&gt; x * 2)
  |&gt; sift(x -&gt; x &gt; 4)
  |&gt; weave(", ")
speak(result)  // "6, 8, 10"
```

## Ternary Operator ~

Inline if/else expression:

```
forge x be 10
forge label be x &gt; 5 ~ "big" : "small"
speak(label)  // "big"

// In function calls
speak(magnitude(-5) &gt; 3 ~ "large magnitude" : "small magnitude"

// Nested ternary
forge grade be 85
forge letter be grade &gt;= 90 ~ "A" : grade &gt;= 80 ~ "B" : grad
speak(letter)  // "B"
```

## Operator Precedence (high to low)

1. `^` (power)
2. Unary `-`, `isnt`
3. `*`, `/`, `%`
4. `+`, `-`
5. `<`, `>`, `<=`, `>=`
6. `equals`, `differs`, `<>`
7. `also`
8. `either`
9. `~` (ternary)

10. |> (pipe)

# String Operations

sdev has a rich string library. All string functions return new strings — strings are immutable.

```
forge s be "  Hello, World!  "

// Case conversion
speak(upper(s))          // "  HELLO, WORLD!  "
speak(lower(s))          // "  hello, world!  "

// Trimming
speak(trim(s))           // "Hello, World!"

// Length
speak(measure(s))        // 18

// Contains / starts / ends
speak(contains(s, "World"))      // yep
speak(startswith(trim(s), "Hello"))  // yep
speak(endswith(trim(s), "!"))        // yep

// Replace (replaces ALL occurrences)
speak(replace("banana", "a", "o"))   // "bonono"

// Find index
speak(locate("hello world", "world"))  // 6
speak(locate("hello", "xyz"))          // -1

// Split and join
forge parts be shatter("a,b,c,d", ",")
speak(parts)                 // ["a", "b", "c", "d"]
speak(weave(parts, " | "))   // "a | b | c | d"
```

```
// Character list
speak(chars("abc"))   // ["a", "b", "c"]

// Reverse
speak(reverse("hello"))  // "olleh"

// Padding
speak(padLeft("5", 4, "0"))    // "0005"
speak(padRight("hi", 6, "!"))  // "hi!!!!"

// Format strings (use {} as placeholders)
speak(format("Hello, {}! You are {} years old.", "Bob", 25))
// "Hello, Bob! You are 25 years old."

// Repeat
speak(repeat("ab", 3))   // "ababab"

// Substring
speak(snatch("hello world", 6, 11))  // "world"
```

# Control Flow

## If / Else — `ponder` / `otherwise`

```
forge score be 87

ponder score &gt;= 90 ::
  speak("Grade: A")
;; otherwise ponder score &gt;= 80 ::
  speak("Grade: B")
;; otherwise ponder score &gt;= 70 ::
  speak("Grade: C")
;; otherwise ponder score &gt;= 60 ::
  speak("Grade: D")
;; otherwise ::
```

```
  speak("Grade: F")
;;
```

Inline (for short bodies):

```
ponder x &gt; 0 :: speak("positive") ;; otherwise :: speak("non-pos
```

## While Loop — `cycle`

```
forge n be 1
cycle n &lt;= 10 ::
  speak(n)
  n be n + 1
;;

// Infinite loop with break
cycle yep ::
  forge x be randint(1, 10)
  speak("Got:", x)
  ponder x equals 7 ::
    speak("Found 7! Stopping.")
    yeet    // break
  ;;
;;
```

## For-Each Loop — `iterate through`

```
forge fruits be ["apple", "banana", "cherry"]

iterate fruit through fruits ::
  speak(fruit)
;;
// apple
// banana
// cherry
```

## For-In Loop — `within`

```
// Iterate over a list
within fruit be ["apple", "banana", "cherry"] ::
  speak(fruit)
;;

// Iterate over a range
within i be sequence(5) ::
  speak(i)   // 0 1 2 3 4
;;

within i be sequence(1, 11) ::
  speak(i)   // 1 2 3 ... 10
;;

within i be sequence(0, 20, 5) ::
  speak(i)   // 0 5 10 15
;;
```

## Loop Control

### `yeet` — Break

Exit the nearest enclosing loop immediately:

```
within i be sequence(100) ::
  ponder i &gt; 5 ::
    yeet
  ;;
  speak(i)
;;
// 0 1 2 3 4 5
```

### `skip` — Continue

Skip the rest of the current iteration and go to the next:

```
within i be sequence(10) ::
  ponder i % 2 equals 0 ::
    skip   // skip even numbers
  ;;
  speak(i)   // prints only odd: 1 3 5 7 9
;;
```

### Nested Loops

```
within i be sequence(1, 4) ::
  within j be sequence(1, 4) ::
    speak(i + " x " + j + " = " + morph(i * j, "text"))
  ;;
;;
```

# Functions

### Basic Functions — `conjure`

```
conjure greet(name) ::
  speak("Hello, " + name + "!")
;;

greet("Alice")
greet("World")
```

### Return Values — `yield`

```
conjure add(a, b) ::
  yield a + b
```

```
;;

conjure max(a, b) ::
  ponder a &gt; b :: yield a ;;
  yield b
;;

forge sum be add(10, 20)
speak(sum)         // 30
speak(max(5, 9))   // 9
```

## Default Parameters

```
conjure greet(name, greeting) ::
  ponder greeting equals void :: greeting be "Hello" ;;
  yield greeting + ", " + name + "!"
;;

speak(greet("Alice"))           // Hello, Alice!
speak(greet("Bob", "Welcome"))  // Welcome, Bob!
```

## Lambda Functions (Arrow Syntax)

Single expression, one parameter:

```
forge double be x -&gt; x * 2
forge square be x -&gt; x ^ 2
forge negate be x -&gt; -x

speak(double(5))   // 10
speak(square(4))   // 16
speak(negate(7))   // -7
```

Multiple parameters (parentheses required):

```
forge add be (a, b) -&gt; a + b
```

```
forge clamp be (v, lo, hi) -&gt; v &lt; lo ~ lo : v &gt; hi ~ hi :

speak(add(3, 4))           // 7
speak(clamp(150, 0, 100))  // 100
```

Multi-statement lambda body with `::` and `;;`:

```
forge process be x -&gt; ::
  forge doubled be x * 2
  forge result be doubled + 10
  yield result
;;

speak(process(5))   // 20
```

## Closures

Functions capture variables from their enclosing scope:

```
conjure makeCounter(start) ::
  forge count be start

  conjure increment() ::
    count be count + 1
    yield count
  ;;

  conjure reset() ::
    count be start
  ;;

  yield :: "next": increment, "reset": reset ;;
;;

forge c be makeCounter(0)
speak(c.next())   // 1
speak(c.next())   // 2
```

```
speak(c.next())   // 3
c.reset()
speak(c.next())   // 1
```

## Recursive Functions

```
conjure factorial(n) ::
  ponder n &lt;= 1 :: yield 1 ;;
  yield n * factorial(n - 1)
;;

speak(factorial(10))   // 3628800

// Mutual recursion
conjure isEven(n) ::
  ponder n equals 0 :: yield yep ;;
  yield isOdd(n - 1)
;;

conjure isOdd(n) ::
  ponder n equals 0 :: yield nope ;;
  yield isEven(n - 1)
;;

speak(isEven(4))   // yep
speak(isOdd(7))    // yep
```

## Higher-Order Functions as Parameters

```
conjure applyTwice(f, x) ::
  yield f(f(x))
;;

forge double be x -&gt; x * 2
speak(applyTwice(double, 3))   // 12

conjure compose(f, g) ::
```

```
  yield x -&gt; f(g(x))
;;

forge addOne be x -&gt; x + 1
forge triple be x -&gt; x * 3
forge addOneThenTriple be compose(triple, addOne)
speak(addOneThenTriple(4))   // 15
```

### Variadic-style Functions

Use lists to simulate variadic arguments:

```
conjure sumAll(nums) ::
  yield fold(nums, 0, (acc, x) -&gt; acc + x)
;;

speak(sumAll([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))   // 55
```

## Object-Oriented Programming

sdev has full class-based OOP using `essence` for class definitions, `new` for instantiation, `self` for the instance reference, `super` for parent class access, and `extend` for inheritance.

### Defining a Class — `essence`

```
essence Person ::
  conjure init(self, name, age) ::
    self.name be name
```

```
    self.age be age
  ;;

  conjure greet(self) ::
    speak("Hi, I'm " + self.name + " and I'm " + morph(self.age, "
  ;;

  conjure birthday(self) ::
    self.age be self.age + 1
    speak(self.name + " is now " + morph(self.age, "text") + "!")
  ;;

  conjure toString(self) ::
    yield "Person(" + self.name + ", " + morph(self.age, "text") +
  ;;
;;
```

## Creating Instances — `new`

```
forge alice be new Person("Alice", 30)
forge bob be new Person("Bob", 25)

alice.greet()   // Hi, I'm Alice and I'm 30 years old.
bob.greet()     // Hi, I'm Bob and I'm 25 years old.

alice.birthday()   // Alice is now 31!
speak(alice.age)   // 31

speak(alice.toString())  // Person(Alice, 31)
```

## Inheritance — `extend`

```
essence Animal ::
  conjure init(self, name, sound) ::
    self.name be name
    self.sound be sound
    self.energy be 100
```

```
    ;;

    conjure makeSound(self) ::
      speak(self.name + " says: " + self.sound)
    ;;

    conjure eat(self, food) ::
      self.energy be self.energy + 20
      speak(self.name + " eats " + food)
    ;;

    conjure status(self) ::
      speak(self.name + " | Energy: " + morph(self.energy, "text"))
    ;;
  ;;

  essence Dog extend Animal ::
    conjure init(self, name, breed) ::
      super.init(name, "Woof!")
      self.breed be breed
      self.tricks be []
    ;;

    conjure makeSound(self) ::
      speak(self.name + " barks: WOOF WOOF!")   // Override parent
    ;;

    conjure learnTrick(self, trick) ::
      gather(self.tricks, trick)
      speak(self.name + " learned: " + trick)
    ;;

    conjure showTricks(self) ::
      ponder measure(self.tricks) equals 0 ::
        speak(self.name + " knows no tricks.")
        yield void
      ;;
      speak(self.name + " can do: " + weave(self.tricks, ", "))
    ;;
```

```
;;

forge rex be new Dog("Rex", "German Shepherd")
rex.makeSound()                // Rex barks: WOOF WOOF!
rex.eat("kibble")              // Rex eats kibble
rex.learnTrick("sit")
rex.learnTrick("shake")
rex.learnTrick("roll over")
rex.showTricks()               // Rex can do: sit, shake, roll ove
rex.status()                   // Rex | Energy: 120
```

## Multi-level Inheritance

```
essence Vehicle ::
  conjure init(self, brand, speed) ::
    self.brand be brand
    self.speed be speed
    self.fuel be 100
  ;;

  conjure describe(self) ::
    yield self.brand + " going " + morph(self.speed, "text") + " kn
  ;;
;;

essence Car extend Vehicle ::
  conjure init(self, brand, speed, doors) ::
    super.init(brand, speed)
    self.doors be doors
  ;;

  conjure honk(self) ::
    speak(self.brand + ": Beep beep!")
  ;;
;;

essence ElectricCar extend Car ::
  conjure init(self, brand, speed, doors, range) ::
```

```
    super.init(brand, speed, doors)
    self.range be range
    self.battery be 100
  ;;

  conjure charge(self) ::
    self.battery be 100
    speak(self.brand + " fully charged!")
  ;;

  conjure describe(self) ::
    yield super.describe() + " (Electric, " + morph(self.range, "to
  ;;
;;

forge tesla be new ElectricCar("Tesla Model 3", 250, 4, 500)
speak(tesla.describe())   // Tesla Model 3 going 250 km/h (Electric
tesla.honk()              // Tesla Model 3: Beep beep!
tesla.charge()            // Tesla Model 3 fully charged!
```

## Properties and Computed Properties

```
essence Circle ::
  conjure init(self, radius) ::
    self.radius be radius
  ;;

  conjure area(self) ::
    yield PI * self.radius ^ 2
  ;;

  conjure circumference(self) ::
    yield 2 * PI * self.radius
  ;;

  conjure scale(self, factor) ::
    self.radius be self.radius * factor
    yield self
```

```
  ;;

  conjure toString(self) ::
    yield "Circle(r=" + morph(nearby(self.radius * 100) / 100, "tex
  ;;
;;

forge c be new Circle(5)
speak(nearby(c.area()))           // 79
speak(nearby(c.circumference()))  // 31

c.scale(2)
speak(c.toString())   // Circle(r=10)
```

# Error Handling

## Try / Rescue — `attempt` / `rescue`

Use `attempt` to try code that might fail, and `rescue` to catch errors:

```
attempt ::
  forge result be 10 / 0
  speak(result)
;; rescue err ::
  speak("Caught an error: " + err)
;;

attempt ::
  forge nums be [1, 2, 3]
  speak(nums[10])   // index out of bounds
;; rescue err ::
  speak("Error accessing list:", err)
;;
```

## Nested Error Handling

```
conjure safeDivide(a, b) ::
  attempt ::
    ponder b equals 0 ::
      speak("Error: division by zero")
      yield void
    ;;
    yield a / b
  ;; rescue err ::
    speak("Unexpected error:", err)
    yield void
  ;;
;;

speak(safeDivide(10, 2))   // 5
speak(safeDivide(10, 0))   // Error: division by zero
```

## Assertion

```
conjure validateAge(age) ::
  attempt ::
    ponder age &lt; 0 ::
      speak("Error: Age cannot be negative")
      yield nope
    ;;
    ponder age &gt; 150 ::
      speak("Error: Age is unreasonably large")
      yield nope
    ;;
    yield yep
  ;; rescue err ::
    speak("Validation error:", err)
    yield nope
  ;;
;;
```

# Async & Concurrency

## Async Functions

Use `async conjure` to define an asynchronous function:

```
async conjure fetchData(url) ::
  forge response be await fetch(url)
  yield response
;;
```

## Await

Use `await` inside `async` functions to wait for asynchronous operations:

```
async conjure loadUserProfile(id) ::
  forge user be await fetchData("https://api.example.com/users/" +
  forge posts be await fetchData("https://api.example.com/posts?us

  yield ::
    "user": user,
    "posts": posts
  ;;
;;
```

## Spawn (Concurrent Execution)

Use `spawn` to run functions concurrently without waiting for them:

```
conjure worker(name, ms) ::
  delay(ms)
  speak(name + " finished after " + morph(ms, "text") + "ms")
;;

spawn worker("Task A", 2000)
spawn worker("Task B", 500)
spawn worker("Task C", 1000)
// Task B completes first, then C, then A
```

### Delay

Pause execution for a number of milliseconds:

```
speak("Starting...")
delay(1000)
speak("One second later...")
delay(2000)
speak("Three seconds total")
```

## Built-in Output Functions

| Function | Description | Example |
|---------|----------|---------| | `speak(...)` | Print values with spaces between them | `speak("x =", 42)` → `x = 42` | | `whisper(...)` | Print values concatenated (no spaces) | `whisper("a", "b")` → `ab` | | `shout(...)` | Print values uppercased | `shout("hello")` → `HELLO` |

```
forge x be 42
forge name be "Alice"

speak("The answer is", x)       // The answer is 42
```

```
speak(name, "scored", x, "%")   // Alice scored 42 %
whisper("[", x, "]")             // [42]
shout("warning: low battery")    // WARNING: LOW BATTERY
```

# Built-in Math Functions

## Core Math

| Function | Description | Example |
|---------|----------|---------|
| `magnitude(x)` | Absolute value | `magnitude(-5)` → `5` |
| `root(x)` | Square root | `root(16)` → `4` |
| `ground(x)` | Floor (round down) | `ground(3.7)` → `3` |
| `elevate(x)` | Ceiling (round up) | `elevate(3.2)` → `4` |
| `nearby(x)` | Round to nearest int | `nearby(3.5)` → `4` |
| `least(...)` | Minimum value | `least(3, 1, 4, 1)` → `1` |
| `greatest(...)` | Maximum value | `greatest(3, 1, 4, 1)` → `4` |

## Trigonometry

| Function | Description |
|---------|----------|
| `sin(x)` | Sine (radians) |
| `cos(x)` | Cosine (radians) |
| `tan(x)` | Tangent (radians) |
| `asin(x)` | Arcsine |
| `acos(x)` | Arccosine |
| `atan(x)` | Arctangent |
| `atan2(y, x)` | Two-argument arctangent |
| `radians(deg)` | Convert degrees to radians |
| `degrees(rad)` | Convert radians to degrees |

## Logarithms & Exponentials

| Function | Description | |----------|----------| | `log(x)` | Natural logarithm | | `log10(x)` | Base-10 logarithm | | `log2(x)` | Base-2 logarithm | | `exp(x)` | e^x |

## Advanced Math

| Function | Description | Example | |----------|----------|---------| | `clamp(v, min, max)` | Constrain value to range | `clamp(150, 0, 100)` → `100` | | `lerp(a, b, t)` | Linear interpolation | `lerp(0, 100, 0.3)` → `30` | | `mapRange(v, fromLo, fromHi, toLo, toHi)` | Map value between ranges | `mapRange(50, 0, 100, 0, 1)` → `0.5` | | `sum(list)` | Sum all elements of a list | `sum([1,2,3,4])` → `10` | | `average(list)` | Mean value of a list | `average([1,2,3,4])` → `2.5` | | `sign(x)` | Sign: -1, 0, or 1 | `sign(-5)` → `-1` | | `dist(x1, y1, x2, y2)` | Distance between two points | `dist(0,0,3,4)` → `5` |

## Random

| Function | Description | Example | |----------|----------|---------| | `chaos()` | Random float 0-1 | `chaos()` → `0.7234` | | `randint(min, max)` | Random integer (inclusive) | `randint(1, 6)` → `4` | | `pick(list)` | Random element from list | `pick(["a","b","c"])` → `"b"` | | `shuffle(list)` | Shuffle a copy of list | `shuffle([1,2,3])` → `[3,1,2]` |

```
// Dice rolling
```

```
conjure rollDice(sides) ::
  yield randint(1, sides)
;;

speak("d6:", rollDice(6))
speak("d20:", rollDice(20))

// Coin flip
speak(chaos() &gt; 0.5 ~ "Heads" : "Tails")
```

# Built-in String Functions

| Function | Description | Signature |
|---------|----------|----------| | `upper(s)` | Uppercase | `upper("hi")` → `"HI"` | | `lower(s)` | Lowercase | `lower("HI")` → `"hi"` | | `trim(s)` | Remove surrounding whitespace | `trim("  hi  ")` → `"hi"` | | `reverse(s)` | Reverse string | `reverse("abc")` → `"cba"` | | `measure(s)` | String length | `measure("hello")` → `5` | | `contains(s, sub)` | Check substring | `contains("hello", "ell")` → `yep` | | `startswith(s, prefix)` | Starts with | `startswith("hello", "he")` → `yep` | | `endswith(s, suffix)` | Ends with | `endswith("hello", "lo")` → `yep` | | `replace(s, old, new)` | Replace all occurrences | `replace("aaa", "a", "b")` → `"bbb"` | | `locate(s, sub)` | Index of substring (-1 if not found) | `locate("hello", "l")` → `2` | | `shatter(s, sep)` | Split string into list | `shatter("a,b", ",")` → `["a","b"]` | | `chars(s)` | Split into list of characters | `chars("abc")` → `["a","b","c"]` | | `weave(list, sep)` | Join list with separator | `weave(["a","b"], "-")` → `"a-b"` | | `padLeft(s, w, c)` | Pad left to width | `padLeft("5", 3, "0")` → `"005"` | |

`padRight(s, w, c)` | Pad right to width | `padRight("hi", 5, ".")` → `"hi..."` | | `format(s, ...)` | Substitute `{}` placeholders | `format("Hi {}!", "Bob")` → `"Hi Bob!"` | | `repeat(s, n)` | Repeat string n times | `repeat("ab", 3)` → `"ababab"` | | `snatch(s, start, end?)` | Substring by index | `snatch("hello", 1, 3)` → `"el"` |

# Built-in List Functions

| Function | Description | Example |
|---------|----------|---------|
| `measure(list)` | Length of list | `measure([1,2,3])` → `3` |
| `gather(list, item)` | Append item (mutates) | `gather([1,2], 3)` → `[1,2,3]` |
| `pluck(list)` | Remove and return last item | `pluck([1,2,3])` → `3` |
| `snatch(list, idx)` | Remove item at index | `snatch([1,2,3], 1)` → removes `2` |
| `insert(list, idx, item)` | Insert at index | `insert([1,3], 1, 2)` → `[1,2,3]` |
| `portion(list, start, end?)` | Slice | `portion([1,2,3,4], 1, 3)` → `[2,3]` |
| `reverse(list)` | Return reversed copy | `reverse([1,2,3])` → `[3,2,1]` |
| `sort(list)` | Sort ascending copy | `sort([3,1,2])` → `[1,2,3]` |
| `sortDesc(list)` | Sort descending copy | `sortDesc([1,3,2])` → `[3,2,1]` |
| `unique(list)` | Remove duplicates | `unique([1,1,2,2,3])` → `[1,2,3]` |
| `flatten(list)` | Flatten nested lists | `flatten([[1,2],[3,4]])` → `[1,2,3,4]` |
| `concat(a, b)` | Concatenate two lists | `concat([1,2],[3,4])` → `[1,2,3,4]` |
| `contains(list, item)` | Check membership | `contains([1,2,3], 2)` → `yep` |
| `first(list)` | First element |

| `first([1,2,3])` → `1` | | `last(list)` | Last element | `last([1,2,3])` → `3` | | `clone(list)` | Deep copy | `clone([1,[2,3]])` |

# Built-in Tome (Dict) Functions

| Function | Description | |----------|----------| | `inscriptions(tome)` | Get all keys as a list | | `contents(tome)` | Get all values as a list | | `entries(tome)` | Get `[[key, value], ...]` pairs | | `contains(tome, key)` | Check if key exists | | `merge(t1, t2, ...)` | Merge dicts (later overrides earlier) | | `erase(tome, key)` | Remove key |

```
forge a be :: "x": 1, "y": 2 ;;
forge b be :: "y": 99, "z": 3 ;;
forge c be merge(a, b)
speak(c)  // {"x": 1, "y": 99, "z": 3}
```

# Input / Output

## `input(prompt?)` — Read User Input

Prompts the user for input and returns the entered text:

```
forge name be input("What is your name? ")
speak("Hello, " + name + "!")
```

```
forge age be morph(input("Enter your age: "), "number")
speak("You are " + age + " years old")
```

### `print(...)` / `println(...)` — Print (Aliases)

Standard aliases for `speak()`:

```
print("Hello")
println("World")
```

## Character & Code Point Functions

| Function | Description | Example |
|---|---|---|
| `chr(n)` | Number to character | `chr(65)` → `"A"` |
| `ord(c)` | Character to number | `ord("A")` → `65` |

```
speak(chr(72) + chr(105))  // "Hi"
speak(ord("A"))             // 65

// Build a Caesar cipher
conjure encrypt(text, shift) ::
  forge result be ""
  iterate through text ::
    forge code be ord(item) + shift
    result be result + chr(code)
  ;;
  yield result
;;
speak(encrypt("ABC", 3))  // "DEF"
```

# Number Base Conversion

| Function | Description | Example | |---------|----------|---------| | `hex(n)` | Number to hex string | `hex(255)` → `"0xFF"` | | `oct(n)` | Number to octal | `oct(8)` → `"0o10"` | | `bin(n)` | Number to binary | `bin(10)` → `"0b1010"` | | `parseNum(str, base?)` | Parse string with base | `parseNum("FF", 16)` → `255` |

```
speak(hex(255))        // "0xFF"
speak(bin(42))         // "0b101010"
speak(oct(64))         // "0o100"
speak(parseNum("1010", 2))  // 10
speak(parseNum("FF", 16))   // 255
```

# Number Formatting & Checking

| Function | Description | Example | |---------|----------|---------| | `toFixed(n, digits)` | Format decimal places | `toFixed(3.14159, 2)` → `"3.14"` | | `toPrecision(n, p)` | Format to precision | `toPrecision(123.456, 4)` → `"123.5"` | | `isNaN(v)` | Check if NaN | `isNaN(0/0)` → `yep` | | `isFinite(v)` | Check if finite | `isFinite(INFINITY)` → `nope` | | `isInteger(v)` | Check if integer | `isInteger(3.0)` → `yep` |

```
forge pi be 3.14159265
speak(toFixed(pi, 2))      // "3.14"
speak(toPrecision(pi, 4))  // "3.142"
```

```
speak(isInteger(42))       // yep
speak(isFinite(1/0))       // nope
```

# String Checking Functions

| Function | Description | Example |
|---------|----------|---------| | `capitalize(s)` | First char uppercase | `capitalize("hello")` → `"Hello"` | | `title(s)` | Title Case | `title("hello world")` → `"Hello World"` | | `center(s, width, char?)` | Center-pad | `center("hi", 10, "-")` → `"----hi----"` | | `trimLeft(s)` | Trim left whitespace | `trimLeft("  hi")` → `"hi"` | | `trimRight(s)` | Trim right whitespace | `trimRight("hi  ")` → `"hi"` | | `isUpper(s)` | All uppercase? | `isUpper("ABC")` → `yep` | | `isLower(s)` | All lowercase? | `isLower("abc")` → `yep` | | `isDigit(s)` | All digits? | `isDigit("123")` → `yep` | | `isAlpha(s)` | All alphabetic? | `isAlpha("abc")` → `yep` | | `isAlphaNum(s)` | All alphanumeric? | `isAlphaNum("abc123")` → `yep` | | `isSpace(s)` | All whitespace? | `isSpace(" ")` → `yep` |

```
speak(capitalize("hello world"))  // "Hello world"
speak(title("the quick brown fox"))  // "The Quick Brown Fox"
speak(center("TITLE", 20, "="))  // "=======TITLE========"
speak(isDigit("42"))   // yep
speak(isAlpha("hello"))  // yep
```

# Regex / Pattern Matching

| Function | Description | |----------|----------| | `match(text, pattern)` | First regex match (returns list or void) | | `matchAll(text, pattern)` | All regex matches | | `replaceRegex(text, pattern, replacement)` | Regex replace (global) | | `test(text, pattern)` | Test if pattern matches |

```
forge text be "Hello 123 World 456"
speak(match(text, "\\d+"))           // ["123"]
speak(matchAll(text, "\\d+"))        // [["123"], ["456"]]
speak(replaceRegex(text, "\\d+", "#"))  // "Hello # World #"
speak(test(text, "\\d+"))           // yep

// Validate email
conjure isEmail(s) ::
  yield test(s, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$
;;
speak(isEmail("user@example.com"))  // yep
```

# Bitwise Operations

| Function | Description | Example | |----------|----------|---------| | `bitAnd(a, b)` | Bitwise AND | `bitAnd(5, 3)` → `1` | | `bitOr(a, b)` | Bitwise OR | `bitOr(5, 3)` → `7` | | `bitXor(a, b)` | Bitwise XOR | `bitXor(5, 3)` → `6` | | `bitNot(a)` | Bitwise NOT | `bitNot(0)` → `-1` | | `bitShiftLeft(a, n)` | Left shift | `bitShiftLeft(1, 3)` → `8` | | `bitShiftRight(a, n)` | Right shift | `bitShiftRight(8, 2)` → `2` |

```
// Flags / bitmask example
forge READ be 1
```

```
forge WRITE be 2
forge EXEC be 4

forge perms be bitOr(READ, WRITE)  // 3
speak(bitAnd(perms, READ) &gt; 0)     // yep (has read)
speak(bitAnd(perms, EXEC) &gt; 0)     // nope (no exec)
```

# Base64 Encoding

| Function | Description | |----------|----------| | `base64encode(text)` | Encode text to base64 | | `base64decode(text)` | Decode base64 to text |

```
forge encoded be base64encode("Hello, World!")
speak(encoded)  // "SGVsbG8sIFdvcmxkIQ=="
speak(base64decode(encoded))  // "Hello, World!"
```

# Hash Function

```
speak(hash("hello"))     // deterministic 32-bit integer hash
speak(hash([1, 2, 3]))   // works on any value
```

# Time & Date

| Function | Description | |----------|----------| | `now()` | Current timestamp in milliseconds | | `timestamp()` | ISO 8601 string | | `time()` | Detailed time tome with year, month, day, etc. | | `formatTime(ms)` | Format milliseconds

to ISO string |

```
forge t be time()
speak("Year: " + t.year)
speak("Month: " + t.month)
speak("Day: " + t.day)
speak("Hour: " + t.hour)
speak("ISO: " + t.iso)

forge start be now()
// ... do work ...
forge elapsed be now() - start
speak("Took " + elapsed + "ms")
```

# Functional Programming

| Function | Description | |----------|----------| | `compose(f, g, ...)` | Right-to-left function composition | | `pipe(value, f, g, ...)` | Left-to-right value piping | | `curry(fn, arity)` | Currying | | `memoize(fn)` | Cache function results | | `tap(value, fn)` | Execute fn with value, return value | | `times(n, fn)` | Call fn n times with index | | `groupBy(list, fn)` | Group elements by key function | | `chunk(list, size)` | Split list into chunks |

```
// Compose
conjure double(x) :: yield x * 2 ;;
conjure addOne(x) :: yield x + 1 ;;
forge doubleAndAdd be compose(addOne, double)
speak(doubleAndAdd(5))  // 11

// Pipe
forge result be pipe(5, double, addOne, double)
```

```
speak(result)  // 22

// Memoize (cache expensive computations)
conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  yield fib(n - 1) + fib(n - 2)
;;
forge fastFib be memoize(fib)
speak(fastFib(10))  // 55

// Group and chunk
forge words be ["apple", "avocado", "banana", "blueberry", "cherry
speak(groupBy(words, w -&gt; charAt(w, 0)))
// :: a: ["apple", "avocado"], b: ["banana", "blueberry"], c: ["che

speak(chunk([1,2,3,4,5,6,7], 3))  // [[1,2,3], [4,5,6], [7]]

// Times
speak(times(5, i -&gt; i * i))  // [0, 1, 4, 9, 16]
```

# Buffer / Memory Operations

sdev provides low-level byte buffer operations for systems programming:

## `buffer(size)` — Create Byte Buffer

```
forge mem be buffer(1024)
mem.set(0, 255)
mem.set(1, 128)
speak(mem.get(0))    // 255
speak(mem.get(1))    // 128
speak(mem.size())    // 1024
```

## Buffer Methods

| Method | Description | |--------|----------| | `buf.get(index)` | Read byte at index | | `buf.set(index, value)` | Write byte at index (0-255) | | `buf.fill(value)` | Fill entire buffer | | `buf.slice(start, end)` | Get portion as list | | `buf.toList()` | Convert to list of numbers | | `buf.toText()` | Decode as UTF-8 text | | `buf.fromString(text)` | Write UTF-8 text into buffer | | `buf.copyTo(target)` | Copy data to another buffer | | `buf.size()` | Get buffer size |

## `pointer(buffer, offset)` — Memory Pointer

```
forge mem be buffer(256)
forge ptr be pointer(mem, 0)
ptr.write(42)
speak(ptr.read())        // 42

forge ptr2 be ptr.advance(4)
ptr2.writeU16(1024)
speak(ptr2.readU16())    // 1024

ptr2.writeU32(0xDEADBEEF)
speak(hex(ptr2.readU32()))  // "0xDEADBEEF"
```

## Pointer Methods

| Method | Description | |--------|----------| | `ptr.read()` | Read byte | | `ptr.write(v)` | Write byte | | `ptr.advance(n?)` | Return new pointer at offset+n | | `ptr.readU16()` | Read 16-bit unsigned (little-endian) | | `ptr.readU32()` | Read 32-bit unsigned (little-endian) | | `ptr.writeU16(v)` | Write

16-bit unsigned | | `ptr.writeU32(v)` | Write 32-bit unsigned |

# Error Handling & Control Flow

| Function | Description | |----------|----------| | `exit(code?)` | Terminate program with exit code | | `panic(message)` | Fatal error (kernel panic) | | `throw(message)` | Throw custom error |

```
// Throw and catch
attempt ::
  forge x be input("Enter a number: ")
  forge n be morph(x, "number")
  ponder n &lt; 0 ::
    throw("Negative numbers not allowed!")
  ;;
  speak("Square root: " + root(n))
rescue err ::
  speak("Error: " + err)
;;

// Exit
ponder badCondition ::
  exit(1)
;;
```

# Additional Aliases

| Alias | Original | Description | |-------|----------|-------------| | `print()` | `speak()` | Standard print | | `range()` | `sequence()` | Standard range | | `typeof()` | `gettype()` | Type checking | | `sleep()` | `delay()` | Pause (no-op in browser) | | `keys()` | `inscriptions()` | Dict keys | | `values()` | `contents()` | Dict values | | `freeze(obj)` | — | Make object immutable | | `isFrozen(obj)` | — | Check if frozen | | `hash(value)` | — | 32-bit hash code |

# Higher-Order Functions

## `each` — Transform (Map)

Apply a function to every element and return a new list:

```
forge nums be [1, 2, 3, 4, 5]
speak(each(nums, x -&gt; x ^ 2))          // [1, 4, 9, 16, 25]
speak(each(nums, (x, i) -&gt; x + i))     // [1, 3, 5, 7, 9]  (valu
speak(each(["a","b","c"], upper))       // ["A", "B", "C"]
```

## `sift` — Filter

Keep only elements where the predicate returns `yep`:

```
forge nums be [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
speak(sift(nums, x -&gt; x % 2 equals 0))       // [2, 4, 6, 8, 10
speak(sift(nums, x -&gt; x &gt; 5))                // [6, 7, 8, 9,
speak(sift(["", "hi", "", "world"], x -&gt; measure(x) &gt; 0))  /
```

## `fold` — Reduce

Reduce a list to a single value using an accumulator:

```
forge nums be [1, 2, 3, 4, 5]
speak(fold(nums, 0, (acc, x) -&gt; acc + x))   // 15 (sum)
speak(fold(nums, 1, (acc, x) -&gt; acc * x))   // 120 (product)

// Build a string
speak(fold(["a","b","c"], "", (acc, x) -&gt; acc + x + "-"))  // "a

// Find max
speak(fold(nums, nums[0], (a, b) -&gt; a &gt; b ~ a : b))   // 5
```

## `seek` — Find First

Return the first element matching a predicate, or `void`:

```
forge people be [
  :: "name": "Alice", "age": 22 ;;,
  :: "name": "Bob", "age": 35 ;;,
  :: "name": "Carol", "age": 28 ;;
]

forge adult be seek(people, p -&gt; p.age &gt;= 30)
speak(adult.name)   // Bob

forge nobody be seek(people, p -&gt; p.age &gt; 100)
speak(nobody)   // void
```

## `every` — All Match

Returns `yep` if ALL elements satisfy the predicate:

```
speak(every([2,4,6,8], x -&gt; x % 2 equals 0))  // yep
speak(every([1,2,3], x -&gt; x &gt; 0))             // yep
speak(every([1,-2,3], x -&gt; x &gt; 0))            // nope
```

### `some` — Any Match

Returns `yep` if ANY element satisfies the predicate:

```
speak(some([1,3,5,6], x -&gt; x % 2 equals 0))  // yep (6 is even)
speak(some([1,3,5,7], x -&gt; x % 2 equals 0))  // nope
```

### `enumerate` — Index + Value

Pairs each element with its index:

```
speak(enumerate(["a","b","c"]))
// [[0,"a"], [1,"b"], [2,"c"]]

within pair be enumerate(["red","green","blue"]) ::
  speak(pair[0] + ": " + pair[1])
;;
// 0: red
// 1: green
// 2: blue
```

### `zip` — Pair Two Lists

Combine two lists element-by-element:

```
forge keys be ["name", "age", "city"]
forge vals be ["Alice", 30, "NYC"]
speak(zip(keys, vals))
// [["name","Alice"], ["age",30], ["city","NYC"]]
```

# Type System & Conversion

### Getting Type — `essence()`

Returns the type name as a string:

```
speak(essence(42))          // "number"
speak(essence("hi"))        // "text"
speak(essence(yep))         // "truth"
speak(essence(void))        // "void"
speak(essence([1,2,3]))     // "list"
speak(essence(::;;))        // "tome"
```

### Converting Types — `morph()`

```
speak(morph("42", "number"))     // 42
speak(morph(42, "text"))         // "42"
speak(morph(1, "truth"))         // yep
speak(morph(0, "truth"))         // nope
speak(morph("3.14", "number"))   // 3.14
```

### Type Checking Helpers

```
speak(essence(x) equals "number")      // Check if number
speak(essence(x) equals "text")        // Check if string
speak(essence(x) equals "list")        // Check if list
speak(essence(x) equals "tome")        // Check if dict
speak(x equals void)                   // Check if null
```

# Collections

## Set

A collection of unique elements. Duplicate additions are silently ignored.

```
forge s be Set()
s.add(1)
s.add(2)
s.add(2)    // Duplicate ignored
s.add(3)

speak(s.size())     // 3
speak(s.has(2))     // yep
speak(s.has(99))    // nope
speak(s.values())   // [1, 2, 3]

s.remove(2)
speak(s.values())   // [1, 3]

s.clear()
speak(s.size())     // 0
speak(s.isEmpty())  // yep
```

**Set methods:** `add(v)`, `remove(v)`, `has(v)`, `values()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## Map

A key-value store where keys can be any type (including objects):

```
forge m be Map()
m.set("name", "Alice")
m.set("age", 30)
m.set("active", yep)

speak(m.get("name"))     // Alice
speak(m.has("age"))      // yep
```

```
speak(m.keys())          // ["name", "age", "active"]
speak(m.values())        // ["Alice", 30, yep]
speak(m.entries())       // [["name","Alice"], ["age",30], ["activ
speak(m.size())          // 3

m.delete("active")
speak(m.size())          // 2
m.clear()
```

**Map methods:** `set(k, v)`, `get(k)`, `has(k)`, `delete(k)`, `keys()`, `values()`, `entries()`, `size()`, `isEmpty()`, `clear()`

## Queue (FIFO)

First-in, first-out data structure:

```
forge q be Queue()
q.enqueue("first")
q.enqueue("second")
q.enqueue("third")

speak(q.peek())      // first  (look without removing)
speak(q.dequeue())   // first  (remove and return)
speak(q.peek())      // second
speak(q.size())      // 2
speak(q.isEmpty())   // nope
```

**Queue methods:** `enqueue(v)`, `dequeue()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## Stack (LIFO)

Last-in, first-out data structure:

```
forge s be Stack()
s.push(10)
```

```
s.push(20)
s.push(30)

speak(s.peek())   // 30  (look without removing)
speak(s.pop())    // 30  (remove and return)
speak(s.pop())    // 20
speak(s.size())   // 1

// Stack-based undo system
forge history be Stack()
history.push("action1")
history.push("action2")
history.push("action3")

speak("Undoing:", history.pop())  // action3
speak("Undoing:", history.pop())  // action2
```

**Stack methods:** `push(v)`, `pop()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

## LinkedList

A doubly-linked list with O(1) front/back operations:

```
forge list be LinkedList()
list.append(1)
list.append(2)
list.append(3)
list.prepend(0)

speak(list.toList())   // [0, 1, 2, 3]
speak(list.size())     // 4
speak(list.get(2))     // 2

list.remove(1)         // Remove element with value 1
speak(list.toList())   // [0, 2, 3]

speak(list.head())     // 0
```

```
speak(list.tail())     // 3
```

**LinkedList methods:** `append(v)`, `prepend(v)`, `remove(v)`, `get(idx)`, `size()`, `head()`, `tail()`, `toList()`, `clear()`

# Matrix Operations

sdev has a comprehensive built-in matrix math library useful for data science and machine learning.

## Creating Matrices

```
// Matrix filled with a value
forge zeros be matrix(3, 3, 0)
// [[0,0,0], [0,0,0], [0,0,0]]

forge ones be matrix(2, 4, 1)
// [[1,1,1,1], [1,1,1,1]]

// Identity matrix
forge eye be identity(4)
// [[1,0,0,0], [0,1,0,0], [0,0,1,0], [0,0,0,1]]

// Reshape a flat list into a matrix
forge m be reshape([1,2,3,4,5,6], 2, 3)
// [[1,2,3], [4,5,6]]
```

## Matrix Arithmetic

```
forge a be [[1, 2], [3, 4]]
forge b be [[5, 6], [7, 8]]

// Element-wise operations
```

```
speak(matadd(a, b))     // [[6,8], [10,12]]
speak(matsub(a, b))     // [[-4,-4], [-4,-4]]
speak(matscale(a, 2))   // [[2,4], [6,8]]

// Matrix multiplication
speak(matmul(a, b))
// [[1*5+2*7, 1*6+2*8], [3*5+4*7, 3*6+4*8]]
// = [[19,22], [43,50]]

// Transpose
speak(transpose(a))
// [[1,3], [2,4]]
```

## Vector Operations

```
forge v1 be [1, 2, 3]
forge v2 be [4, 5, 6]

// Dot product
speak(dot(v1, v2))   // 1*4 + 2*5 + 3*6 = 32

// Element-wise multiply via each
forge elem be each(v1, (x, i) -&gt; x * v2[i])
speak(elem)   // [4, 10, 18]
```

## Matrix Utilities

| Function | Description | |----------|----------| | `matrix(rows, cols, fill)` | Create filled matrix | | `identity(n)` | Create n×n identity matrix | | `transpose(m)` | Transpose rows and columns | | `matmul(a, b)` | Matrix multiplication | | `matadd(a, b)` | Element-wise addition | | `matsub(a, b)` | Element-wise subtraction | | `matscale(m, s)` | Scalar multiplication | | `dot(a, b)` | Dot product of

two vectors | | `reshape(list, rows, cols)` | Reshape list to 2D matrix | | `flatten(m)` | Flatten 2D matrix to 1D list | | `shape(m)` | Get `[rows, cols]` of matrix | | `sum(m)` | Sum all elements | | `mean(list)` | Mean of a 1D list |

## Neural Network Example

```
// Sigmoid activation function
conjure sigmoid(x) ::
  yield 1 / (1 + exp(-x))
;;

// Feedforward neural network layer
conjure layer(inputs, weights, biases) ::
  // inputs: [1 x n], weights: [n x m], biases: [1 x m]
  forge z be matmul(inputs, weights)
  forge zb be matadd(z, biases)
  // Apply sigmoid element-wise
  yield each(zb, row -&gt; each(row, x -&gt; sigmoid(x)))
;;

forge input be [[0.5, 0.3, 0.2]]
forge w1 be [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]]
forge b1 be [[0.1, 0.1]]

forge output be layer(input, w1, b1)
speak("Layer output:", output)
```

# Graphics & Game Development

sdev has a built-in 2D graphics and turtle graphics API. Graphics commands render to the Canvas panel in the IDE.

## Canvas Setup

```
canvas(800, 600)    // Set canvas dimensions (width, height)
clear("#1a1a2e")    // Clear with a background color
```

## Fill and Stroke

```
fill("blue")          // Set fill color
noFill()              // Disable fill
stroke("red", 2)      // Set stroke: color, width
noStroke()            // Disable stroke
lineWidth(3)          // Set line width separately
alpha(0.5)            // Set global transparency (0–1)
lineCap("round")      // Line cap: "round", "square", "butt"
lineJoin("round")     // Line join: "round", "bevel", "miter"
```

## Basic Shapes

```
// Rectangle: rect(x, y, width, height, cornerRadius?)
fill("steelblue")
rect(50, 50, 200, 100)
rect(300, 50, 200, 100, 20)    // Rounded corners

// Circle: circle(x, y, radius)
fill("tomato")
circle(150, 300, 60)

// Ellipse: ellipse(x, y, radiusX, radiusY, rotation?)
fill("gold")
ellipse(350, 300, 100, 50)
```

```
ellipse(550, 300, 100, 50, 45)   // Rotated 45°

// Line: line(x1, y1, x2, y2)
stroke("white", 2)
line(50, 400, 750, 400)

// Arc: arc(x, y, radius, startAngle, endAngle, counterclockwise?)
stroke("lime", 3)
arc(200, 500, 50, 0, 270)

// Triangle: triangle(x1, y1, x2, y2, x3, y3)
fill("orchid")
triangle(400, 450, 450, 550, 350, 550)

// Polygon: polygon([[x,y], ...])
fill("coral")
polygon([[600, 400], [650, 430], [640, 490], [560, 490], [550, 430

// Star: star(x, y, outerR, innerR, points?)
fill("gold")
star(150, 150, 50, 25)          // 5-pointed star
star(300, 150, 60, 25, 8)       // 8-pointed star

// Heart: heart(x, y, size)
fill("red")
heart(500, 150, 50)

// Point: point(x, y, size?)
fill("white")
point(100, 100)
point(150, 100, 8)
```

## Text Drawing

```
fill("white")
text("Hello, World!", 100, 100)           // Basic text
text("Large Text", 100, 150, 36)          // With size
text("Bold Text", 100, 200, 24, "bold")   // With style
```

```
font("Arial", "bold")                    // Set font family and s
textAlign("center", "middle")            // Horizontal: left/cente
```

## Gradients

```
// Linear gradient: fill with gradient from (x1,y1) to (x2,y2)
// Color stops: [position, color] where position is 0.0 to 1.0
linearGradient(0, 0, 400, 0, [0, "red"], [0.5, "yellow"], [1, "gree
rect(0, 0, 400, 100)

// Radial gradient: from inner circle to outer circle
radialGradient(200, 200, 0, 200, 200, 150, [0, "white"], [0.5, "cya
circle(200, 200, 150)
```

## Shadows

```
shadow("rgba(0,0,0,0.5)", 15, 5, 5)   // color, blur, offsetX, offs
fill("gold")
circle(200, 200, 80)
noShadow()
```

## Transformations

```
save()                // Push current state
translate(200, 200)   // Move origin to (200, 200)
rotate(0.785)         // Rotate 45° (in radians)
scale(1.5)            // Scale uniformly
rect(-50, -50, 100, 100)   // Draw centered rectangle
restore()             // Pop saved state

resetTransform()      // Reset all transforms to identity
```

## Path Drawing

```
// Custom polygon path
fill("purple")
beginPath()
moveTo(100, 100)
lineTo(200, 80)
lineTo(250, 150)
lineTo(200, 220)
lineTo(100, 200)
closePath()
fillPath()
strokePath()

// Bezier curve
stroke("cyan", 2)
beginPath()
moveTo(100, 300)
bezierTo(150, 200, 250, 400, 300, 300)   // cp1x, cp1y, cp2x, cp2y
strokePath()

// Quadratic curve
beginPath()
moveTo(100, 400)
quadraticTo(200, 350, 300, 400)           // cpx, cpy, x, y
strokePath()
```

## Turtle Graphics

The turtle graphics API lets you draw by commanding a turtle that moves and draws lines as it goes.

```
canvas(500, 500)
clear("#0d0d20")
turtle()           // Create turtle at center, facing up

// Basic movement
forward(100)       // Move forward 100 units (drawing as it goes)
right(90)          // Turn right 90 degrees
forward(100)
```

```
right(90)
forward(100)
right(90)
forward(100)       // This draws a square!

// Pen control
penup()            // Lift pen — move without drawing
goto(250, 250)     // Jump to coordinates
pendown()          // Lower pen — start drawing again

// Pen style
pencolor("lime")   // Set pen color (any CSS color)
penwidth(3)        // Set pen width

// Advanced
setheading(45)     // Set absolute direction (degrees)
speak(heading())   // Get current heading
speak(pos())       // Get current [x, y] position
home()             // Return to start position

// Turtle circle
turtleCircle(50)        // Draw full circle
turtleCircle(50, 72)    // Polygon approximation (72 steps)

// Dot and stamp
dot(10, "red")     // Draw a filled dot
stamp()            // Leave a mark of the turtle shape

// EXAMPLE: Psychedelic spiral
canvas(600, 600)
clear("#000000")
turtle()
within i be sequence(0, 300) ::
  pencolor(hue(i * 1.2))
  penwidth(1 + i * 0.02)
  forward(i)
  right(91)
;;
```

## Sprites & Game Objects

```
// Create sprite: createSprite(x, y, width, height, color?)
forge player be createSprite(100, 100, 40, 40, "blue")
forge enemy be createSprite(400, 300, 40, 40, "red")

// Access/set properties
player.x be 150
player.velocityX be 5
player.velocityY be -2

// Update position (applies velocity)
updateSprite(player)

// Draw sprite
drawSprite(player)
drawSprite(enemy)

// Collision detection
ponder spriteCollides(player, enemy) ::
  speak("HIT!")
;;

// Move by delta
moveSprite(player, 10, 5)
```

## Color Functions

```
// CSS color names work directly
fill("red")
fill("steelblue")
fill("rgba(255, 0, 0, 0.5)")

// Helper functions
forge red be rgb(255, 0, 0)
forge transparent be rgba(0, 255, 0, 0.3)

// HSL color — hue(hue, saturation?, lightness?)
```

```
forge cyan be hue(180)              // Cyan (s=100, l=50 defaults)
forge purple be hue(270, 80, 60)    // Custom HSL
forge faded be hsla(240, 100, 50, 0.4)

// Rainbow cycle (great for spirals!)
within i be sequence(360) ::
  fill(hue(i))
  rect(i, 0, 1, 50)
;;

// Random color
fill(randomColor())
```

## Graphics Math

```
// Convert degrees/radians
speak(radians(180))   // 3.14159...
speak(degrees(PI))    // 180

// Lerp (smooth transitions)
forge pos be lerp(0, 100, 0.25)   // 25

// Map value between ranges
forge brightness be mapRange(50, 0, 100, 0, 255)   // 127.5

// Clamp
forge safe be clamp(150, 0, 100)   // 100

// Distance between two points
speak(dist(0, 0, 3, 4))   // 5
```

## Complete 2D Graphics Example — Starfield

```
canvas(800, 600)
```

```
// Create 200 stars
forge stars be each(sequence(200), i -&gt; ::
  "x": randint(0, 800),
  "y": randint(0, 600),
  "r": chaos() * 2 + 0.5,
  "speed": chaos() * 2 + 0.5,
  "bright": chaos() * 0.7 + 0.3
;;)

clear("#000011")

within star be stars ::
  fill(rgba(255, 255, 255, star.bright))
  circle(star.x, star.y, star.r)
;;

// Draw nebula in center
radialGradient(400, 300, 0, 400, 300, 200,
  [0, "rgba(100,50,200,0.4)"],
  [0.5, "rgba(50,100,255,0.2)"],
  [1, "rgba(0,0,0,0)"]
)
circle(400, 300, 200)
```

# File I/O

> ***Note:*** *File I/O is available in the Python standalone interpreter and native desktop builds. In the browser-based IDE, these functions are simulated.*

## Reading Files

```
// Read entire file as text
```

```
forge content be decipher("data.txt")
speak(content)

// Read and parse JSON
forge config be unetch(decipher("config.json"))
speak(config.version)
speak(config.name)
```

## Writing Files

```
// Write text file
inscribe("output.txt", "Hello, World!\n")

// Write JSON with indentation
forge data be :: "score": 100, "name": "Alice", "level": 5 ;;
inscribe("save.json", etch(data))
```

## Append to File

```
appendFile("log.txt", "User logged in\n")
appendFile("log.txt", "User viewed dashboard\n")
```

## File Utilities

| Function | Description | |----------|----------| | `decipher(path)` | Read file as text | | `inscribe(path, content)` | Write text to file | | `appendFile(path, content)` | Append to file | | `fileExists(path)` | Returns `yep`/`nope` | | `deleteFile(path)` | Delete a file | | `listDir(path)` | Get list of files in directory |

# Networking

> ***Note:*** *Network functions are available in async contexts. In the browser they use* `fetch`*. In native builds they use Node.js http.*

## HTTP GET

```
async conjure getUsers() ::
  forge data be await fetch("https://jsonplaceholder.typicode.com/u
  yield data
;;
```

## HTTP POST

```
async conjure createUser(name, email) ::
  forge response be await fetch("https://api.example.com/users", :
    "method": "POST",
    "headers": :: "Content-Type": "application/json" ;;,
    "body": :: "name": name, "email": email ;;
  ;;)
  yield response
;;
```

# Examples & Recipes

## FizzBuzz

```
within i be sequence(1, 101) ::
  ponder i % 15 equals 0 ::
```

```
    speak("FizzBuzz")
  ;; otherwise ponder i % 3 equals 0 ::
    speak("Fizz")
  ;; otherwise ponder i % 5 equals 0 ::
    speak("Buzz")
  ;; otherwise ::
    speak(i)
  ;;
;;
```

## Fibonacci (Memoized)

```
forge memo be :: ;;

conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  ponder contains(memo, morph(n, "text")) ::
    yield memo[morph(n, "text")]
  ;;
  forge result be fib(n - 1) + fib(n - 2)
  memo[morph(n, "text")] be result
  yield result
;;

within i be sequence(0, 30) ::
  speak("fib(" + morph(i, "text") + ") =", fib(i))
;;
```

## Binary Search

```
conjure binarySearch(arr, target) ::
  forge lo be 0
  forge hi be measure(arr) - 1

  cycle lo &lt;= hi ::
    forge mid be ground((lo + hi) / 2)
    ponder arr[mid] equals target ::
```

```
      yield mid
    ;; otherwise ponder arr[mid] &lt; target ::
      lo be mid + 1
    ;; otherwise ::
      hi be mid - 1
    ;;
  ;;

  yield -1
;;

forge sorted be [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
speak(binarySearch(sorted, 23))   // 5
speak(binarySearch(sorted, 50))   // -1
```

## Quicksort

```
conjure quicksort(arr) ::
  ponder measure(arr) &lt;= 1 :: yield arr ;;

  forge pivot be arr[ground(measure(arr) / 2)]
  forge less be sift(arr, x -&gt; x &lt; pivot)
  forge equal be sift(arr, x -&gt; x equals pivot)
  forge greater be sift(arr, x -&gt; x &gt; pivot)

  yield quicksort(less) + equal + quicksort(greater)
;;

speak(quicksort([3, 6, 8, 10, 1, 2, 1]))
// [1, 1, 2, 3, 6, 8, 10]
```

## Stack-Based Calculator

```
conjure calculate(expression) ::
  forge tokens be shatter(expression, " ")
  forge stack be Stack()
```

```
  within token be tokens ::
    ponder token equals "+" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a + b)
    ;; otherwise ponder token equals "-" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a - b)
    ;; otherwise ponder token equals "*" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a * b)
    ;; otherwise ponder token equals "/" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a / b)
    ;; otherwise ::
      stack.push(morph(token, "number"))
    ;;
  ;;

  yield stack.pop()
;;

speak(calculate("3 4 + 2 *"))    // (3+4)*2 = 14
speak(calculate("5 1 2 + 4 * + 3 -"))  // 5+(1+2)*4-3 = 14
```

## Caesar Cipher

```
conjure caesarEncode(text, shift) ::
  forge result be ""
  within ch be chars(text) ::
    forge code be locate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", upper(ch))
    ponder code &gt;= 0 ::
      forge shifted be (code + shift) % 26
      result be result + "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[shifted]
    ;; otherwise ::
```

```
      result be result + ch
    ;;
  ;;
  yield result
;;

speak(caesarEncode("HELLO WORLD", 3))   // KHOOR ZRUOG
speak(caesarEncode("KHOOR ZRUOG", 23))  // HELLO WORLD (decode with
```

## Word Frequency Counter

```
conjure wordFrequency(text) ::
  forge words be shatter(lower(trim(text)), " ")
  forge freq be ::;;

  within word be words ::
    forge w be replace(replace(word, ".", ""), ",", "")
    ponder measure(w) &gt; 0 ::
      ponder contains(freq, w) ::
        freq[w] be freq[w] + 1
      ;; otherwise ::
        freq[w] be 1
      ;;
    ;;
  ;;

  yield freq
;;

forge text be "the quick brown fox jumps over the lazy dog the fox
forge freq be wordFrequency(text)
speak(freq)
// Sort by frequency
forge sorted be sort(inscriptions(freq), k -&gt; -freq[k])
```

## Prime Sieve

```
conjure sieve(limit) ::
  forge primes be each(sequence(2, limit + 1), x -&gt; x)
  forge result be []

  cycle measure(primes) &gt; 0 ::
    forge p be primes[0]
    gather(result, p)
    primes be sift(primes, x -&gt; x % p differs 0)
  ;;

  yield result
;;

speak(sieve(50))
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

## Linked-List Graph Traversal (BFS)

```
// BFS using a Queue
conjure bfs(graph, start) ::
  forge visited be Set()
  forge queue be Queue()
  forge order be []

  queue.enqueue(start)
  visited.add(start)

  cycle isnt queue.isEmpty() ::
    forge node be queue.dequeue()
    gather(order, node)

    within neighbor be graph[node] ::
      ponder isnt visited.has(neighbor) ::
        visited.add(neighbor)
        queue.enqueue(neighbor)
      ;;
    ;;
  ;;
```

```
  yield order
;;

forge graph be ::
  "A": ["B", "C"],
  "B": ["A", "D", "E"],
  "C": ["A", "F"],
  "D": ["B"],
  "E": ["B", "F"],
  "F": ["C", "E"]
;;

speak(bfs(graph, "A"))  // ["A", "B", "C", "D", "E", "F"]
```

## Turtle Mandala

```
canvas(600, 600)
clear("#000000")
turtle()
penwidth(1.5)

forge petals be 12
forge petalAngle be 360 / petals

within i be sequence(petals) ::
  pencolor(hue(i * petalAngle))
  within j be sequence(60) ::
    forward(j * 0.5)
    right(6)
  ;;
  penup()
  home()
  pendown()
  setheading(i * petalAngle)
;;
```

# JavaScript Interop (JS Interpreter Only)

The browser-based interpreter includes a special `js` keyword for calling JavaScript functions directly from sdev code. This allows integration with browser APIs, DOM manipulation, and third-party JS libraries.

## Three Syntax Forms

```
// 1. Single-line expression
js &lt;expression&gt;

// 2. Parenthesized form (for objects, multi-line expressions)
js (
  &lt;expression&gt;
)

// 3. Statement block
js {
  &lt;statements&gt;
}
```

## Basic Examples

```
// Browser dialog
js alert("Hello from sdev!")

// Read browser properties
forge width be js window.innerWidth
speak("Width:", width)

// DOM manipulation
js document.body.style.backgroundColor = "#1a1a2e"
```

```
js document.title = "My sdev App"

// Math
forge angle be js Math.PI / 6
forge result be js Math.sin(0.5)
```

## Object Literals

```
// Use parenthesized form for object literals
forge opts be js ({
  dragging: true,
  zoom: 12,
  center: [51.505, -0.09]
})

// Nested objects
forge config be js ({
  api: {
    url: "https://api.example.com",
    timeout: 5000
  },
  debug: false
})
```

## Arrow Functions

```
// Map/filter in JS
forge doubled be js [1, 2, 3, 4].map(x =&gt; x * 2)
forge filtered be js [1, 2, 3, 4].filter(x =&gt; x &gt; 2)

// Event handlers
js document.getElementById("btn").addEventListener("click", (e) =&g
  console.log("Clicked!", e.target)
})

// Use sdev variables in JS arrows
forge multiplier be 5
```

```
forge result be js [1, 2, 3].map(x =&gt; x * multiplier)
```

## Multi-line JS Blocks

```
js {
  const canvas = document.createElement("canvas")
  canvas.width = 400
  canvas.height = 300
  document.body.appendChild(canvas)

  const ctx = canvas.getContext("2d")
  ctx.fillStyle = "navy"
  ctx.fillRect(0, 0, 400, 300)
  ctx.fillStyle = "white"
  ctx.font = "24px monospace"
  ctx.fillText("Made in sdev!", 100, 150)
}
```

## Leaflet Map Integration

```
// Initialize a Leaflet map
forge map be js L.map("map-container").setView([51.505, -0.09], 13

js L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
  attribution: "© OpenStreetMap contributors"
})).addTo(map)

// Add markers
conjure addMarker(lat, lng, label) ::
  js L.marker([lat, lng]).addTo(map).bindPopup(label)
;;

addMarker(51.505, -0.09, "London, UK")
addMarker(48.8566, 2.3522, "Paris, France")
addMarker(52.52, 13.405, "Berlin, Germany")
```

# Complete Reference Card

## Keywords

| Keyword | Purpose | |---------|---------| | `forge` | Declare a variable | | `be` | Assign a value | | `conjure` | Declare a function | | `yield` | Return a value from a function | | `ponder` | `if` statement | | `otherwise` | `else` clause | | `cycle` | `while` loop | | `iterate` | `for-each` loop (iterate x through list) | | `through` | Used with iterate | | `within` | `for-in` loop (within x be list) | | `yeet` | `break` — exit loop | | `skip` | `continue` — skip iteration | | `attempt` | `try` block | | `rescue` | `catch` block | | `yep` | Boolean `true` | | `nope` | Boolean `false` | | `void` | Null / absent value | | `also` | Logical AND | | `either` | Logical OR | | `isnt` | Logical NOT | | `equals` | Equality comparison | | `differs` | Inequality comparison | | `essence` | Define a class | | `extend` | Class inheritance | | `self` | Instance reference inside class methods | | `super` | Parent class reference | | `new` | Create class instance | | `summon` | Import from Gist | | `async` | Mark function as async | | `await` | Wait for async operation | | `spawn` | Run function concurrently |

## Special Symbols

| Symbol | Purpose | |--------|---------| | `::` | Start a block | | `;;` | End a block | | `->` | Lambda arrow | | `\|>` | Pipe operator | | `~` | Ternary condition operator | | `:` | Ternary else / dict key-value separator | | `//` | Line comment | | `#`

| Line comment (Python style) | | ^ | Power operator | | <> | Inequality (alternative to differs) |

## Built-in Constants

| Constant | Value |
|---|---|
| PI | 3.141592653589793 |
| TAU | 6.283185307179586 |
| E | 2.718281828459045 |
| INFINITY | Infinity |

## Quick Reference Examples

```
// ── Variables ──
forge x be 42
forge name be "Alice"
forge active be yep

// ── Arithmetic ──
speak(2 ^ 10)     // 1024
speak(10 % 3)     // 1
speak(-5 + 8)     // 3

// ── String ──
speak("Hello" + " " + "World")
speak(upper("hello"))        // HELLO
speak(measure("hello"))      // 5

// ── List ──
forge nums be [1, 2, 3, 4, 5]
gather(nums, 6)              // [1,2,3,4,5,6]
speak(nums[-1])              // 6
speak(sort(nums))            // [1,2,3,4,5,6]

// ── Conditionals ──
ponder x &gt; 0 ::
  speak("positive")
;; otherwise ::
```

```
  speak("non-positive")
;;

// ── Ternary ──
forge label be x &gt; 0 ~ "pos" : "neg"

// ── Loop ──
within i be sequence(5) :: speak(i) ;;  // 0 1 2 3 4
cycle x &gt; 0 :: x be x - 1 ;;

// ── Function ──
conjure add(a, b) :: yield a + b ;;
forge double be x -&gt; x * 2

// ── Pipe ──
[1,2,3,4,5] |&gt; each(x -&gt; x * 2) |&gt; sift(x -&gt; x &gt; 4)
// [6, 8, 10]

// ── OOP ──
essence Dog ::
  conjure init(self, name) :: self.name be name ;;
  conjure bark(self) :: speak(self.name + ": WOOF!") ;;
;;
forge rex be new Dog("Rex")
rex.bark()

// ── Error Handling ──
attempt ::
  forge n be morph("not a number", "number")
;; rescue err ::
  speak("Error:", err)
;;

// ── Higher-order ──
forge evens be sift(sequence(10), x -&gt; x % 2 equals 0)
forge sum be fold(evens, 0, (a, b) -&gt; a + b)
speak(sum)   // 20

// ── Collections ──
```

```
forge s be Set()
s.add(1) s.add(2) s.add(2)
speak(s.values())   // [1, 2]

forge q be Queue()
q.enqueue("a") q.enqueue("b")
speak(q.dequeue())   // a

// ── Matrix ──
forge a be [[1,2],[3,4]]
forge b be [[5,6],[7,8]]
speak(matmul(a, b))   // [[19,22],[43,50]]

// ── Graphics ──
canvas(400, 400)
clear("#111")
fill(hue(120))
circle(200, 200, 100)

// ── Turtle ──
turtle()
within i be sequence(4) ::
  forward(100)
  right(90)
;;
```

*This documentation covers sdev version 1.x. For the latest updates and additional examples, visit the sdev IDE.*